QuADMESH+: A Quadrangular ADvanced Mesh Generator for Hydrodynamic Models

A Thesis

Presented in Partial Fulfillment of the Requirements for the Degree Master of Science in the Graduate School of The Ohio State University

By

Dominik D. Mattioli, B.S.

Graduate Program in Civil Engineering

The Ohio State University

2017

Master's Examination Committee:

Ethan J. Kubatko, Advisor

Alan Zundel

Gil Bohrer

Abdollah Shafieezadeh

© Copyright by

Dominik D. Mattioli

2017

Abstract

The purpose of this research is to provide a two-dimensional quadrangular mesh-generating tool for modeling shallow water flow within a Discontinuous Galerkin finite element framework – a setting where meshes of triangular or quadrilateral composition are both easily accommodated and demonstrably advantageous [32]. Prior research and development via ADMESH+ [8] shows that automatic mesh generators can produce high-quality meshes with an appealing gradation of edge size given a minimal set of input, e.g., minimum element size, maximum element size, and domain shoreline and bathymetry. This research aims to expand upon the accomplishments of ADMESH+ by developing a novel method (QuADMESH+) for converting high quality triangular meshes into quadrangular meshes that retain the geometric and aesthetic facets of the original mesh. Techniques employed in our approach include: triangular-mesh generation techniques used in ADMESH+; a heuristic-based algorithm for matching two triangular elements together to form quadrilaterals; a routine for removing any unmatched triangular elements; another heuristic-based algorithm employing common topological operations to improve the quality of quadrilateral elements in the resultant mesh; and a finite element based smoothing algorithm [2].

This is dedicated to my loving family.

Acknowledgments

I could not have conducted this research without the assistance of my wise and benevolent mentor, Professor Ethan Kubatko. I have learned and matured more than I could have ever hoped while under his tutelage. I am incredibly grateful for the opportunity and guidance that he and Dr. Alan Zundel have given me.

I would also like to thank my family and friends for all of their love and company whenever I need it.

Vita

2011 ..... McHenry High School - West Campus

2015 ..... B.S. Civil Engineering,
The Ohio State University

2015-present ..... Graduate Research Assistant,
Department of Civil Engineering,
The Ohio State University

Fields of Study

Major Field: Civil Engineering

Area of Specialization: Environmental and Water Resources Engineering

Table of Contents

Page

Abstract ii

Dedication iii

Acknowledgments iv

Vita v

List of Tables ix

List of Figures x

1. Introduction 1

    1.1 Motivation 6
    1.2 Objective 8
    1.3 Prior Work 10
    1.4 Thesis Outline 12

2. ADMESH+: An advanced, automatic unstructured mesh generator for shallow water models 13

    2.1 Methods 14
        2.1.1 Domain Representation 15
        2.1.2 Mesh Size Function 16
        2.1.3 Spring-Based Force Equilibrium Smoothing 18
    2.2 Implications for QuADMESH+ 19

3. Building an Indirect Method 20

3.1 Preliminary Research 20
3.1.1 The Greedy Approach 20
3.1.2 The Perfect Matching Approach 24
3.1.3 The Advancing Front Approach 25
3.1.4 The Distance Approach 27
3.1.5 The Hybrid Weighted Approach 30
3.2 The Mesh Layers Approach 32
3.2.1 Defining Mesh Layers 35
3.2.2 Using Mesh Layers 40

4. Matching Algorithms 43

4.1 Interior Layer Matchings 44
4.1.1 Choosing a Candidate for Matching 46
4.1.2 Completing a Matching 47
4.2 Boundary Layer Matchings 52
4.2.1 Pre-Processing Boundary Triangles 52
4.2.2 Boundary Matching Algorithm 53

5. Post-Processing By Layer 56

5.1 Quad Quality Improvement Operations 56
5.2 Procedure 58
5.3 Smoothing 63

6. Analysis of Test Cases 64

6.1 Test Case 1: An Artificial Domain 64
6.2 Test Case 2: Wetting and Drying 67
6.3 Test Case 3: Italy 69
6.4 Test Case 4: Lake Erie 73
6.5 Test Case 5: Delaware Bay 75
6.6 Mixed-Element Test Cases 77

7. Conclusions 89

7.1 Future Work 90

Appendices 92

A. Statistics for Test Case Meshes 92

Bibliography 94

List of Tables

Table **Page**

A.1 Computational time of each algorithm step for each test case domain. 92

A.2 Mesh details for each test case domain. 93

A.3 Incidence of IE_L attachments for high and low valence vertices in various mesh test cases. 93

List of Figures

Figure **Page**

1.1 A structured mesh failing to resolve the geometry of a complicated boundary [21]. 2

1.2 An unstructured mesh capturing the domain boundaries of the Louisiana Gulf Coast. The complex geometry of the coastline and the bathymetrical features of this hydrodynamic domain are well represented by an unstructured mesh [21]. 2

1.3 A proposed mixed-element mesh concept. *Right* is a mesh for the Western North Atlantic Tidal domain (WNAT), *top left* is a zoom-in of complicated coastline and bathymetry where triangular elements could be used, and *bottom left* is a zoom-in of open-ocean with relatively-homogeneous edge lengths and bathymetry where quadrilateral elements could be used [21]. 4

1.4 Simple triangular and quadrangular mesh domains. 5

1.5 High-order DG results for simple triangular and quadrangular meshes for an example domain [32]. 5

1.6 Error convergence of high-order DG approximations over simple triangular and quadrangular meshes [32]. 6

1.7 An ADMESH+ automatically generated mesh of the UK and Ireland (with color-mapped bathymetric depths) [8]. 8

1.8 Two distinct indirect methods for converting triangles into quads [17]. 9

2.1 Cublic spline representation of Maumee Bay, Lake Erie. *Bottom left* is a zoomed view of Box A, *bottom right* is the zoomed view of Box B. The *dashed line* is the piecewise boundary and the *blue line* is the cubic spline approximation [8]. 14

2.2 Distance function for the WNAT domain [8]. 15

2.3 An example of the medial axis, the signed distance function $\phi(\mathbf{x})$, and the local feature size $lfs(\mathbf{x})$ [8]. 16

2.4 Identical meshes with varying curvature resolution [8]. *Left to right* shows increasing resolution at the boundary domain. 16

2.5 Mesh size function $h(\mathbf{x})$ for the WNAT domain [8]. 18

2.6 *Left* is a Delaunay Triangulation before static force equilibrium smoothing, *right* is after the spring-based static equilibrium smoothing [8]. . 19

3.1 *Left* is the original triangular mesh, *right* is the quad-dominant mesh resulting from the Greedy Approach (255 *triangles*, 1,087 quads, avg. element quality of 0.8095 after smoothing). 21

3.2 Example of the Edge Swap operation on two *triangles* sharing one vertex. 22

3.3 Example of the Vertex Duplication operation on two *triangles* sharing one vertex. 22

3.4 Converting a *triangle* into a quad by adding a new point to the domain boundary. 23

3.5 Removing a *triangle* by collapsing an edge of the domain boundary. . 23

3.6 Walking a *triangle* toward another via edge flipping. 23

3.7 Two examples of meshes that do not have a perfect matching [28]. . . 24

3.8 A direct method for creating quadrilaterals via advancing front technique. Steps *0* - *3* show the direct creation of quadrilateral elements as the front advances from the domain boundary toward the interior. 26

3.9 An advancing front technique that uses the vertices and edges of a background triangular mesh to create quadrilateral elements along the front [26]. 26

3.10 The Signed Distance Function $\phi(\mathbf{x})$ applied to the domain of Lake Erie. The contours show interpolated distance of every point from the *red* domain boundaries. 27

3.11 Triangular mesh of Lake Erie with the Signed Distance Function $\phi(\mathbf{x})$. 28

3.12 The Greedy Approach matching algorithm applied to Lake Erie. Result: 11,128 quads, 2,654 *triangles*, avg. quality of 0.8428. 29

3.13 The Distance Approach matching algorithm applied to Lake Erie. Result: 11,749 quads, 1,952 *triangles*, avg. quality of .7550. 29

3.14 Applying a range of Weighting Factor values to the Hybrid Weighted Approach. *Top* is the number of triangles on the mesh boundary, *middle* is the total number of triangles remaining in the mesh, and *bottom* is the mean element quality. A Weighting Factor of 0.0 weights only distance while a Weighting Factor of 1.0 weights only quality. . . 31

3.15 Forming quadrilaterals along a triangle strip example. *Arrows* show a clear topological direction of the adjacent triangles, which is a key characteristic of triangle strips. 33

3.16 An example set of triangles near the boundary of a domain. $\phi(\mathbf{x})$ is the euclidean distance of $\mathbf{x}$ to the nearest domain boundary. *Arrows* indicate the "direction" of the strip of triangles along the boundary. . 34

3.17 Example domain (step *0*) discretized into mesh layers (blue, green, and yellow) in steps *1 - 9*. The *Outer Vertices* and *Inner Vertices* are red and orange of each layer, respectively. 36

3.18 Example domain's mesh layers (step *0*) discretized into *Outer Elements* and *Inner Elements* in steps *1 - 9*. *Outer vertices* and *inner vertices* of each layer are also colored red and orange, respectively. 38

3.19 An example mesh domain with a flagged edge. The edge defined by two inner vertices within Layer 3 (step *1*) of the example mesh domain becomes a flagged edge in step *2*. 39

4.1 Example domain's mesh layers (step *0*) and a hypothetical intra-layer matching with respect to the flagged edges (steps *1-2*). 45

4.2 Example domain's interior matching process, steps *0* - *8*. Blue, green, and yellow are the domain's mesh layers, orange are the IE_L of a given layer, purple are flagged edges, red indicates a triangle selected for matching (T_1), gray indicates the completion of a matching (T_1 matched with it's neighbor T_2). 50

4.3 Isolated triangle removal process, steps *0* - *4*. Green and yellow are the domain's unmatched mesh layers, gray are matched pairs of triangles, i.e., quads. 51

4.4 An example of the commonly-used edge flip operation [24]. 52

4.5 Example domain's boundary matching process, steps *0* - *3*. Blue and green are the domain's mesh layers, orange are the IE_L of the boundary layer, cyan are the edge-flipped RE_L and their neighbors on the boundary, and gray are matched pairs of triangles, i.e., quads. 55

5.1 Formation of a concave quad (a to b) and removal of a concave quad via Doublet Collapse (b to c). 57

5.2 Removing a quad via Quad Vertex Merge. 58

5.3 Two possibilities for the Quad Vertex Merge Operation on an example sub-domain. *Red lines with arrows* in *a* indicate a possible Quad Vertex Merge and the location of the merged vertices. 59

5.4 An example operation that removes low-quality quads formed on the domain boundary by the matching of two OE_L. These low-quality quads are removed from the mesh and their neighbors are adjusted to maintain the original domain boundary definition. 61

5.5 A finite element method implementation of element smoothing. *Dotted* lines indicate the ideal shape of a particular element E and, consequently, the ideal location of Vertex A for each respective element, i.e., A_1, A_2, A_3, A_4, and A_4. 62

6.1 Mesh Layers for Test Case 1: An Artificial Domain. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh. 66

6.2 Mesh Layers for Test Case 2: Wetting and Drying. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh. 68

6.3 Mesh Layers for Test Case 3: Italy. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh. 70

6.4 Quad quality colormap and histogram for Test Case 3: Italy. Note that the minimum element quality ($q = 0.02$) of the mesh is an outlier. 71

6.5 Boxplots of the element quality distributions for each layer of Test Case 3: Italy. The red horizontal line is the median element quality of each layer, the blue box indicates the first (*bottom*) and third (*third*) quartile for element quality of each layer, the *dashed* lines indicate the min. and max element quality of each layer (not including outlier elements), and the red markers indicate outlier element qualities of each layer. 72

6.6 Mesh Layers for Test Case 4: Lake Erie. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh. 74

6.7 Mesh Layers for Test Case 5: Delaware Bay. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh. 76

6.8 The triangular mesh layers (via ADMESH+) for the WNAT domain. The mesh contains 246,186 triangles defined by 127,572 vertices, the average element quality is 0.86, and the minimum element quality is 0.21. 78

6.9 Mesh layers of the triangles in WNAT selected for quadrangulation. These triangles were selected because geometry exceeded a target edge size. 79

6.10 The mixed-element mesh layers (via ADMESH+) for the WNAT domain. The triangle-dominant mesh contains 243,416 elements defined by 127,494 vertices, the average element quality is 0.89, and the minimum element quality is 0.21. 80

6.11 A zoomed view of Gulf Coast of the WNAT mixed-element Mesh Layers. 81

6.12 Mapping the element quality for the WNAT mixed-element mesh. . . 82

6.13 A histogram element quality for the WNAT mixed-element mesh. Note that the colors correspond to the colorbar and the colormap in Figure 6.12. 83

6.14 The mixed-element mesh layers (via ADMESH+) for the Delaware Bay domain. The triangle-dominant mesh contains 25,600 elements defined by 14,430 vertices, the average element quality is 0.83, and the minimum element quality is 0.34. Identifying triangles in this mesh for quadrangulation was performed by selecting all triangles of a specific edge size or located beyond a specific distance from the domain boundaries. 85

6.15 A zoomed view of the Delaware Bay mixed-element Mesh Layers. . . 86

6.16 The mixed-element mesh layers (via ADMESH+) for the Lake Erie domain. The triangle-dominant mesh contains 24,446 elements defined by 13,257 vertices, the average element quality is 0.79, and the minimum element quality is 0.25. Identifying triangles in this mesh for quadrangulation was performed by selecting triangles within specific mesh layers. 87

6.17 The mixed-element mesh layers (via ADMESH+) for the Delaware Bay domain. The triangle-dominant mesh contains 23,648 elements defined by 12,990 vertices, the average element quality is 0.81, and the minimum element quality is 0.09. Identifying triangles in this mesh for quadrangulation was performed by selecting triangles of specific edge sizes. 88

Chapter 1: Introduction

An important first step in obtaining an approximate solution to some computational model is discretizing the target domain into a discrete number of smaller polygonal elements. These polygonal elements collectively comprise what is known as a "mesh" and they provide a platform to numerical methods that solve systems of differential equations. The process of constructing a mesh is expansive and there are many topological characteristics of meshes that may affect the difficulty of their generation. The foremost characteristic of a mesh that determines generation complexity is whether its topology is structured – node/vertex distribution is of *equal*-density throughout the domain – or unstructured – node/vertex distribution is of *unequal*-density throughout the domain. Two-dimensional (2D) structured meshes typically utilize quadrilateral elements (quads) because the resultant aesthetic resembles a grid, which is relatively simple to produce. This topology provides the basis for some of the most notable advantages of structured meshes, which include simple generation methods, efficient discretization of space, a predictable (gridded) topology, and easy implementation within finite difference method frameworks. Typically, the advantageous aspects of one mesh structure are the disadvantageous aspects of the other, and vice versa. Therefore, when deciding on the structure of a mesh, it is important to consider which physical phenomena is being modeled.

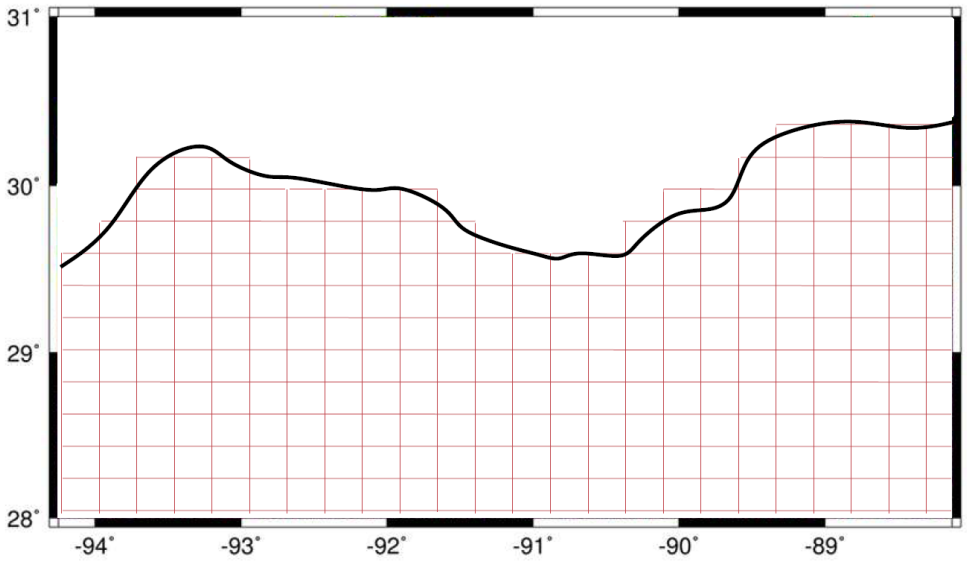


Figure 1.1: A structured mesh failing to resolve the geometry of a complicated boundary [21].

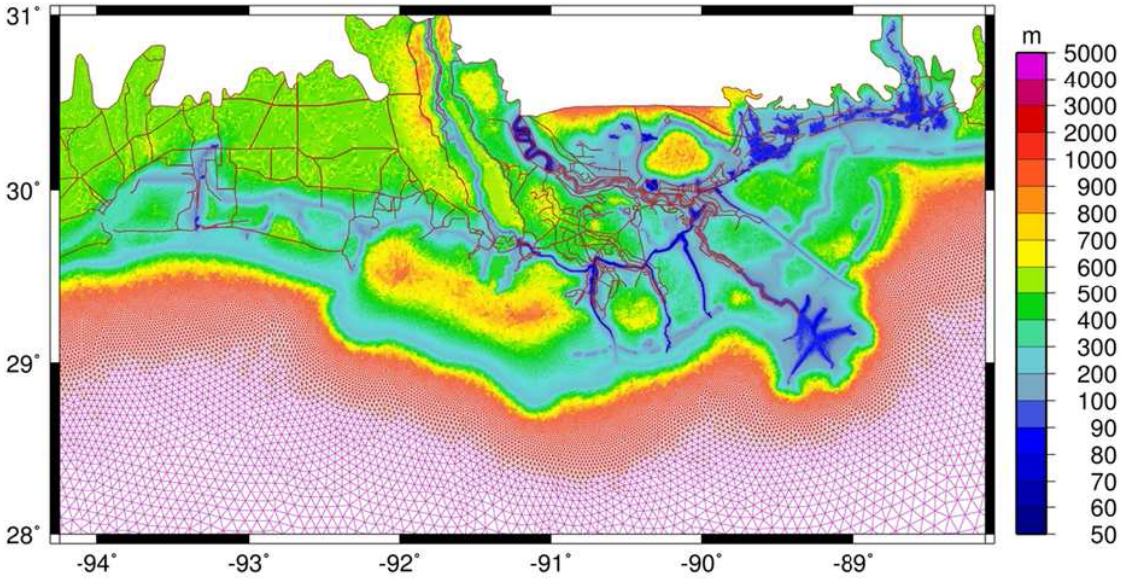


Figure 1.2: An unstructured mesh capturing the domain boundaries of the Louisiana Gulf Coast. The complex geometry of the coastline and the bathymetrical features of this hydrodynamic domain are well represented by an unstructured mesh [21].

In the context of hydrodynamic problems – transport, surface flow, etc., – there is often a need for greater resolution around domain (coastal) boundaries and areas of acute bathymetric gradients or heterogeneous/anisotropic flow properties. Due to their rigidity, structured meshes cannot always accommodate these intricacies without increasing the mesh resolution and consequently decreasing computational efficiency. For example, Figure 1.1 shows a structure mesh insufficiently capturing the boundary of a domain.

Alternatively, unstructured meshes relax requirements vertex density throughout some domain, which allows the mesh generator to place more nodes wherever higher resolution is necessary and fewer nodes where it is not. The resultant unstructured mesh contains polygons of varying edge lengths and element sizes. This feature enables a more accurate rendering of the domain boundaries and the aforementioned topographic features while also allowing for increased computational efficiency by having larger elements wherever higher resolution is unnecessary; see, for example, Figure 1.2. The advantage in refinement makes 2D unstructured meshes more congruent with the features of hydrodynamic domains and other finite element method problems. However, generating unstructured meshes is an involved process and the resulting meshes often cause decreased computational efficiency for numerical models.

As computational capabilities and numerical techniques continue to evolve, so too do the desires to model increasingly complex processes. Such models may require the utilization of fine meshes in order to attain reasonable accuracy. Contemporary methods for mesh generation have shifted toward automation, i.e., minimal requirements for user input and interaction. The purpose of this research is to explore new and advanced tools for automatic unstructured mesh generation. Conroy et al. [9, 8, 31]

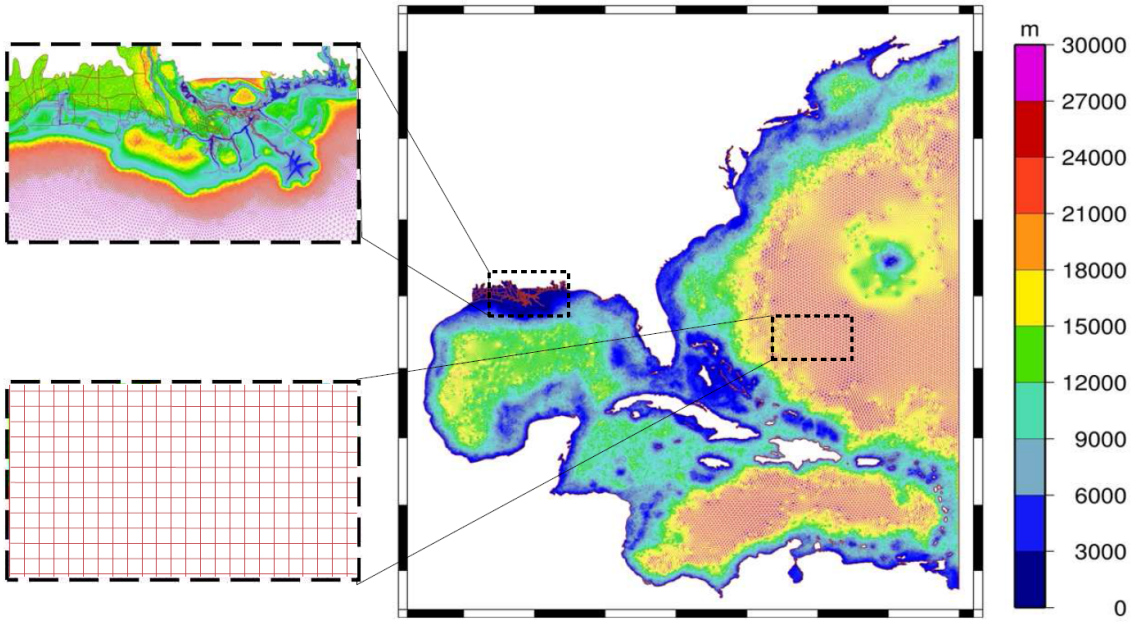


Figure 1.3: A proposed mixed-element mesh concept. *Right* is a mesh for the Western North Atlantic Tidal domain (WNAT), *top left* is a zoom-in of complicated coastline and bathymetry where triangular elements could be used, and *bottom left* is a zoom-in of open-ocean with relatively-homogeneous edge lengths and bathymetry where quadrilateral elements could be used [21].

have shown the viability and convenience of employing tools for automatic generation of high-order triangular meshes via ADMESH+ in the context of coastal ocean modeling and other hydrodynamic phenomena.

Some research efforts have explored the idea of using mixed-element meshes (triangular and quadrilateral elements) in an effort to increase efficiency relative to triangular meshes [21, 32]. Computational properties of quad may be likened to structured meshes and the computational properties of triangular elements to unstructured meshes. When used strategically, the implementation of mixed-element meshes can

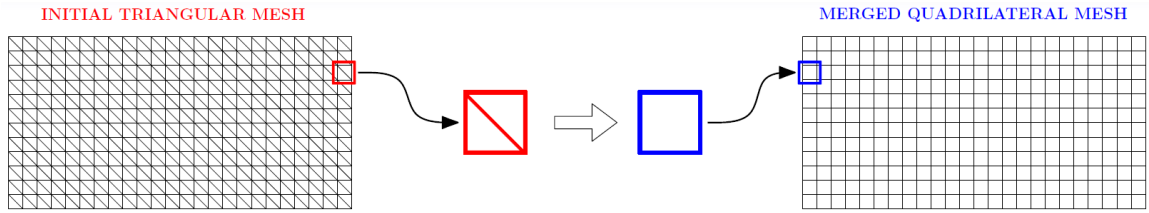


Figure 1.4: Simple triangular and quadrangular mesh domains.

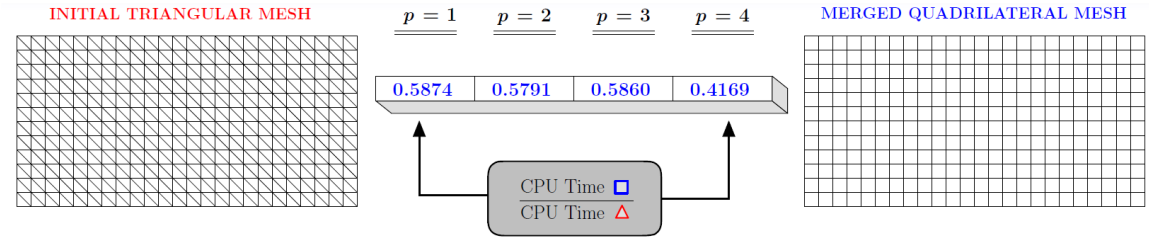


Figure 1.5: High-order DG results for simple triangular and quadrangular meshes for an example domain [32].

blend the advantages of both mesh types while reducing the computational disadvantages individually associated with each mesh type. Figure 1.3 shows the mesh a hydrodynamic domain with edge lengths that vary considerably, an ideal example for utilizing mixed-element meshes. This research aims to build on the techniques of ADMESH+ by creating a quadrangular mesh capability for ADMESH+, which would allow the existing software to handle both quadrangular and mixed-element meshes.

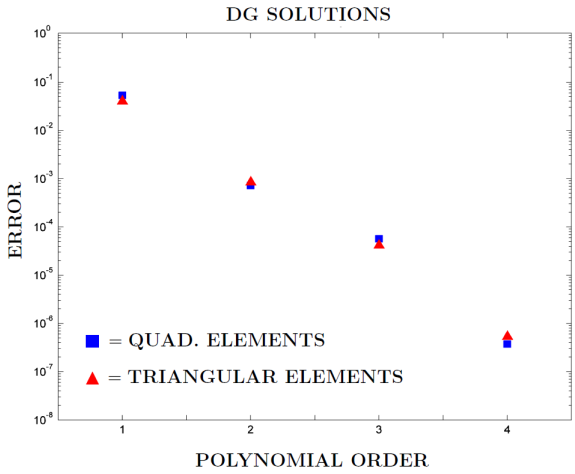


Figure 1.6: Error convergence of high-order DG approximations over simple triangular and quadrangular meshes [32].

1.1 Motivation

Indeed, using quads instead of triangles can be advantageous, first and foremost because a quadrangular mesh could use as few as half the elements as a triangular mesh of the same domain [32]. Using high-order Discontinuous Galerkin (DG) schemes, Wirasaet et al. approximate solutions to various hydrodynamic physical phenomena over simple example domains like those seen in Figure 1.4. The authors then compare the numerical accuracy and computational efficiency of triangular, quadrangular, and mixed-element meshes over the same domain and show that meshes using quads achieve considerably faster results than the meshes using triangles (Figure 1.5) while also retaining a similar accuracy and rate of convergence in error (Figure 1.6). These results indicate that quads can be more computationally

efficient than triangles. It is also worth noting that the authors achieve similar results by using mixed-element meshes.

The efficiency of quads notwithstanding, it is important to recognize that a notable characteristic of both meshes used by Wirasaet et al. is that the elements used are all of high-quality (the triangular elements resemble equilateral triangles and the quads resemble perfect squares). For the purposes of this research, quality is a reference to the equiangular skew of an element and is defined as

$$q = 1 - \max\left(\frac{\theta_{max} - \theta_e}{180 - \theta_e}, \frac{\theta_e - \theta_{min}}{\theta_e}\right) \tag{1.1}$$

where θ_e is the ideal angle for an element type, i.e., 60 degrees for a triangle and 90 degrees for a quad, θ_{max} is the largest angle in the element, θ_{min} is the smallest angle in the element, and with $q = 0.0$ meaning completely degenerate (high skewness) and $q = 1.0$ meaning perfect quality (no skewness).

Numerical accuracy and stability of computational models depend on the quality of the elements in the mesh. Therefore, it is prudent to ensure that quadrangular meshes are of high-quality in order to best replicate the increased efficiency of quads. Moreover, any mesh generator capable of producing high-quality meshes should do so without being at the expense of efficiency. ADMESH+ currently produces high-quality triangular meshes, however its usefulness would expand if it could additionally produce high-quality quadrangular meshes in an efficient manner. An automatic mesh generator capable of producing meshes using either element type is undoubtedly a valuable tool, which is additional motivation for this research.

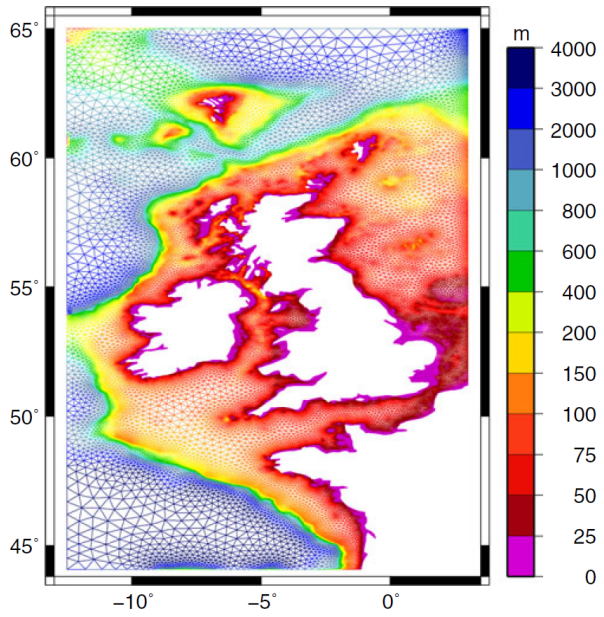


Figure 1.7: An ADMESH+ automatically generated mesh of the UK and Ireland (with color-mapped bathymetric depths) [8].

1.2 Objective

On a reasonable time scale, ADMESH+ produces high-quality 2D triangular meshes for shallow water models with minimal inputs, such as the points defining the boundary domain, the domain bathymetry, and the minimum and maximum allowable edge lengths. Given appropriate inputs, the resulting mesh adequately resolves both the geometry and the flow characteristics of the domain, provides properly-graded refinement where necessary (i.e., near complex boundary or bathymetric topography), and maintains low vertex valence throughout the mesh (where valence is defined as the number of edges attached to a vertex and ideal valence being 6 for triangular meshes). Figure 1.4 shows an example of ADMESH+ and its features [8].

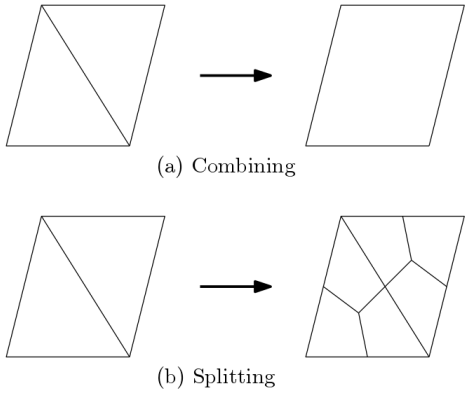


Figure 1.8: Two distinct indirect methods for converting triangles into quads [17].

This research theorizes that a quadrangular mesh will be of high-quality if it is created with similar resolution and topological characteristics as triangular meshes generated by ADMESH+. Combining the existing functionality of ADMESH+ with a novel approach to building a quadrangular advanced mesh (QuADMESH+) should provide hydrodynamic modelers with a comprehensive tool for generating high-quality meshes and the opportunity to capitalize on the increased numerical efficiency of quads.

Methods for creating unstructured quadrangular meshes are generally categorized into two distinct approaches: *indirect* methods and *direct* methods [25]. *Indirect* methods require the creation of a triangular mesh [8, 18] for the domain and then manipulation of that topology to create quads via triangle matching/combining or splitting [6, 28, 20]. An example of two such methods are seen in Figure 1.8. Conversely, direct methods create a quandrangulation of the domain directly, without the intermediate step of creating a triangulation first. These methods generally involve quad-trees, partition of the domain, advancing fronts, or paving techniques

[1, 5]. Many of these methods have proven to be effective; however, they are sometimes viewed as non-robust [19]. Additionally, some indirect methods exist that augment the indirect method of triangle-combining with direct methods like advancing front techniques in order to create more robust hybrid methods for generating quads [26, 34].

This research makes use of the existing framework of ADMESH+ to conveniently create triangular meshes with ideal topological characteristics and then utilizes an indirect method to create a quadrangular mesh of similarly high quality. While there is no shortage of successful indirect methods, many of these methods can be complex algorithms. This research investigates a more approachable method to indirect creation of quadrangular meshes.

1.3 Prior Work

Research and development of mesh generators – especially in the context of hydrodynamic domains – is a continually evolving field of study. Historically, triangular mesh generators have been the primary focus of automatic mesh generation research. Many widely used generators are not full automated and require some level of user-interaction in order to create a mesh. One such generator is the Surface Water Modeling System (SMS) [35]. Created by Zundel et al., SMS allows for the input of shoreline and bathymetry data before requiring some manual inputs such as node spacing. A mesh is then generated (sometimes in sections) and various element quality enhancement operations are applied. Another 2D unstructured triangular mesh generator called BatTri relies heavily on mesh refinement in order to obtain a satisfactory mesh

[4]. Lastly, one other generation technique employs location truncation error analysis (LTEA) to create a graded mesh with threshold element sizes for specific regions of the domain. These maximum mesh element sizes are based on *a posteriori* error calculations with respect to some initial triangular mesh. Hagen et al. [13, 14, 15] developed a well known mesh generator that uses this method. Within the mesh generation community there is still debate regarding the advantages of quadrangular meshes. Nonetheless, efforts to create advanced algorithms for building these types of meshes deserve attention.

The main inspiration for this research comes from a variety of indirect methods for quadrangulation. Remacle et al. present an indirect approach that applies a perfect matching algorithm (first created by Edmonds et al. [11]) to an existing triangular mesh. The resultant mesh is often composed solely of quads unless there is no so-called perfect matching for the given mesh [28]. Worst-case scenario complexity for this algorithm – while constantly improving – is still non-ideal for large meshes. Other indirect methods employ sophisticated sequential decision-making heuristics [16] that predict which matchings of triangles will result in a high quality quad-dominant (mostly composed of quadrilaterals) or fully quadrangular mesh. Lastly, some approaches combine elements from various existing direct and indirect methods. Some algorithms generate triangular meshes first and then apply an advancing front technique to form quadrilaterals from the existing triangles [22, 26], while some algorithms simultaneously convert triangles into quads as the triangular mesh is being generated [17]. Review of this literature led to the investigation of other techniques unique to this research that are discussed further in Chapter 3.

1.4 Thesis Outline

The remainder of this thesis will describe the QuADMESH+ concept for indirectly creating quadrangular meshes. The steps of that concept are as follows: First, discretization of the domain with a triangular mesh via ADMESH+ is explained and the fundamentals of that algorithm are summarized in Chapter 2. Then Chapter 3 expands the concept of triangular strips into a method for discretizing all triangles of the mesh into distinct groupings of strips, known as *Mesh Layers*. Next, Chapter 4 explains that the QuADMESH+ algorithm begins a matching process within the innermost layers of the mesh before advancing outwards toward the domain boundaries. Additionally, a set of heuristics used for creating matchings within the layers is defined. Finally, Chapter 5 describes a layered approach to utilizing some basic post-processing topological improvement operators and Chapter 6 reveals the results for some example mesh test cases. Chapter 7 closes the thesis with an important discussion of conclusions and necessary future work for QuADMESH+.

Chapter 2: ADMESH+: An advanced, automatic unstructured mesh generator for shallow water models

Developed by Conroy et al. in the Computational Hydrodynamics and Informatics Laboratory (CHIL) at The Ohio State University in 2011, ADMESH+ is a software tool for automatically generating high-quality triangular meshes [8]. ADMESH+ requires a minimal set of input from the user, i.e., target minimum and maximum element sizes and points defining the domain boundary and bathymetry. The software employs three notable advanced methods:

1. Signed distance function
2. Mesh-size function
3. Spring-based force equilibrium smoother

ADMESH+ owes much of its inspiration to the open-source MATLAB mesh generator called Distmesh. Developed by Persson et al. [27], Distmesh uses similar methods as mentioned above to generate non-uniform element sizes and densities for a given mesh domain. What follows is a brief summary of each of these methods and a discussion on the implications that the resulting mesh has on the remainder of the QuADMESH+ algorithm.

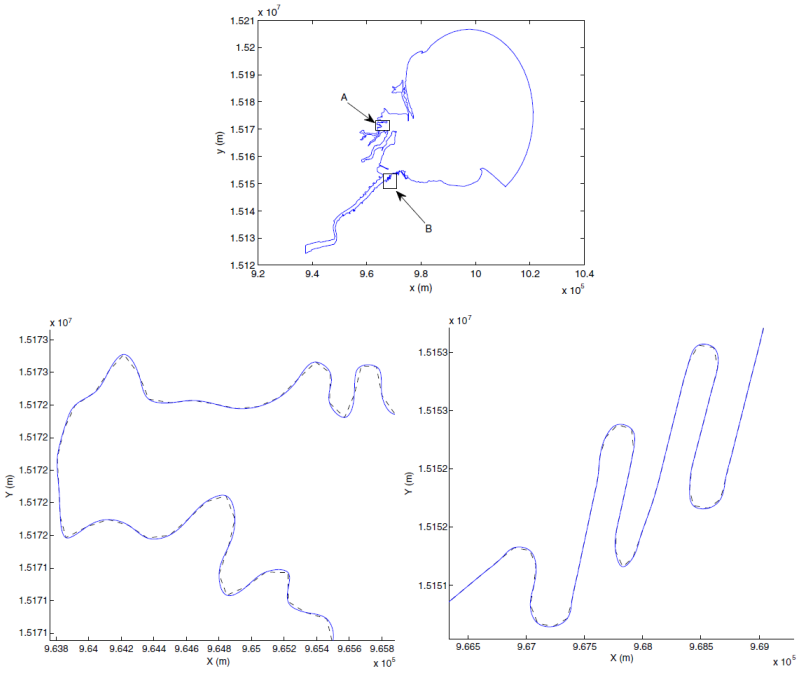


Figure 2.1: Cublic spline representation of Maumee Bay, Lake Erie. *Bottom left* is a zoomed view of Box A, *bottom right* is the zoomed view of Box B. The *dashed line* is the piecewise boundary and the *blue line* is the cubic spline approximation [8].

2.1 Methods

There are a few pre-processing operations performed by ADMESH+. Given a viable domain boundary input of ordered points, ADMESH+ processes the information by identifying the external and (any) internal boundaries and correcting them for any redundancies and inconsistent point-ordering (conventional ordering is counter-clockwise orientation). Additionally, approximation of the domain boundaries via cubic splines occurs (Figure 2.1) in order to resolved the boundaries of the mesh.

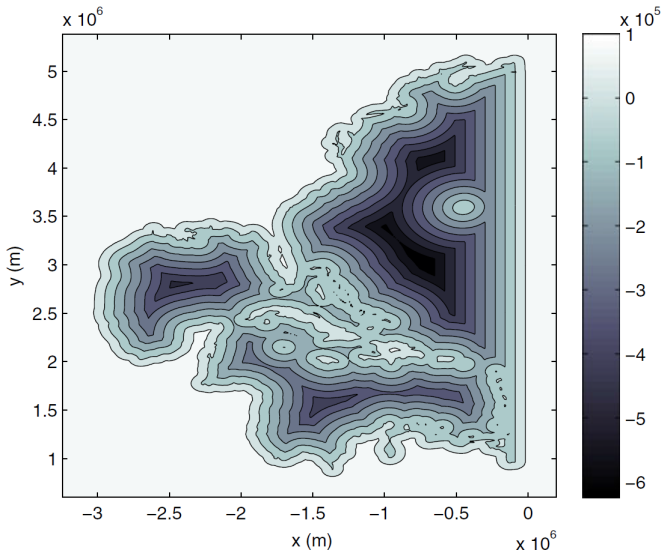


Figure 2.2: Distance function for the WNAT domain [8].

2.1.1 Domain Representation

Given the inputs for domain boundaries, the first step in the algorithm is to generate a signed distance function $\phi(\mathbf{x})$ that interpolates the distance of any coordinate $\mathbf{x} = [x, y]$ in the domain to the nearest boundary relative to a background grid (Figure 2.2). The "signed" aspect of the distance function indicates whether a point is within the domain $\phi < 0$ or outside the domain $\phi > 0$. The bathymetry of the domain is used to create the bathymetry function $B(\mathbf{x})$, where x is the interpolated bathymetric data relative to the same background grid.

Both of these functions provide inputs to the other two methods. For instance, the distance function $\phi < 0$ enables the computation of the medial axis for a mesh, which is the set of interior domain points that are equidistant to at least two boundary locations (Figure 2.3). Information provided by the medial axis serves as an input

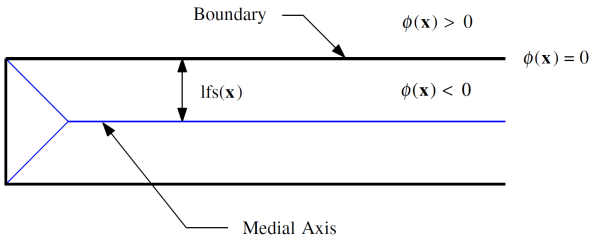


Figure 2.3: An example of the medial axis, the signed distance function $\phi(\mathbf{x})$, and the local feature size $lfs(\mathbf{x})$ [8].

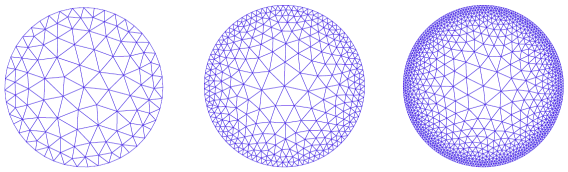


Figure 2.4: Identical meshes with varying curvature resolution [8]. *Left to right* shows increasing resolution at the boundary domain.

into the mesh size function, which provides less resolution at larger distances from the domain boundaries where the characteristics of flow may be more homogeneous than at smaller distances. Additionally, the bathymetry function $B(\mathbf{x})$ enables the mesh size function to account for the necessary resolution in areas with associated bathymetric gradients and the associated tidal wave lengths.

2.1.2 Mesh Size Function

The mesh size function $h(\mathbf{x})$ provides the user with five optional parameters for attaining sufficient resolution where needed. These parameters are dictated by the geometry, bathymetry/topography, and the domain's flow characteristics.

Unstructured meshes allow elements to accurately capture the intricacies of coastlines within a domain. Since the geography of the coastlines can range from monotonous to meandering, their curvature must be prescribed ample resolution. ADMESH+ addresses this concern with the application of a curvature function that feeds into the mesh size function. Depending on the curvature, this function determines the degree of resolution at the mesh boundary.

Other optional inputs to the mesh size function are the local feature size $h_{lfs}(\mathbf{x})$, bathymetric gradients $h_{bathymetry}(\mathbf{x})$, and tidal wavelengths $h_{tidal}(\mathbf{x})$. The local feature size (i.e., channels and tributaries) designates a minimum number of elements that must span half of the local feature. The latter two features employ unique quantifications of hydrodynamic phenomena that are not always accounted for in mesh generation. The bathymetric gradients are computed from the domain bathymetry input and the tidal wavelengths are derived from the equation for linear shallow water wave speeds $\mathrm{c} = \lambda/\mathrm{T} = \sqrt{gB}$, where T is the wave period, g is the gravity acceleration constant, B is the bathymetry, and C is the number of elements per wavelength (optional user input).

Lastly, minimum h_{min} and maximum h_{max} target element sizes constrain the mesh size function, which in its final form is written as follows:

$$h_{IC}(\mathbf{x}) = \min\left(h_{shore}(\mathbf{x}), h_{lfs}(\mathbf{x}), h_{bathymetry}(\mathbf{x}), h_{tide}(\mathbf{x})\right) \tag{2.1}$$

$$h_0(\mathbf{x}) = \max\left(h_{min}, h_{IC}(\mathbf{x})\right) \tag{2.2}$$

$$|\Delta h(\mathbf{x})| \leq g \approx G - 1 \tag{2.3}$$

The $h_0(\mathbf{x})$ variable is the resulting mesh size function and G is a gradation factor/maximum allowable ratio of neighboring element sizes. Next, $h_0(\mathbf{x})$ is used as an

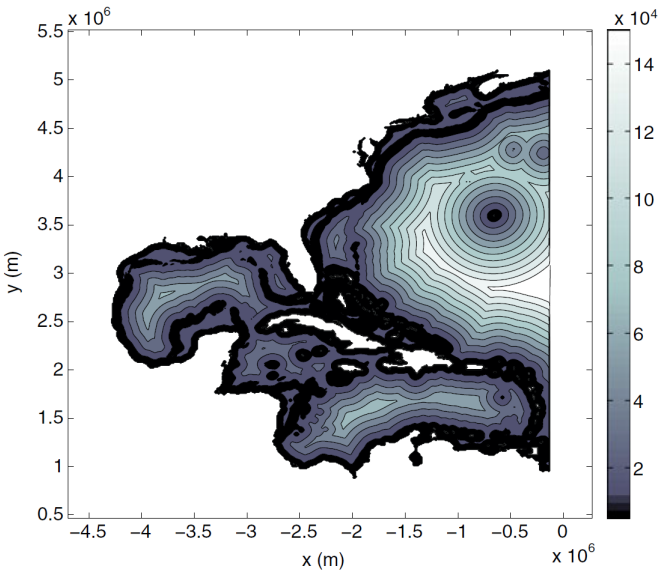


Figure 2.5: Mesh size function $h(\mathbf{x})$ for the WNAT domain [8].

initial condition for a partial differential equation, the solution to which yields the final mesh size function $h(\mathbf{x})$.

2.1.3 Spring-Based Force Equilibrium Smoothing

The final step is the placement of vertices and the formation of triangular elements from those vertices. The vertices are initially distributed throughout the domain according to a density proportional to $1/h(\mathbf{x}^2)$. Then, a Delaunay Triangulation is computed for the given graph, forming the triangular elements for the mesh. Finally, the edges of the mesh act are treated as springs, pushing the vertices toward a static force equilibrium as seen in Figure 2.6. The result is a high-quality triangular mesh.

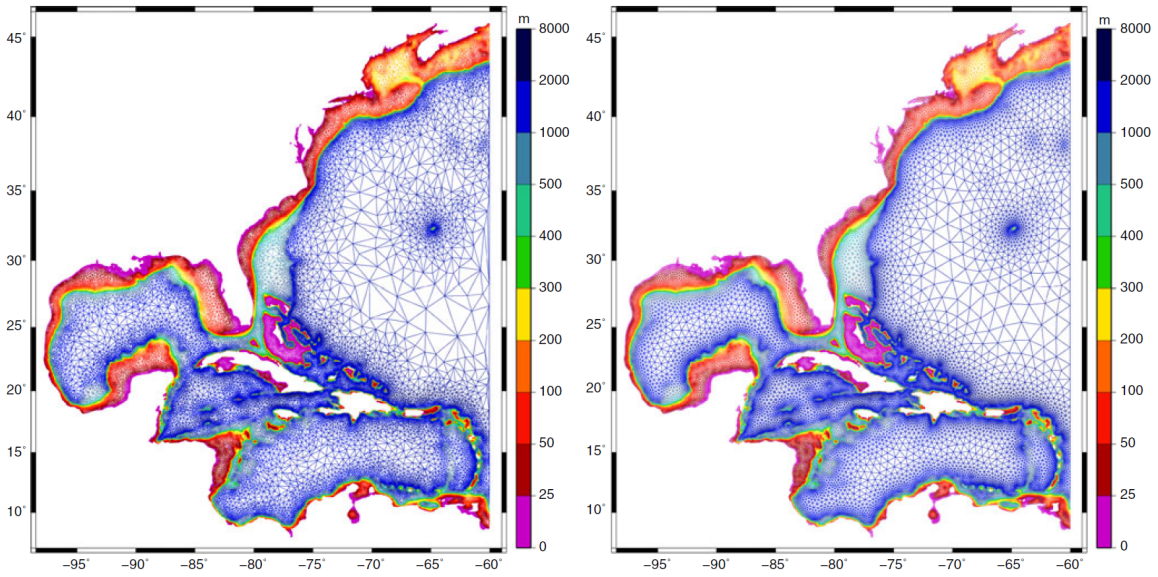


Figure 2.6: *Left* is a Delaunay Triangulation before static force equilibrium smoothing, *right* is after the spring-based static equilibrium smoothing [8].

2.2 Implications for QuADMESH+

A quadrangular mesh generated indirectly from an ADMESH+ generated triangular mesh conceivably should retain many of the aforementioned attributes of that original mesh, such as appropriate resolution wherever needed. Assuming minimal changes to the mesh sizes, the resultant quadrangulation should avoid many of the issues that are commonly associated with quadrangular meshes, such as element size gradation and adequate resolution of domain boundaries. Additionally, a high-quality ADMESH+ quadrangulation should yield considerably greater accuracy for numerical model approximations given the computational benefits of quadrilateral elements.

Chapter 3: Building an Indirect Method

3.1 Preliminary Research

With a triangular mesh generated via ADMESH+, the next step in the process is to apply an indirect approach for creating quadrilateral elements from the existing mesh. This research examined several approaches for indirectly creating quadrangular meshes. Analysis of each of these techniques brought additional knowledge and inspiration for the development of the final QuADMESH+ algorithm. Several of these methods are presented below, along with their strengths, weaknesses, and implications for the development of QuADMESH+.

3.1.1 The Greedy Approach

In the *Greedy Approach*, the first step is to create a list of every possible quad created by any two adjacent triangular neighbors within the mesh and the quality of that potential quad. This list of potential quads is then sorted by their respective element quality and the best potential quad is selected for creation. With the creation of this quad, any other potential quad within the list derived from one of the two triangles involved in the previous quad creation (merging) are removed. This process is then repeated until no potential quads remain in the list. This procedure

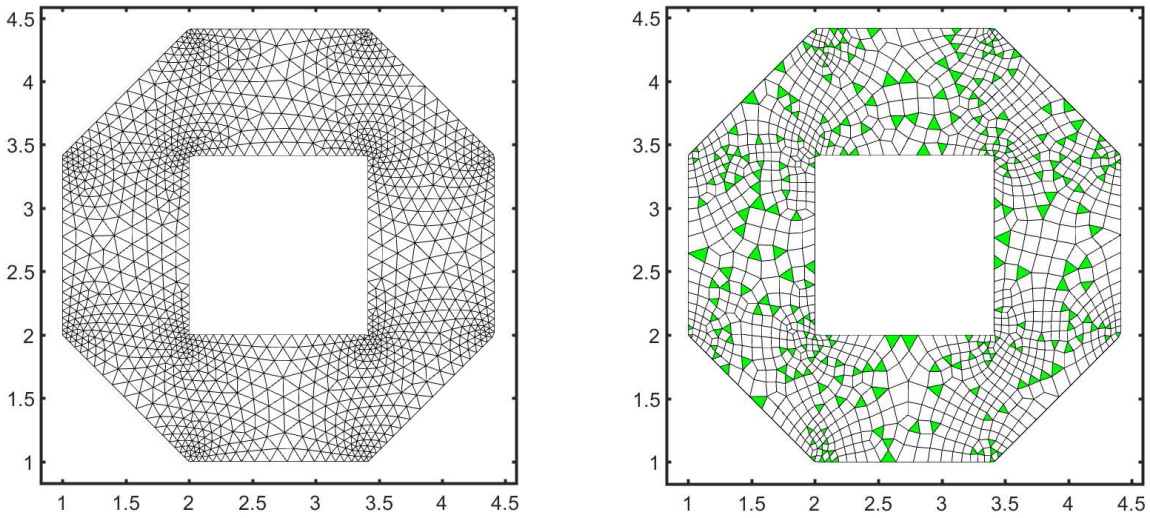


Figure 3.1: *Left* is the original triangular mesh, *right* is the quad-dominant mesh resulting from the Greedy Approach (255 *triangles*, 1,087 quads, avg. element quality of 0.8095 after smoothing).

is easily implemented and very quick due to its simplicity; however, it falls far short of producing a quad-only mesh. Instead, the procedure produces what is known as a *quad-dominant* mesh because of the relative number of triangular elements to quadrilateral elements in the mesh. Figure 3.1 shows results of the Greedy Approach performed on a small test case mesh, which originally contains 2,417 triangles with an average element quality of 0.913.

After applying the Greed Approach algorithm there is never any two adjacent triangles. However, this algorithm is unlikely to create a global matching such that all triangles in the mesh are used in the formation of quads. Quite often the Greedy Approach results in a quad-dominant mesh having many remaining triangles. Removal of these triangles can be quite involved. Several common recombination operations for

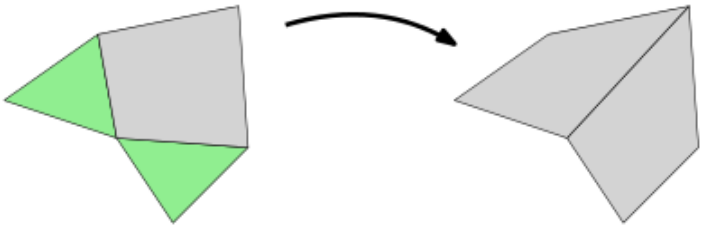


Figure 3.2: Example of the Edge Swap operation on two *triangles* sharing one vertex.

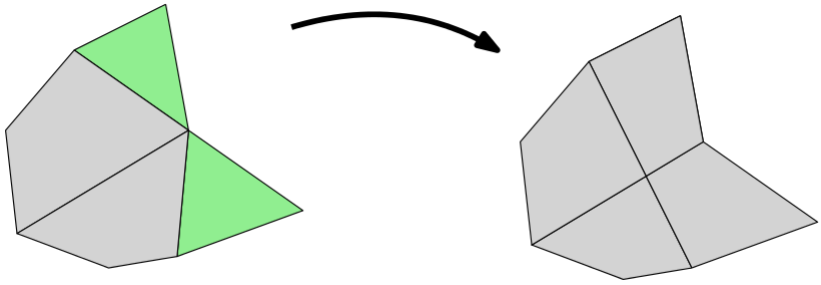


Figure 3.3: Example of the Vertex Duplication operation on two *triangles* sharing one vertex.

removing remaining triangles described by Remacle et al. are applicable only when two triangles are both attachments of a the same vertex [28]. Figure 3.2 details one operation called *Edge Swap* and Figure 3.3 details another operation called *Vertex Duplication* [28]. These operations alone are not a robust method for removing all remaining triangles in the mesh.

Following the application of the Greedy Algorithm algorithm and the Edge Swap and Vertex Duplication operations, any triangles remaining in the mesh are completely isolated from other triangles in the mesh, i.e., no two triangles in the mesh share a node. These triangles are known as *isolated* triangles and they are marginally

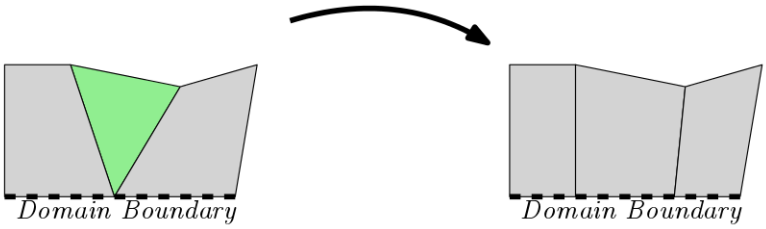


Figure 3.4: Converting a *triangle* into a quad by adding a new point to the domain boundary.

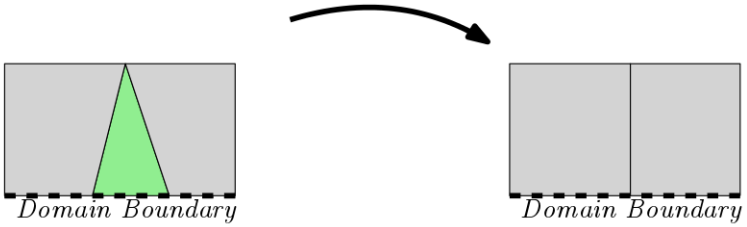


Figure 3.5: Removing a *triangle* by collapsing an edge of the domain boundary.

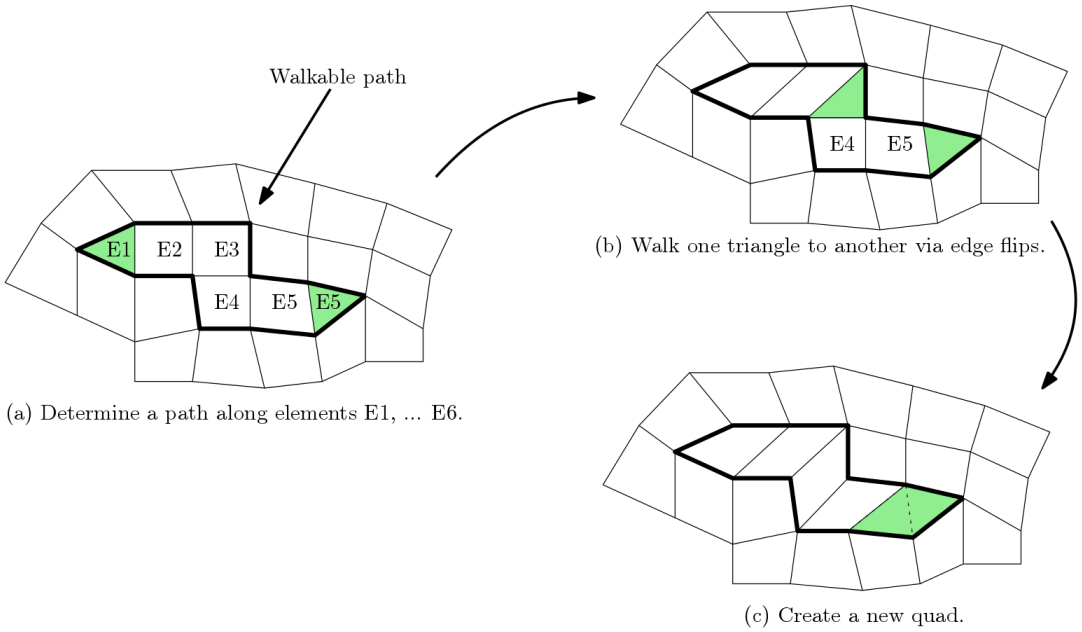


Figure 3.6: Walking a *triangle* toward another via edge flipping.

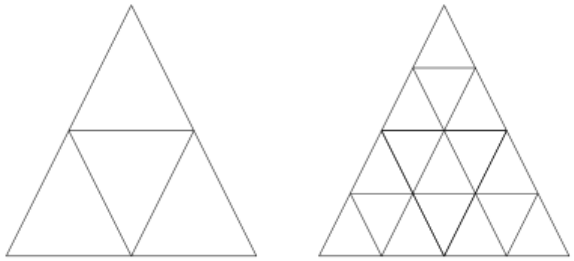


Figure 3.7: Two examples of meshes that do not have a perfect matching [28].

more difficult to remove from the mesh. In the event of an isolated triangle being located on the domain boundary, commonly used operations to remove those triangles include adding boundary points to create a new edge (Figure 3.4) or removing an edge and collapsing the triangle (Figure 3.5). These operations require an alteration of the boundary, which is typically non-ideal. An alternative method for removing isolated triangles involves "walking" one isolated triangle toward another within the mesh and then merging the two triangles once they share an edge [22]. Figure 3.6 shows that this process is effective for removing two isolated triangles; however, the consequences of doing so is an increased potential for creating degenerate (low quality) quads. Additionally, this operation can quickly become difficult to implement when the only two remaining triangles in the mesh are not within close proximity of each other or if there is only one triangle remaining in the mesh, which is referred to as an issue of *walkability*.

3.1.2 The Perfect Matching Approach

Through an adapted use of Edmonds' *Blossom algorithm* [11] and with the assistance of the supplemental operations like edge swap and vertex duplication, Remacle

et al. address the issue of isolated triangles and demonstrate the approach for finding a *perfect matching* for all elements in a triangular mesh. In short, a perfect matching for a triangular mesh is any matching and merging of adjacent triangles throughout the mesh that yields a completely quadrangular mesh. The authors show some impressive consequences of their algorithm, with many test cases resulting in meshes solely consisting of quadrilateral elements. However, there may not always be a perfect matching for a mesh (Figure 3.7), which is often due to the mesh having an odd-number of boundary vertices [30]. Although the Blossom algorithm is incredibly effective, the best "worst-case" computational requirement for the algorithm is still quite intense and time consuming, making the algorithm potentially problematic for very large meshes.

3.1.3 The Advancing Front Approach

Many algorithms have been developed that do not explicitly search for a perfect matching of a triangular mesh. Instead, these algorithms employ strategies from both direct and indirect methods for building quadrangular meshes. One commonly cited algorithm is Q-MORPH, which uses the technique of advancing fronts (Figure 3.8) with the assistance of a background triangular mesh to guide the creation of quadrilateral elements [26]. This effective algorithm uses the vertices and edges near the domain boundaries of the initial triangular mesh to define a "front" on which new quads are formed, see, for example, Figure 3.9. This process continues along the entire domain boundary and advances inward until all edges of the original triangular mesh have been "depleted". Unless the number of vertices defining the boundary is even, the result is a completely quadrangular mesh. Although, when given an odd

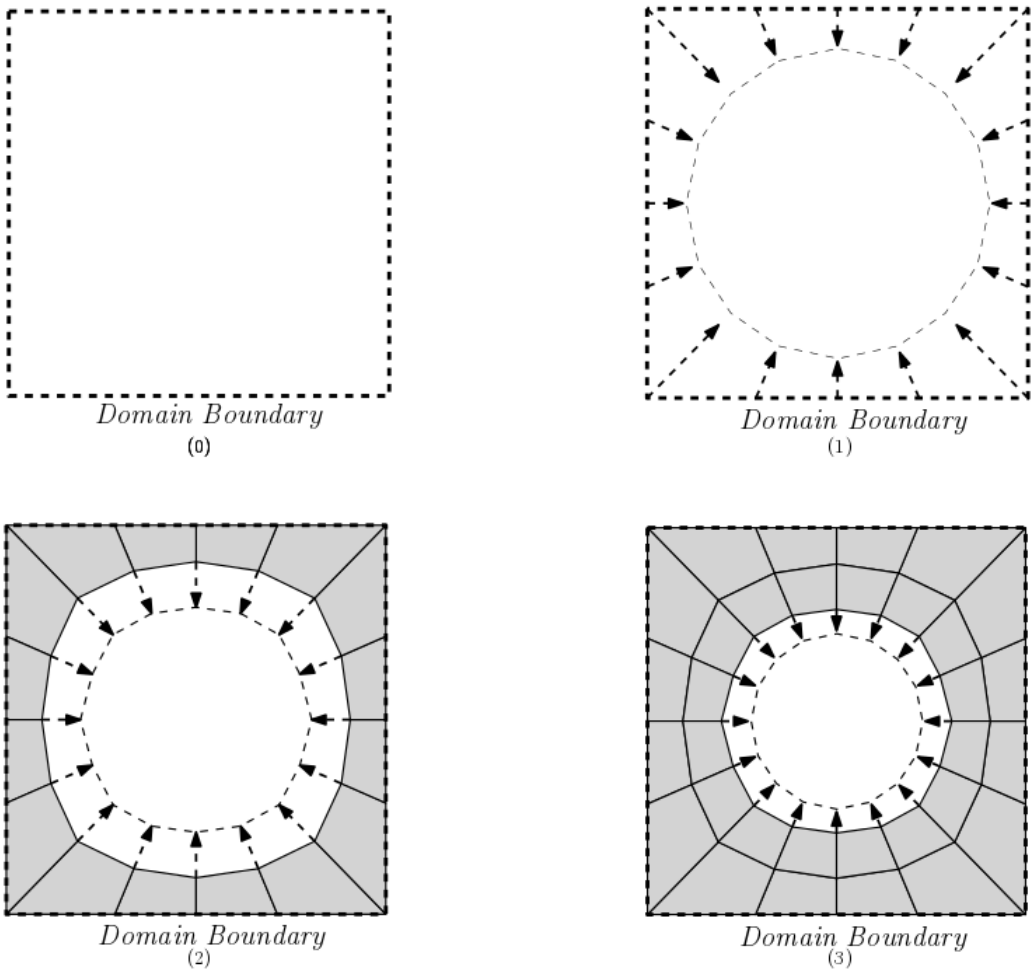


Figure 3.8: A direct method for creating quadrilaterals via advancing front technique. Steps *0* - *3* show the direct creation of quadrilateral elements as the front advances from the domain boundary toward the interior.

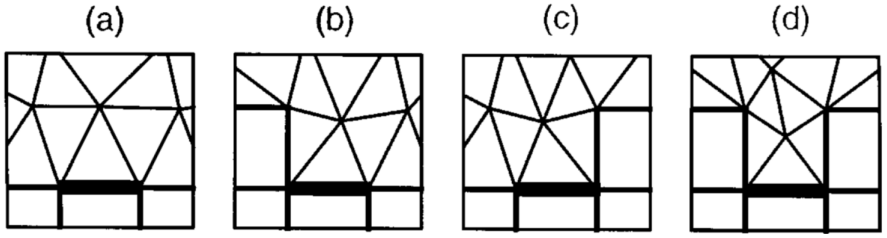


Figure 3.9: An advancing front technique that uses the vertices and edges of a background triangular mesh to create quadrilateral elements along the front [26].

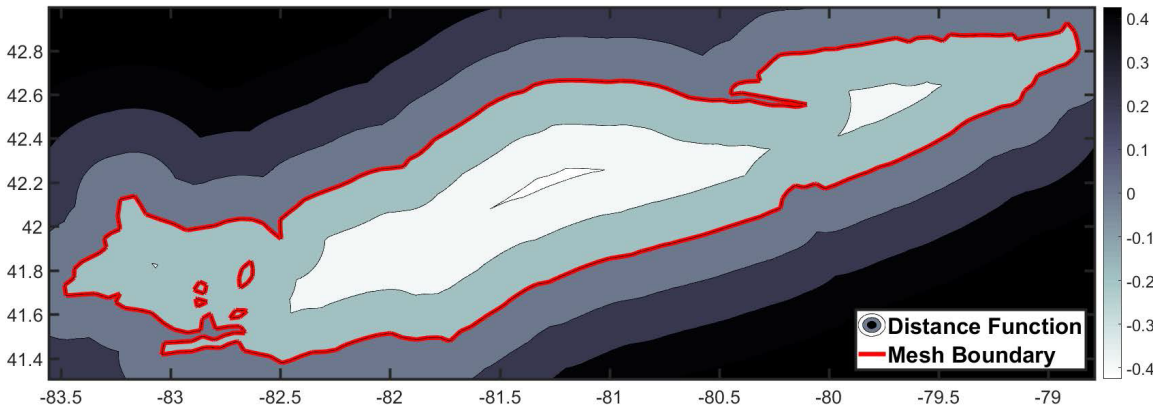


Figure 3.10: The Signed Distance Function $\phi(\mathbf{x})$ applied to the domain of Lake Erie. The contours show interpolated distance of every point from the *red* domain boundaries.

number of vertices defining the domain boundary, a single triangle in the interior of the mesh is inevitable without altering the boundary beforehand – similar to an imperfect matching in the Blossom Algorithm. One particular step of Q-MORPH algorithm is devoted to improving the quality of quads as the front advances through the mesh. Although the algorithm is efficient as a whole, this tact may be unnecessary. Inspired by this and the drawback of possibly having interior isolated triangles, QuADMESH+ research explores a similar method for quadrangulation that addresses these shortcomings.

3.1.4 The Distance Approach

Inspired by lessons learned from the Q-MORPH and Blossom algorithms, the early stages of research for QuADMESH+ focused on developing a strategy for iteratively building quadrilateral elements that avoids isolating triangles in the interior. The initial methodology deployed to achieve this was through the use of existing framework

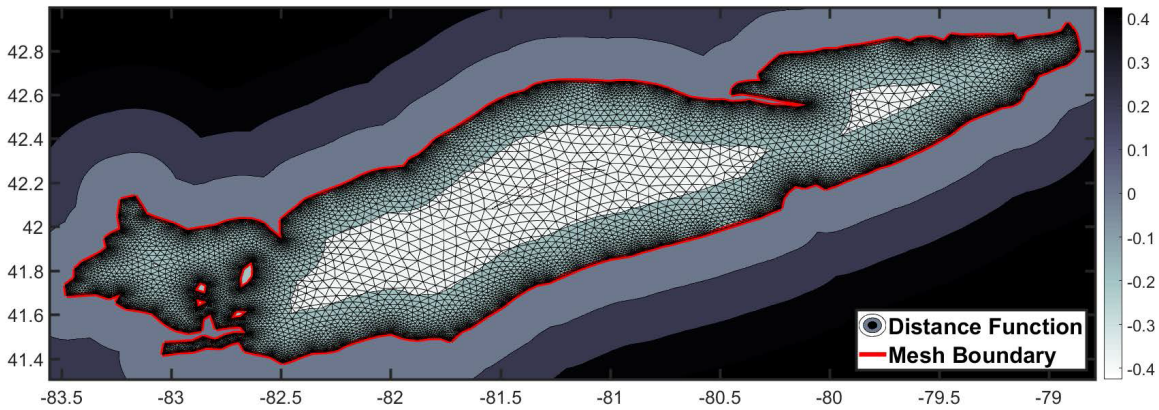


Figure 3.11: Triangular mesh of Lake Erie with the Signed Distance Function $\phi(\mathbf{x})$.

from the ADMESH+ signed distance function $\phi(\mathbf{x})$. Figure 3.10 shows an example of a signed distance function on a proposed mesh for the domain of Lake Erie.

Through the use of a signed distance function, another basic algorithm – dubbed the Distance Approach – creates a prioritized list of all potential quads in the mesh by ranking the distance of that potential quad's centroid to the boundary. The algorithm is identical to the Greedy Approach, except the sorted list of potential quads is with respect to the distance of the potential quads' centroids to the nearest boundary rather than their element quality. As in the Greedy Approach, any other potential quad within the list derived from one of the two triangles involved in the previous merging must be removed from future consideration. This process continues until the list is exhausted. While this process has not shown any indication that it can create a completely quadrangular mesh, it has a considerable effect on the number of triangles remaining after the matching.

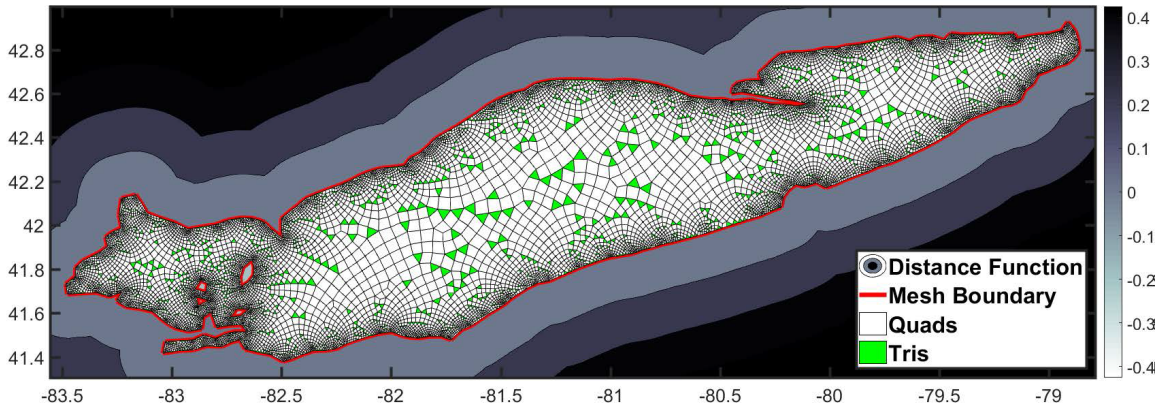


Figure 3.12: The Greedy Approach matching algorithm applied to Lake Erie. Result: 11,128 quads, 2,654 *triangles*, avg. quality of 0.8428.

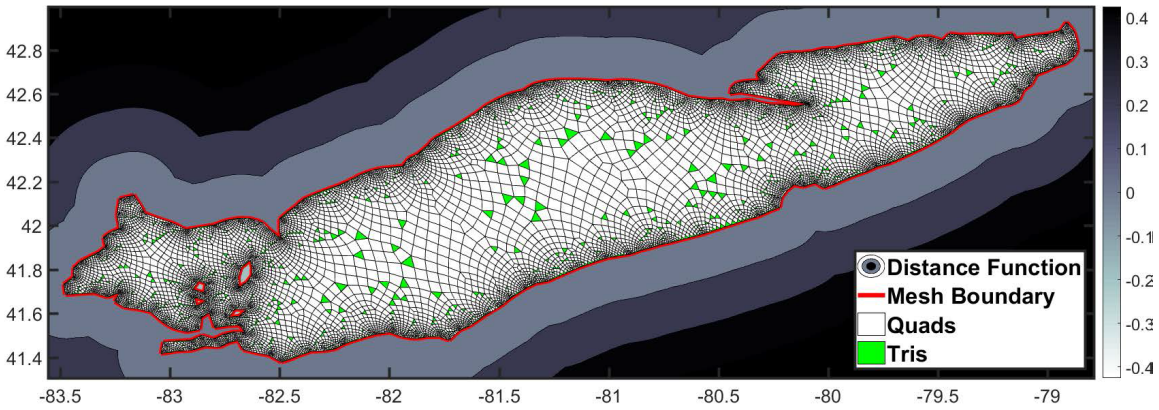


Figure 3.13: The Distance Approach matching algorithm applied to Lake Erie. Result: 11,749 quads, 1,952 *triangles*, avg. quality of .7550.

Testing the effectiveness of this new approach requires a modestly sized mesh, an example of which is a triangular mesh of Lake Erie with 24,910 elements and an initial average element quality of 0.7756 (Figure 3.11). Applying both the Greedy Approach and the Distance Approach yields the results seen in Figures 3.12 and 3.13. A comparison of the results show that the Distance Approach decreases the number of isolated triangles while also causing the average element quality of the mesh to slightly suffer with respect to the Greedy Approach. Further visual inspection also points to a greater concentration of isolated triangles near the domain boundaries for the Distance Approach compared to the Greedy Approach.

To confirm the trend of these results, consider a potential quad and the $x-$ and $y-$ coordinates of its centroid defined as $\mathbf{x} = [x, y]$. A weighting function $W\mathbf{x}$ accounts for the quality and the distance of $\mathbf{x}$ from the nearest boundary and is defined as

$$W(\mathbf{x}) = (1 - F) * \frac{\phi(\mathbf{x})}{D_{max}} + F * q(\mathbf{x}) \tag{3.1}$$

where F is a factor dictating the preference of quality over distance in the potential quad weighting, i.e., $F = 1.0$ weights only quality and $F = 0.0$ weights only distance, $\phi(\mathbf{x})$ is the distance of a potential quad to the nearest boundary, D_{max} is the maximum possible distance of any point in the mesh to the nearest boundary, and $q(\mathbf{x})$ is the quality of a potential quad. This function may be used to create a sorted list that is then processed in an identical manner as the Greedy Approach and Distance Approach algorithms. This new algorithm is referred to as the Hybrid Weighted Approach.

3.1.5 The Hybrid Weighted Approach

Using the weighting function $W(\mathbf{x})$ and a range of values $F = [0.0 : 0.1 : 1.0]$ (where $F = 1.0$ does not consider distance and $F = 0.0$ does not consider quality)

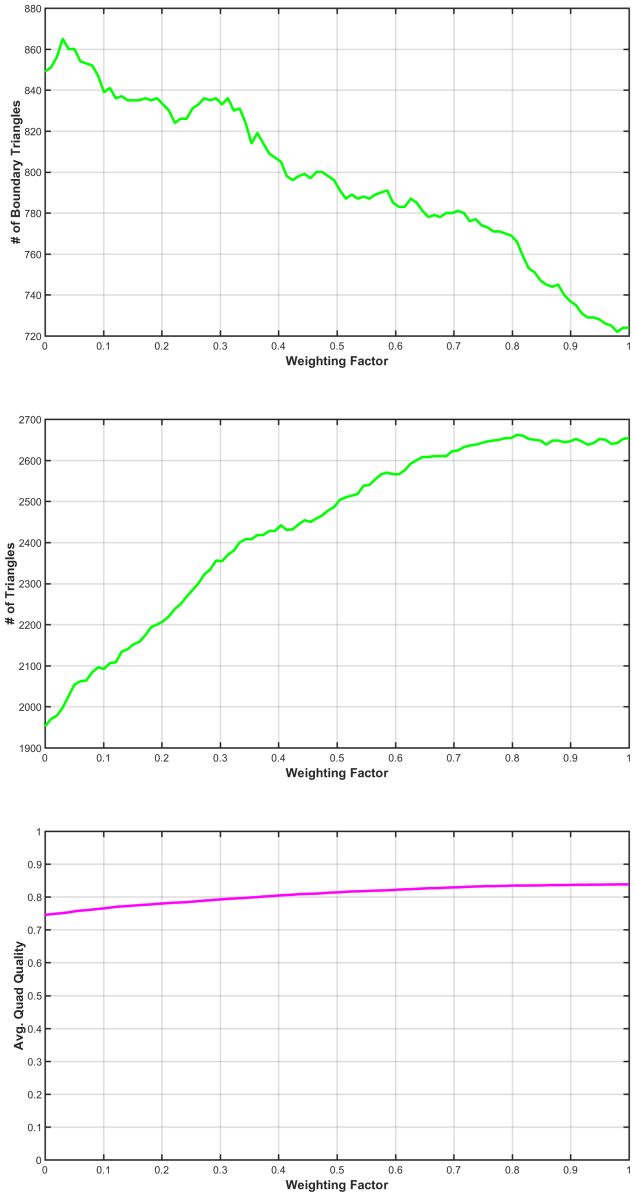


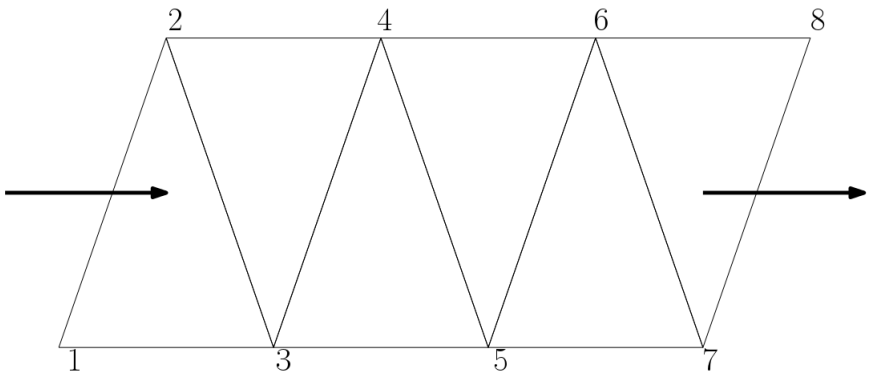
Figure 3.14: Applying a range of Weighting Factor values to the Hybrid Weighted Approach. *Top* is the number of triangles on the mesh boundary, *middle* is the total number of triangles remaining in the mesh, and *bottom* is the mean element quality. A Weighting Factor of 0.0 weights only distance while a Weighting Factor of 1.0 weights only quality.

for the original triangular mesh of Lake Erie, the effect of quality and distance in a simple matching becomes clear. Figure 3.14 shows that the total number of triangles remaining in the mesh increases and the total number of triangles with at least one vertex on the boundary decreases as quality is weighted increasingly more than distance. Also worth noting is that the meshes' average element quality remains within an acceptable range regardless of the value of weighting factor. From this it is concluded that prioritizing distance over quality has only a marginal effect on the average element quality, implying that perhaps quality should not be the highest priority during the matching process. Instead, a more fruitful strategy might focus on reducing the number of remaining triangles to zero and then applying topology improvements in a post-processing scheme to increase element quality.

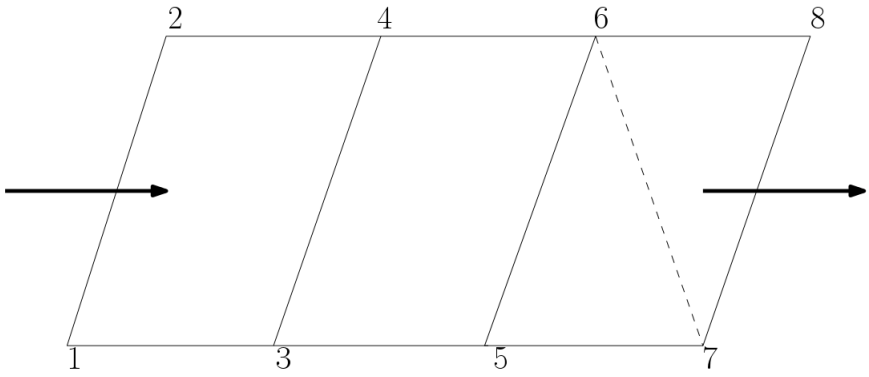
3.2 The Mesh Layers Approach

The Hybrid Weighted Approach uses a Euclidean definition for distance; however, quantifying the "distance" of a element from the mesh boundary exclusively in a Euclidean sense can be misleading. Euclidean distance is misleading because it does not necessarily imply any relationship between an element and its distance to the boundary. Therefore, developing an algorithm based on the results seen in section 3.1.5 alone is fruitless without a more precise definition for an element's boundary proximity. Instead, a more appropriate method for quantifying boundary proximity is made from a graphical/topological perspective.

An important concept used to describe the topology of elements within a mesh is triangle *strips*. A strip of triangles is a set of triangles with vertices $(1, 2, 3, 4, 5, 6, 7, 8)$ and which are consecutive, adjacent, and show a clear topological "direction". Due



(a) A Triangle Strip and its topological direction



(b) Formation of quadrilaterals along the topological direction

Figure 3.15: Forming quadrilaterals along a triangle strip example. *Arrows* show a clear topological direction of the adjacent triangles, which is a key characteristic of triangle strips.

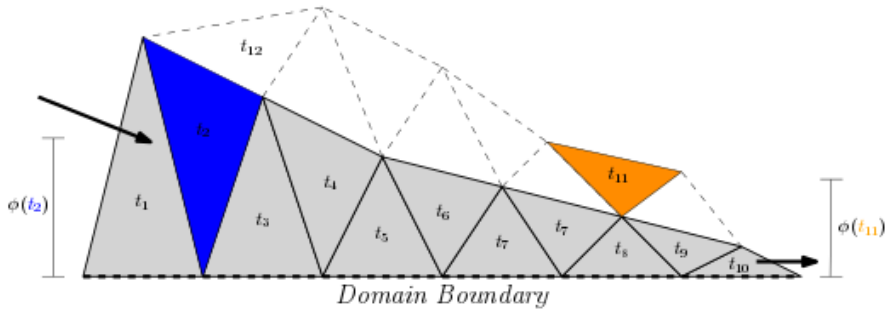


Figure 3.16: An example set of triangles near the boundary of a domain. $\phi(\mathbf{x})$ is the euclidean distance of $\mathbf{x}$ to the nearest domain boundary. *Arrows* indicate the "direction" of the strip of triangles along the boundary.

to these characteristics, any *ith* triangle in the strip may be described by the *ith*, $(i+1)st$ and the $(i+2)nd$ vertices of the strip [12]. The direction of the strip is maintained as long as this pattern continues. Evans et al. exploit this concept in various ways throughout their indirect quadrangular mesh generating algorithm by using the strips of triangles to form quadrilateral elements as seen in Figure 3.15.

Consider a set of triangles near some domain boundary as shown in Figure 3.16. The Euclidean distance of triangle t_2 is greater than that of triangle t_{11} despite triangle t_2 being located on the domain boundary. Under the Hybrid Weighted Approach matching regime, triangle t_2 and one of its neighbors would be prioritized for matching more than triangle t_{11} and one of its neighbors. This is an inefficient outcome if the goal of the matching is to minimize the number of unmatched triangles in the interior of the mesh. A more sophisticated matching algorithm could instead acknowledge that triangle t_2 is part of a strip that is closer to the boundary than the strip of triangles that triangle t_{11} may belong to.

3.2.1 Defining Mesh Layers

Expanding the concept of classifying triangles as strips of elements could be used to discretize the entire mesh into strips – or layers – of topologically-similar triangles. Instead of a value for Euclidean distance, a description of an element's proximity to the nearest boundary is made in terms of which mesh layer it resides in. Called *Mesh Layers*, this concept is pivotal to the matching process of this algorithm because it allows for the quantification of the geometrically-innermost region of the mesh, irrespective of the Euclidean distance.

A mesh has a finite number of layers because the elements of each mesh layer are mutually exclusive to all other mesh layers. The procedure for discretizing a mesh into layers can be described as follows: consider a domain Ω consisting of n triangular elements defineby an $nx3$ connectivity list with respect to an ordered set of p points (vertices), where $\Omega_E = [t_1, t_2, ..., t_n]$ is a unique list of all elements in the domain and $\Omega_V = [V_1, V_2, ..., V_p]$. Identifying the mesh layers must begin at the domain boundary; however, the process can be described in abstract terms, i.e., for some Layer L. The procedural steps are:

1. The vertices define the boundary of the domain Ω are referred to as Layer L's *Outer Vertices*, OV_L, where $OV_L \subset \Omega_V$.

2. Identify any element of Ω who's connectivity list includes at least one OV_L. All of these elements belong to Layer L of the mesh, Ω_L, i.e., $\Omega_L \subset \Omega_E$ and $\Omega_L \neq \emptyset$.

3. Create a unique list of vertices that define Layer L (Ω_{V_L}) from the connectivity list of each Ω_L.

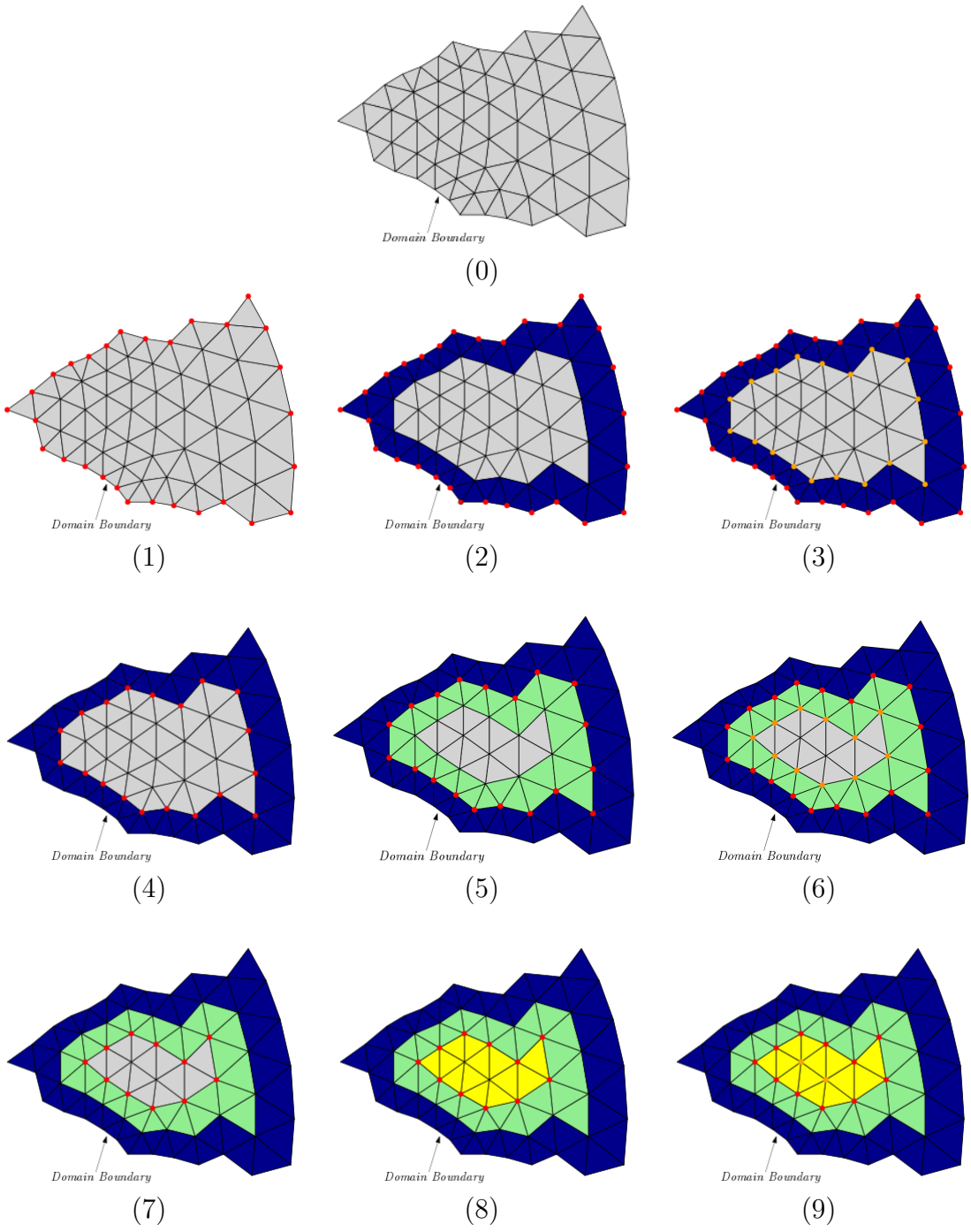


Figure 3.17: Example domain (step *0*) discretized into mesh layers (blue, green, and yellow) in steps *1* - *9*. The *Outer Vertices* and *Inner Vertices* are red and orange of each layer, respectively.

4. The vertices defined by the set difference between Ω_{V_L} and OV_L are known as the *Inner Vertices* of Layer L, IV_L, i.e., $IV_L = \Omega_{V_L} - OV_L$, where $IV_L \subset \Omega_V$ and $IV_L \cap OV_L = \emptyset$. This is the last piece of information for a given Layer L.

5. Advance to Layer $L + 1$ and repeat steps 1, 2, 3, and 4.

    - If $\Omega_E = \emptyset$, then Layer $L-1$ was the last layer of the mesh and the procedure is complete.

Figure 3.17 illustrates the mesh layers process on a small example domain.

With the mesh layers established, the elements within each layer may be further decomposed by their topological similarities. Designating so-called *Outer Elements* and *Inner Elements* of each layer allows for an even more refined definition of an element's location within a mesh. Given that the outer boundary of some mesh layer L is defined by the outer vertices of that layer OV_L and that any edge along the boundary of Layer L is a *boundary edge*, the definitions for these Outer Elements and Inner Elements are as follows:

- An Outer Elements (OE_L) within Layer L are those with at least one edge that is a boundary edge of Layer L.

- An Inner Element (IE_L) within Layer L are the set difference between Ω_L and OE_L. Note that an element may be defined by as many as three OV_L but still be classified as an IE_L because none of the element's edges are boundary edges of Layer L.

These definitions provides perspective as to which elements of some Layer L are neighbors of elements in the previous layer $L - 1$ (OE_L) and which are not (IE_L).

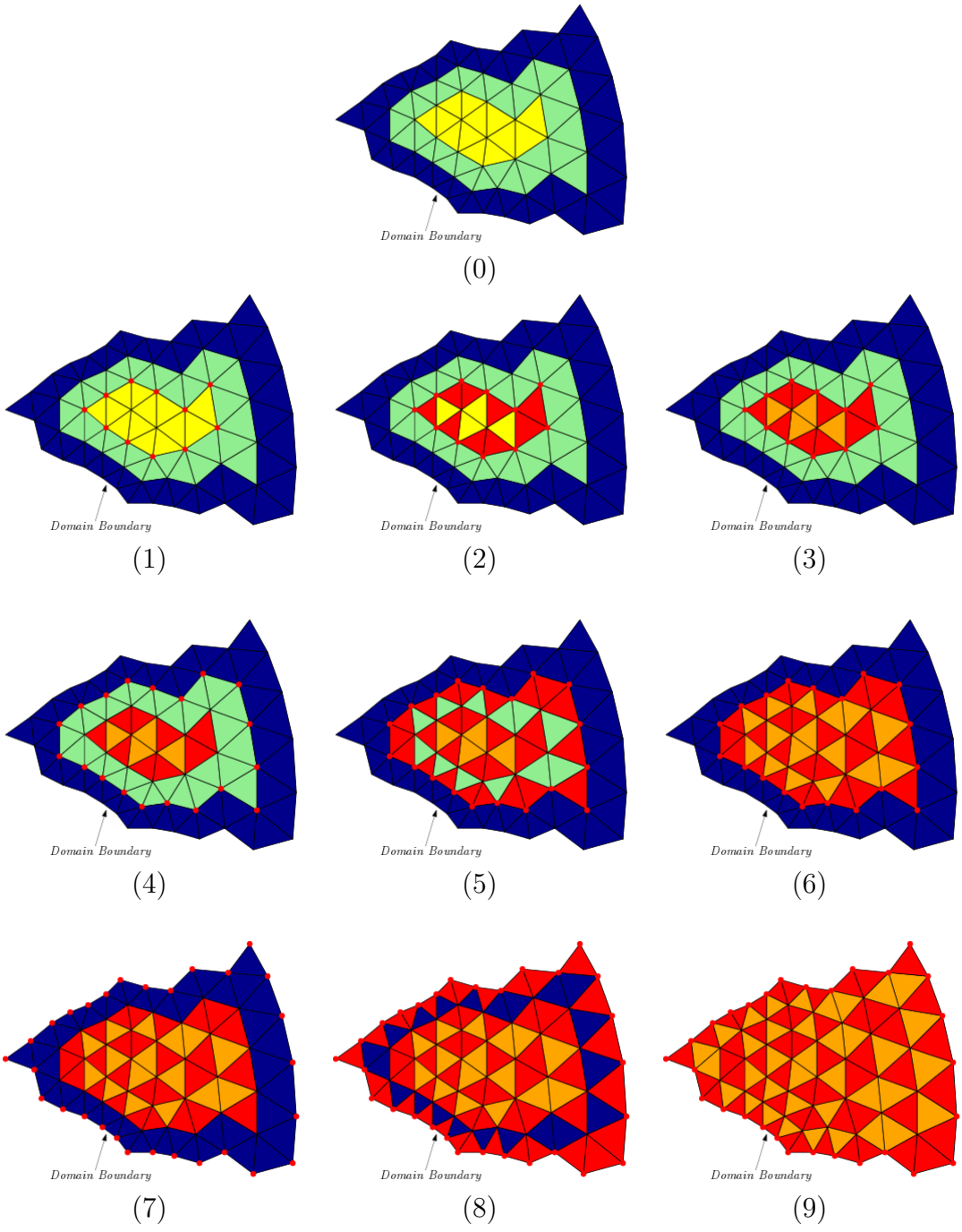


Figure 3.18: Example domain's mesh layers (step *0*) discretized into *Outer Elements* and *Inner Elements* in steps *1 - 9*. *Outer vertices* and *inner vertices* of each layer are also colored red and orange, respectively.

These definitions derive some properties of OE_L and IE_L, most notably of which is that no OE_L can have more than two neighbors that are also OE_L. Figure 3.18 presents an example of Outer Elements and Inner Elements defined on a small example domain.

Another important property of mesh layers is the instance of *self-intersection*. Ideally, the topology of elements in a mesh layer is relatively simple and resemble triangle strips or layers of an onion like Layer 1 and Layer 2 in Figure 3.17. Due to the inherent complexity of meshes and their domain boundaries, it is typically inevitable for layers to border themselves like Layer 3 in Figure 3.17. An obvious visual characteristic of an self-intersecting layer is an unclear path (recall walkability as seen in Figure 3.6). An elegant solution to resolving the walkability issues seen in self-intersection layers is the creation of *flagged edges*. A flagged edge is any edge within a mesh composed of two inner vertices. Figure 3.19 highlights the helpfulness of

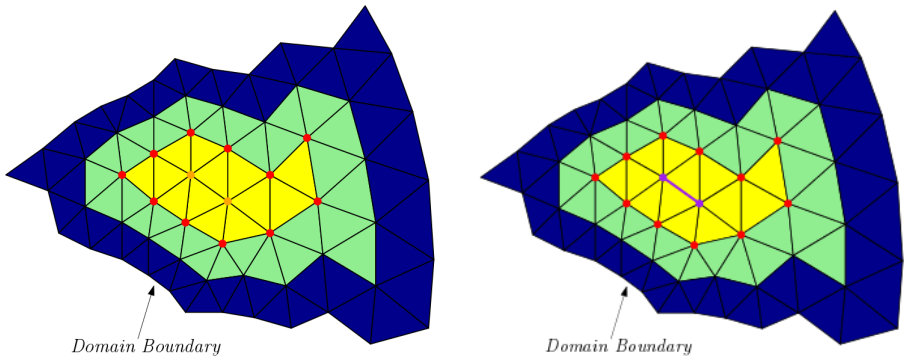


Figure 3.19: An example mesh domain with a flagged edge. The edge defined by two inner vertices within Layer 3 (step *1*) of the example mesh domain becomes a flagged edge in step *2*.

flagged edges. Applications of flagged edges are advantageous in many ways, notably in the context of edge neighbors. Consider two elements that neighbor each other across a flagged edge; this flagged edge can make the two elements "invisible" to each other, effectively meaning that these two elements are not neighbors of each other.

3.2.2 Using Mesh Layers

The Mesh Layers procedure may be applied to meshes with triangular elements, quadrilateral elements, or both. The effectiveness of discretizing a mesh into layers is three-fold:

1. Layers allow for the discretization of the mesh domain by topological distance to the boundary, therefore the innermost region(s) of the mesh is identifiable and indisputable. An algorithm that begins matching triangles within the innermost layer of a mesh and iteratively advances outward to the next innermost layers should provide an improved matching compared to just weighting Euclidean distance of an element. Similar to Q-Morph and other hybrid advancing front techniques, this method would attempt to match every element of the innermost Layer L (a "Front") – so-called *inter*-layer matchings – before repeating that process outward one layer in Layer $L-1$, only performing *intra*-layer matchings when necessary. The best-case scenario of this method is that it produces a matching for each layer and no triangle remains unmatched as the algorithm approaches the domain boundary. Instead of a complicated matching algorithm like the Blossom algorithm, this algorithm could employ simpler decision-making heuristics to match triangles together. Any remaining

triangles within the boundary layer (Layer 1) may be dealt with via one of the multiple recombination operations described earlier.

2. The creation of inter-layer matchings is a novel idea in-line with the methodology of using triangle strips to produce quadrilaterals. Within a triangle layer, each triangle typically has only one or two neighbors (in that layer) as an option for matching. Similarly for mesh layers, any inter-layer matching heuristics typically need only decide between one or two layer-neighbors of some target triangle after the implementation of flagged edges, although it is possible for an element to have three neighbors. Regardless, the decision-making process for matching triangles within mesh layers is notably simpler than algorithms like Blossom-Quad or The Hybrid Weighted Approach.

3. Finally, any algorithm implementing mesh layers is prepared to create mixed-element meshes. These mixed-element meshes ideally would use quadrilateral elements wherever high resolution is not needed and triangular elements where resolution is needed, i.e., domain boundaries. This approach is advantageous because the quadrilaterals would provide increased computational efficiency but not at the expense of mesh quality or resolution. Such an approach could split the initial triangular mesh into two sub-mesh domains; one containing triangles that shall remain unaltered and one containing targeted triangles to be converted to quads because the triangles exceed some threshold for edge-size, distance to boundary, or mesh layer classification. The Mesh Layers Approach to creating matchings would then be applied explicitly to the latter sub-mesh domain, the result being mixed-element mesh.

Under the proposed infrastructure, an algorithm using the Mesh Layers Approach to match triangles to form quads begins in the innermost layer. Triangles of the innermost layer match to form quads via inter-layer matchings and then intra-layer matchings with the next-innermost layer. Heuristic decision-making forms the basis for creating all matchings within the layers. The algorithm iteratively repeats this procedure with each layer of the mesh until the boundary layer. The boundary layer undergoes a similar matching process, with any remaining triangles easily removed via local recombinations and boundary point manipulations. The resulting mesh is completely quadrangular.

Chapter 4: Matching Algorithms

With the Mesh Layers Approach there are clear priorities for creating matchings, i.e., triangles located within the innermost layers of a mesh must be matched before triangles located within layers that are closer to the domain boundary. The purpose of these priorities is that ideally the matching algorithm does not leave any triangle unmatched as it advances to the boundary of the mesh.

Several observations inspired the development of a set of decision-making heuristics for creating triangle matchings within a given mesh layer. These heuristics are designed to identify triangles susceptible to becoming isolated and then to elevate their priority for matching as the matching process advances through each layer. One of these heuristics defines the order in which matchings occur. This order dictates that for a given Layer L, all possible *intra*-layer matchings must be exhausted before any *inter*-layer matchings with Layer $L-1$ may occur. This pivotal heuristic satisfies the constraint that triangle matchings progress from the innermost layers outward to the boundary layer.

As discussed previously, the matching of triangles located in the boundary layer is a slightly different process than the matching of triangles located elsewhere. For this reason, the matching algorithm is broken down into two stages:

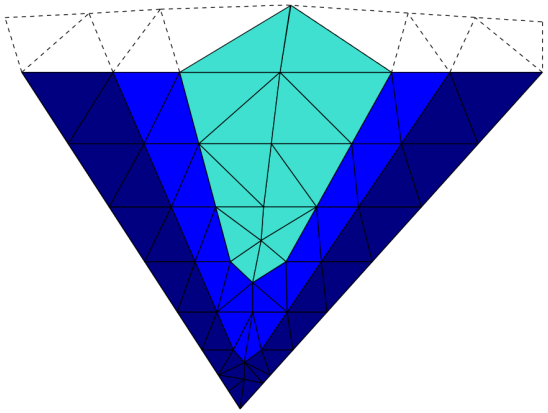
1. Matching triangles in the interior layers of a mesh

2. Matching triangles in the boundary layer of a mesh

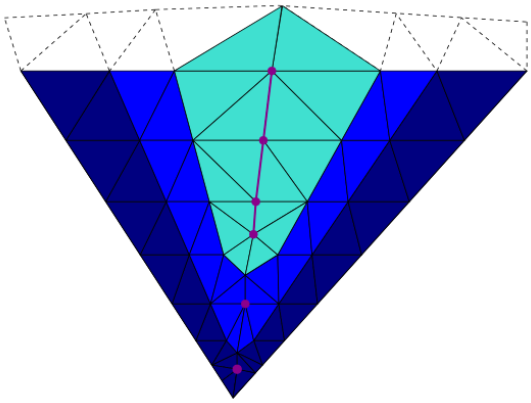
4.1 Interior Layer Matchings

Most of the heuristics used in QuADMESH+ try to determine which triangles in a layer are most susceptible to becoming isolated as a result of its neighbors being "used" in a matching. This determination is made for each unmatched triangle in Layer L by counting the number of unmatched – appropriately referred to as *eligible* – intra-layer neighbors of that triangle. Triangles with the most eligible neighbors (three) are of lowest priority, conversely triangles with the fewest eligible neighbors (one) are a top priority. Ideally, triangles with only one eligible neighbor are matched immediately, otherwise they risk becoming isolated if that only eligible neighbor matches with a different triangle. It is important to note the role played by *flagged edges*. When some triangle t has two eligible neighbors but one of them lies across a flagged edge, t truly has only one eligible neighbor. While this heuristic may sound like an unnecessary handicap, it has shown to be quite effective in simplifying the matching decision-making. Figure 4.1 illustrates this concept.

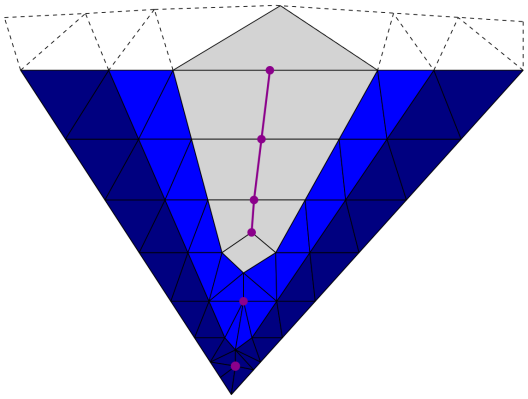
Prioritizing triangles with only one eligible neighbor is a fast and simple way to create matchings, but inevitably only triangles with more than one eligible neighbor will exist. Additional decision-making heuristics are needed to determine which of the triangles with more than one eligible neighbor should be prioritized. Thus, the other principal rule governing triangle matching priorities within some Layer L of a mesh is the criticality of *Inner Elements* (IE_L) being matched before *Outer Elements* (OE_L). By definition, only OE_L share edges with triangles of Layer $L-1$ (see Figure 3.18). This indicates that an intra-layer matching involving an IE_L is impossible.



(0)



(1)



(2)

Figure 4.1: Example domain's mesh layers (step *0*) and a hypothetical intra-layer matching with respect to the flagged edges (steps *1-2*).

Under the governing heuristic rule stating that intra-layer matchings within Layer L must precede inter-layer matchings between layers L and $L-1$, only OE_L will ever be eligible for inter-layer matchings. Therefore, only IE_L can become isolated triangles, which is why IE_L must always be chosen for matchings first when all triangles in Layer L have more than one eligible neighbor.

Creating a matching in QuADMESH+ is a delicate procedure. The priorities for many triangles within a layer may change as each matching occurs and the eligible neighbors of remaining triangles decreases. The process for creating a matching is described in two parts:

1. Choosing a candidate, T_1, for a new matching

2. Completing a matching by choosing one of T_1's eligible neighbors, T_2

4.1.1 Choosing a Candidate for Matching

The use of heuristic decision-making to identify one triangle within Layer L that is more susceptible to becoming isolated than the other eligible triangles in the layer is a process called *Selecting* T_1. The hierarchy of priorities for choosing T_1 for a new matching within some layer L are simple; triangles in Layer L with only one eligible neighbor, followed by triangles with two eligible neighbors, and lastly, triangles with three eligible neighbors. The resultant list of potential candidates is known as C_{T_1}. Depending on the size of the mesh, this initial set C_{T_1} may contain as few as one triangle or as many as thousands or even millions. Additional heuristic rules serve as tie-breakers when C_{T_1} contains more than one eligible triangle. These tie-breakers are applicable only when C_{T_1} contains eligible triangles with at least two eligible neighbors (recall that triangles with only one eligible neighbor must be merged immediately).

Different tiebreakers are assigned to eligible triangles in C_{T_1} depending on whether they are IE_L or OE_L. The tiebreakers for C_{T_1} that are IE_L each are as follows:

1. The first tie-breaker selects a triangle listed in C_{T_1} that has the most eligible IE_L-neighbors. The purpose of this tie-breaker is to match IE_L with each other, thus preventing two isolation-susceptible triangles from becoming isolated instead of only one.

2. If all (or none) of C_{T_1} have the same number of IE_L-neighbors, the second tie-breaker selects a triangle listed in C_{T_1} whose eligible neighbors have the fewest eligible neighbors. This second tie-breaker assumes that said neighbor is marginally more susceptible to becoming isolated and must be matched sooner rather than later.

If all of C_{T_1} are OE_L, there are no longer any unmatched IE_L remaining in Layer L. When there are multiple C_{T_1} that are OE_L there is only one tie-breaker, which selects a triangle listed in C_{T_1} that has the most neighbors (by default, these are OE_L-neighbors). Like many of the heuristic rules used in QuADMESH+, this rule in particular requires further investigation to confirm its appropriateness. Visual observations of this rule in effect suggest that fewer OE_L remain unmatched at the conclusion of a layer matching.

4.1.2 Completing a Matching

Once a T_1 is selected within Layer L it must be matched with one of its eligible neighbors. This process is named *Selecting* T_2. Selecting T_2 is another heuristic decision-making process and involves several steps. While the process of selecting T_2 – when the selected T_1 has only one eligible neighbor – is simple (there is only

one option, after all), other situations require an analysis of T_1's multiple neighbors. Similar to the process of selecting T_1, rules for selecting T_2 are specific to whether the eligible triangles are an IE_L and an OE_L.

1. If the selected T_1 has two eligible neighbors, the set tie-breakers are as follows:

   (a) The first tie-breaker of this set selects the eligible neighbor of T_1 that is a IE_L. Pairing T_1 with an IE_L is beneficial because it prevents an IE_L from becoming isolated, as described in step 1 of 4.1.1.

   (b) If both of T_1's neighbors are IE_L, the second tie-breaker examines the neighbors of T_1's neighbors. This tie-breaker selects the neighbor of T_1 that has the fewest IE_L-neighbors. The purpose of this tie-breaker is to assume that T_1 is in a local strip of triangles that are all IE_L and to select the neighbor of T_1 that is furthest from the "middle" of the strip as possible.

   (c) If step 1b does not provide a clear choice for T_2 or if both of T_1's neighbors are an OE_L, then a third tie-breaker attempts to determine which of T_1's neighbors have the fewest total number of neighbors. The purpose of this tie-breaker is similar to steps 2 in 4.1.1.

2. If the selected T_1 has three eligible neighbors, the set of tie-breakers for selecting T_2 are as follows:

   (a) The first tie-breaker attempts to select the eligible neighbor of T_1 sharing the most edges with previously-matched triangles of Layer L. The purpose of this tie-breaker is to create new matches that are flush to the existing quadrangular portion of the mesh.

(b) If step 2a does not present a clear option for selecting T_2, a second tie-breaker selects the eligible neighbor of T_1 that has the most eligible neighbors that are an OE_L. The purpose of this tie-breaker is similar to step 4.1.1 in 4.1.1 and the latter clause of step 1c in 4.1.2.

When all of the possible inter-layer matchings of Layer L are exhausted, the intra-layer matching process begins by permitting any remaining OE_L to match with triangles of Layer $L-1$. OE_L are treated as an IE_{L-1}, i.e., any tie-breakers used with respect to inner elements are in turn applicable to OE_L. All OE_L must be matched before advancing to Layer $L-1$. Note that an OE_L may neighbor either an IE_{L-1} or an OE_{L-1}.

If, at any point, these tie-breakers do not point to one clear choice for T_1 or T_2, then the first entry in the lists of C_{T_1} or T_1's neighbors is selected. This is just one example of the potentially suboptimal decision-making of this matching methodology. Proof of this sub-optimality is presented when triangles unintentionally become isolated. Despite the best efforts of these heuristics, this rare circumstance can still occur. However, an elegant solution to this occurrence is to intentionally isolate another triangle (previously unmatched) that shares a vertex with the initial isolated triangle, forming a so-called "pair" of isolated triangles. The algorithm returns to the pair of isolated triangles and applies an edge swap recombination once the mesh matching process is completed. Despite the fact that this added procedure does marginally decrease the efficiency of the algorithm on the whole, the results are nonetheless impressive. Figure 4.2 illustrates a progression of the interior matching algorithm for an example domain up until the boundary layer. Figure 4.3 shows the process that accommodates for isolated triangles during the interior matching algorithm.

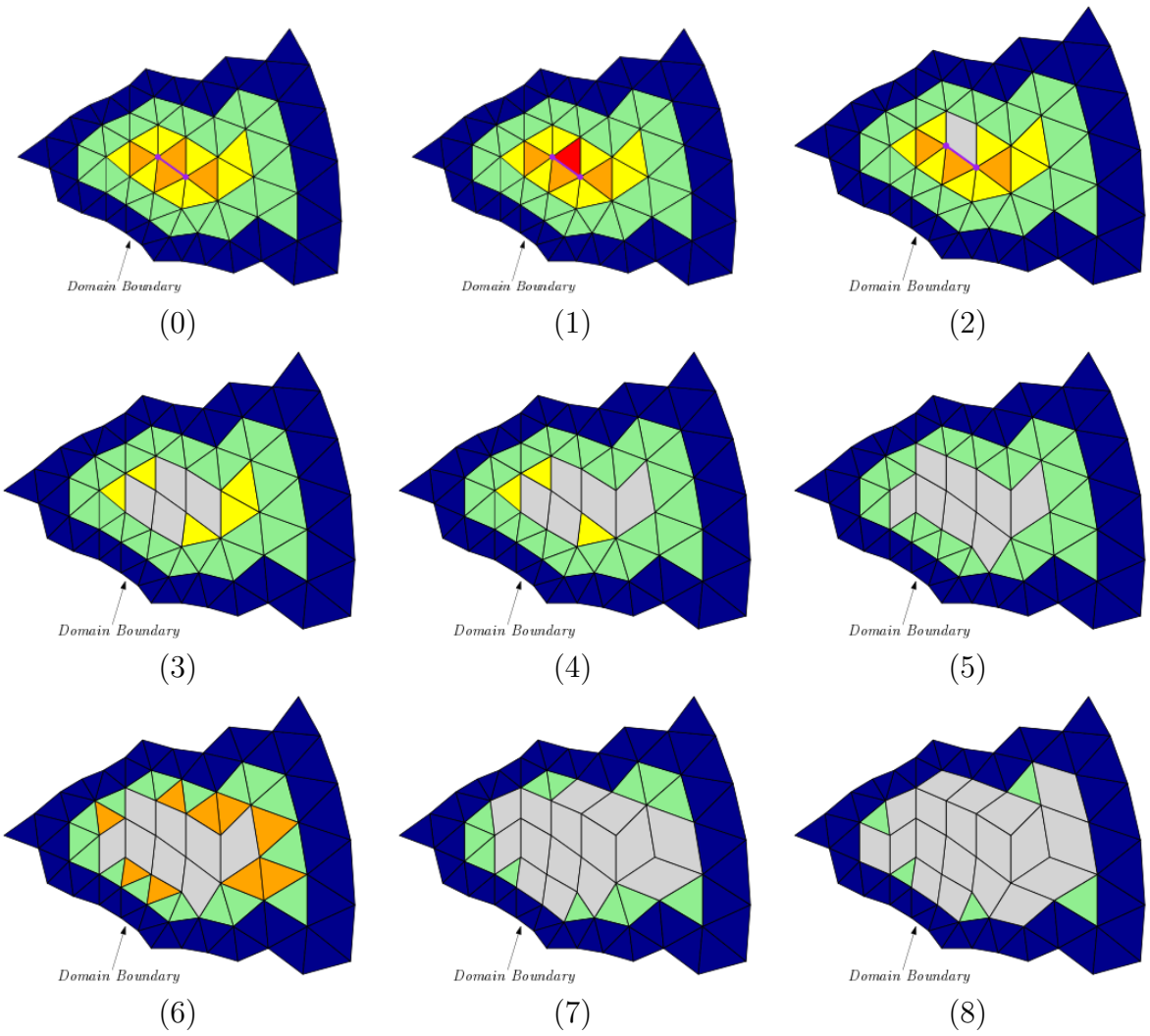


Figure 4.2: Example domain's interior matching process, steps *0* - *8*. Blue, green, and yellow are the domain's mesh layers, orange are the IE_L of a given layer, purple are flagged edges, red indicates a triangle selected for matching (T_1), gray indicates the completion of a matching (T_1 matched with it's neighbor T_2).

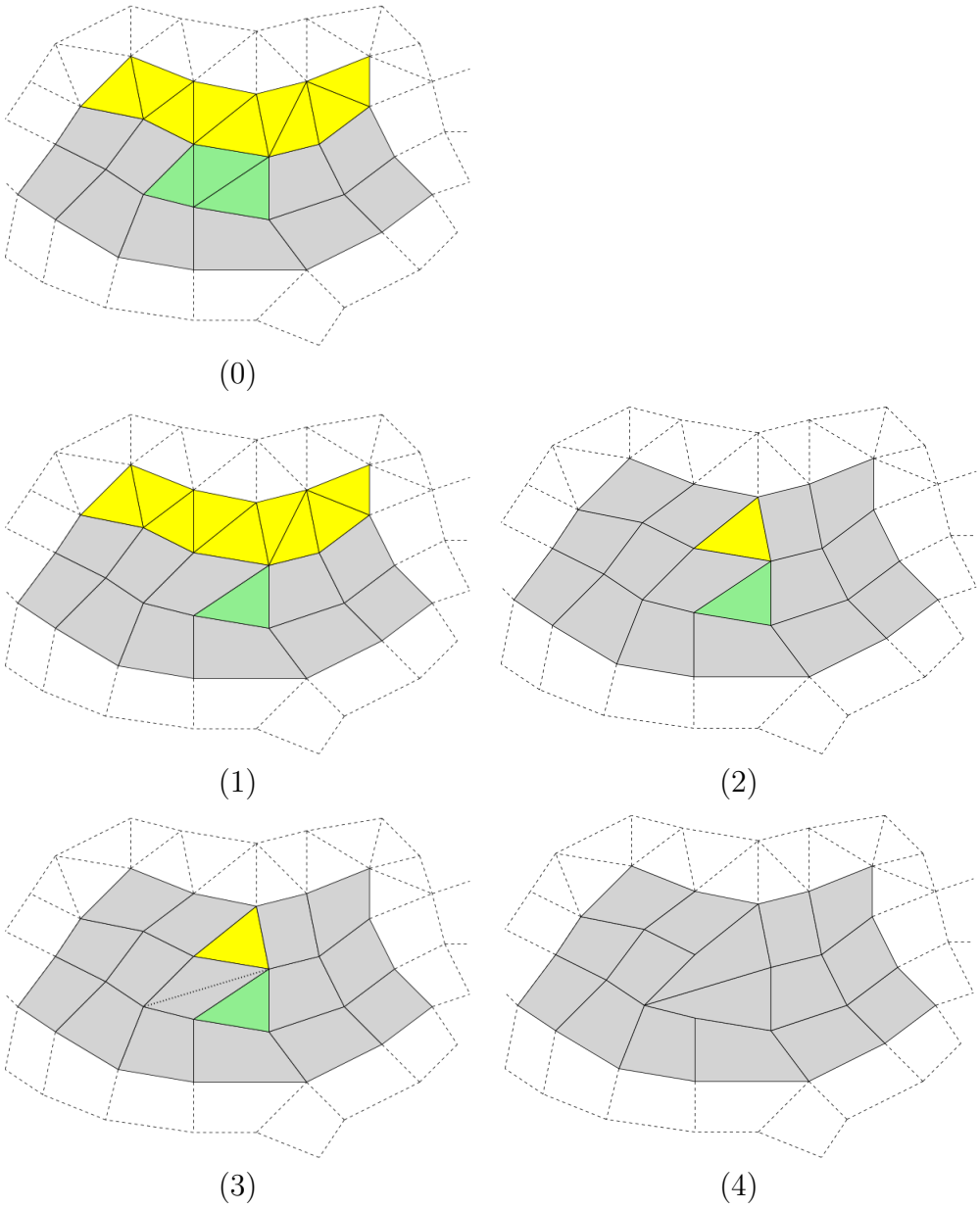


Figure 4.3: Isolated triangle removal process, steps *0* - *4*. Green and yellow are the domain's unmatched mesh layers, gray are matched pairs of triangles, i.e., quads.

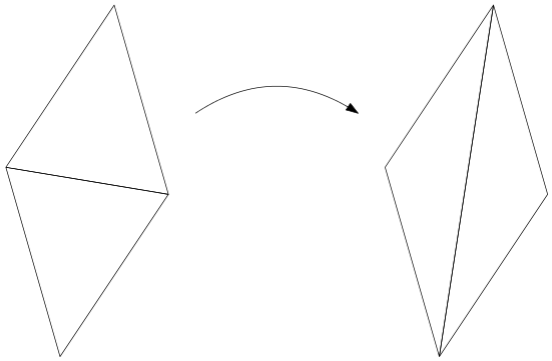


Figure 4.4: An example of the commonly-used edge flip operation [24].

4.2 Boundary Layer Matchings

Once all of the possible inter-layer matchings of Layer L are exhausted, the interior matching process ceases, i.e., no intra-layer matchings between layers L and $L-1$ are permitted. Boundary layer matchings are unique because the constraints of the boundary mean that any unmatched triangle of the boundary layer will remain unmatched. Dealing with unmatched triangles on the boundary requires manipulation of the boundary points, which is often not ideal (see Section 3.1.1). Instead, a new set of operations must be applied to create matchings in the boundary layer.

4.2.1 Pre-Processing Boundary Triangles

Once the matching process for Layer $L+1$ concludes, the set of elements comprising the boundary layer (now Layer L) consists of IE_L, OE_L, and RE_{L+1}, the latter of which are the unmatched OE_{L+1}. Before creating any triangle matchings in Layer L, first the walkability of the layer must be improved. This is accomplished by performing edge flips on each RE_L and one of their neighbors in Layer L (see Figure

4.4 for an example of the edge flip operation). Nest, with every triangle in Layer L having at least one vertex on the boundary (they are all OE_L) and any flagged edges taken into account, the boundary layer matching process may begin.

4.2.2 Boundary Matching Algorithm

As with the interior matching algorithm, the top priority for selecting T_1 in boundary Layer L is any element that has only one eligible neighbor (by default, this neighbor becomes T_2). The fundamental difference between the interior matching algorithm and the boundary layer matching algorithm occurs when there are elements in Layer L that have more than one eligible neighbor. In this circumstance, the priority is given to OE_L instead of IE_L. The purpose of this heuristic rule is to attempt to match all OE_L first and to try to have the last remaining triangle in Layer L (if any) be a IE_L. Recall that IE_L do not share an edge with the preceding layer, which is the domain boundary in this case. Any remaining triangle(s) along the boundary layer that are IE_L can be recombined into quad(s) by adding a point to the boundary (Figure 3.4). Generally, a more appealing outcome of manipulating a mesh boundary involves the addition of a point rather than the removal of one, i.e., added resolution is better than diminished resolution.

This boundary matching algorithm typically results in no more than one triangle remaining on the boundary at the conclusion of the process. This is often a direct result of the boundary being defined by an odd number of vertices, although this is not always the case. For instance, large meshes with increasingly complex boundary geometries may cause multiple triangles to remain after the matching process, including OE_L triangles. Applicable operations for removing remaining triangles from

the boundary include Edge Swap and Vertex Duplication if there exists two triangles sharing one vertex. Remaining triangles that do not share vertices with other triangles are removed via edge collapse (for OE_L) or edge insertion (for IE_L).

Ideally, the boundary matching algorithm advances sequentially along the boundary in an effort to reduce the number of unmatched triangles. This methodology is effective; however, the disadvantages of this algorithm include an inability to prevent undesirable matchings. These undesirable matchings often result in degenerate quads with three or more vertices on the domain boundary, which can be difficult to correct with quality improvement operators, i.e., smoothing. Typically, degenerate quads are a product of matching two adjacent OE_L triangles, which collectively have at least two edges on the boundary domain. Given that this algorithm performs minimal changes to the edges and vertices of the mesh, it will be difficult to alter the algorithm to prevent undesirable side effects such as degenerate quads and remaining triangles. Figure 4.5 illustrates the boundary algorithm on a small example domain.

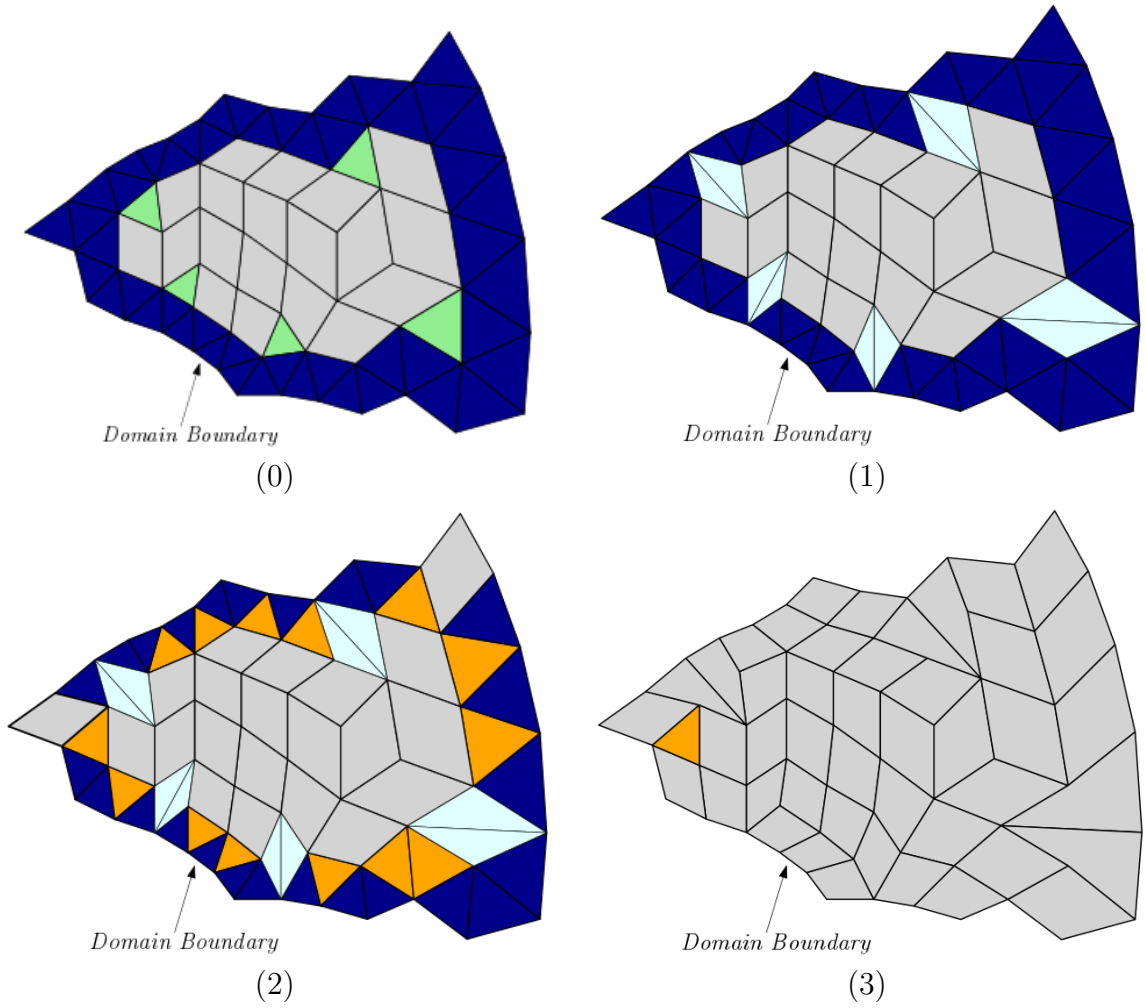


Figure 4.5: Example domain's boundary matching process, steps *0* - *3*. Blue and green are the domain's mesh layers, orange are the IE_L of the boundary layer, cyan are the edge-flipped RE_L and their neighbors on the boundary, and gray are matched pairs of triangles, i.e., quads.

Chapter 5: Post-Processing By Layer

With the matching process concluded, it is important to address any non-ideal topological circumstance of the quadrilateral elements. Some methods for improving the quality of quads often involve an assortment of recombination operations for many different circumstances. Despite the premise of ignoring quality in order to form quadrilaterals, QuADMESH+ currently employs only two of these operations in order to show that even minimal attention to quality improvement can result in high-quality meshes. The two basic and most common recombinations for non-ideal quads are *Doublet Collapse* and *Quad Vertex Merge* [17, 22, 28].

5.1 Quad Quality Improvement Operations

The Doublet Collapse operation is initiated when a *concave quad* exists in the mesh where the concave edges of that quad share a vertex of valence-2. A quad is concave when the angle between two of its edges exceeds 180 degrees. This quad is removed by collapsing the two edges that form the concavity. A method for identifying a concave quad is by searching for non-boundary vertices in the mesh that are valence-2. Figure 5.1 shows how a concave quad can be formed in the matching/merging process and subsequently be recombined into a convex quad.

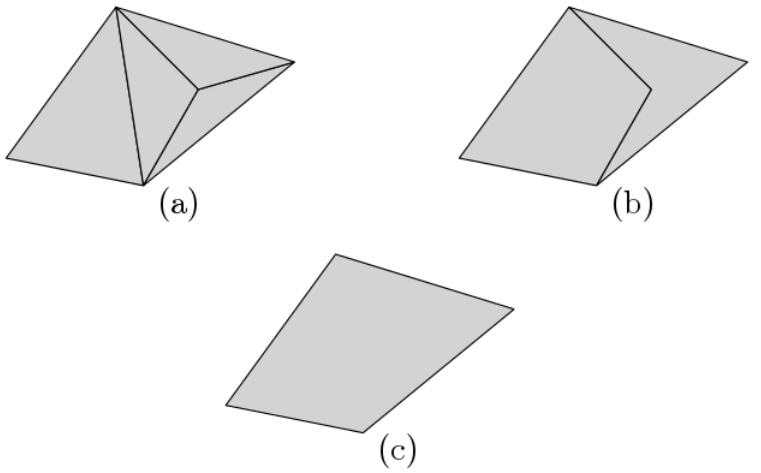


Figure 5.1: Formation of a concave quad (a to b) and removal of a concave quad via Doublet Collapse (b to c).

It is worth noting that any possible matching involving only the triangles show in Figure 5.1 would have inevitably resulted in a concave quad because of the valence-3 vertex. A combination of the original triangular mesh topology and of inter-layer matchings often causes quads to form that are removed by Quad Vertex Merge. It is conceivable that a more sophisticated matching process and/or pre-process valence control of the triangular mesh could prevent either of these operations from being necessary.

Doublet Collapse is a necessary operation because it removes concave quads, which cause numerical instability. Unlike Doublet Collapse, Quad Vertex Merge generally does not remove concave quads in the mesh. Instead, Quad Vertex Merge serves as a valence control that improves the gridded aesthetic of a quad mesh. Quads that are candidates for Quad Vertex Merge are those with diagonals who's vertices are both of valence-3. Similar to the concept of triangle strips, Quad Vertex Merge removes

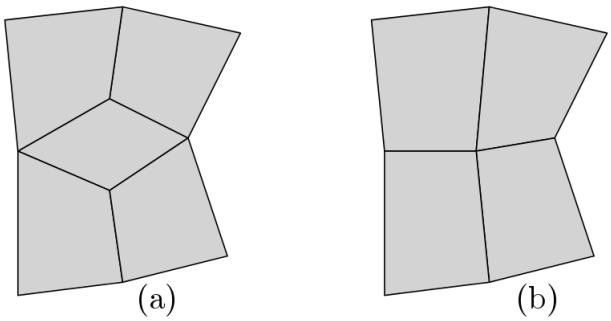


Figure 5.2: Removing a quad via Quad Vertex Merge.

quads that disrupt a clear topological direction of a neighborhood of quads and in effect creates a clear direction for a strip of quads. The application of Quad Vertex Merge results in a set that more resembles a structured mesh, i.e. gridded. Figure 5.2 illustrates the Quad Vertex Merge operation.

5.2 Procedure

Given that five quads define a set that is eligible for the Quad Vertex Merge operation, it is often the case that one of those five quads may be a part of another set of five quads that are also eligible. Performing Quad Vertex Merge on one of these sets may affect the eligibility of another. Figure 5.3 shows an example of how the sequence of applying Quad Vertex Merge to multiple sets of quads can affect the resultant sets. QuADMESH+ attempts to address this problem by devising a method for prioritizing certain sets of quads for quality improvement.

As stated previously, the application of Quad Vertex Merge has the consequence of creating *quad strips*. Much like the Mesh Layer matching process of completing matchings within some Layer L before advancing to Layer $L-1$, the QuADMESH+

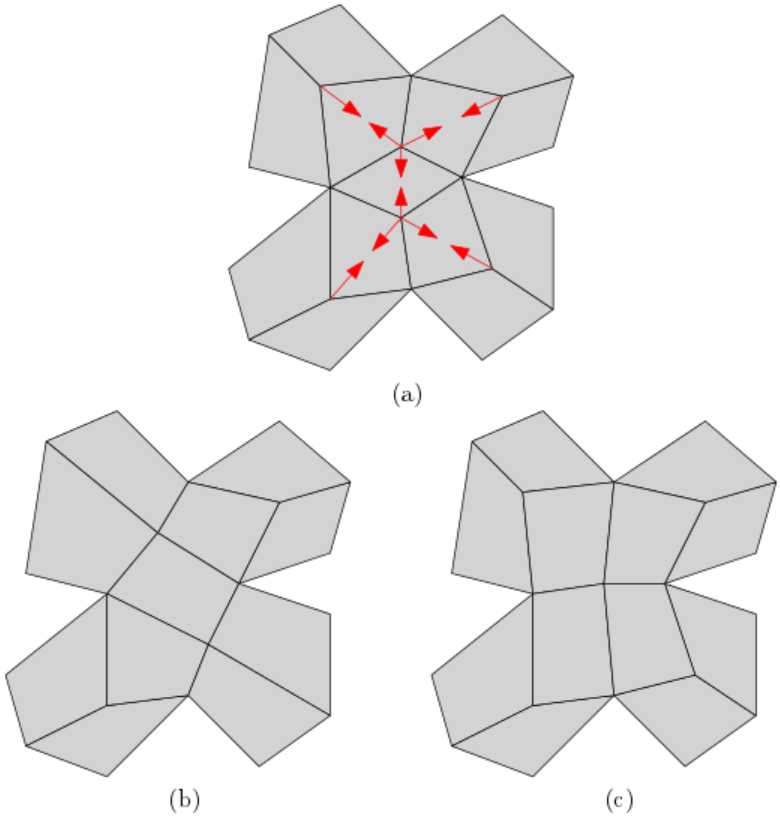


Figure 5.3: Two possibilities for the Quad Vertex Merge Operation on an example sub-domain. *Red lines with arrows* in *a* indicate a possible Quad Vertex Merge and the location of the merged vertices.

post-processing procedure applies Quad Vertex Merge and Doublet Collapse to the quadrangular mesh by layer. The post-processing procedure differs from the matching process by beginning in Layer 1 and advancing inward toward Layer L.

Recall that triangles resolve the geometry of a domain boundary better than quadrilaterals. Consequently, quadrangular meshes typically have lower-quality elements closer to the boundary than triangular meshes. Therefore, the heuristic regime of this post-processing procedure is chosen because it prioritizes sets of quads closer to the boundary for quality improvement. Note that IE_L quads do not share edges with the outer boundary of the layer that it resides in, so preference is given to IE_L for post-processing operations. The steps for the post-processing procedure are as follows:

1. Apply operations to all eligible IE_1 and then to all eligible OE_1.
2. As the topology and mesh layers information changes, update mesh layers accordingly and continue Step 1 until all possible operations have been exhausted.
3. Advance to Layer $L + 1$.
4. Repeat Steps 1 - 3 until no eligible quads remain.

Low-quality quads occurring at the domain boundary is not an uncommon occurrence. Many operations recombine these low-quality quads with neighboring quads via edge flips instead of adding/removing boundary vertices [22]. These operations are ideal; however, the current algorithm has not yet implemented them (see Section 7.1). Instead, a removal operation is used to remove low-quality quads on the boundary. Quads selected for this operation must have an interior angle greater than 134

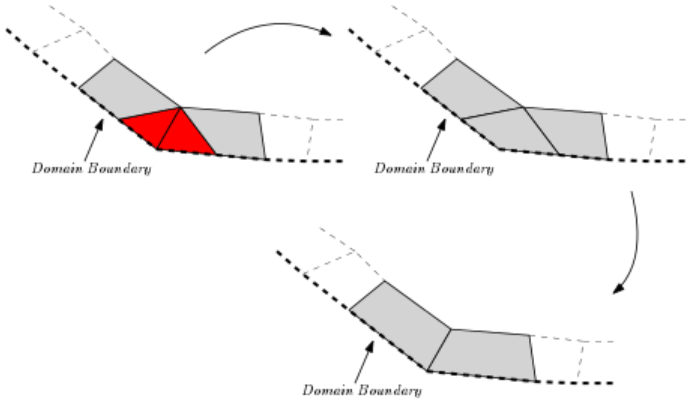


Figure 5.4: An example operation that removes low-quality quads formed on the domain boundary by the matching of two OE_L. These low-quality quads are removed from the mesh and their neighbors are adjusted to maintain the original domain boundary definition.

degrees that is defined by two boundary edges. Any angle of a quad that is greater than 134 degrees has an equiangular skew of 1.0, which is equal to an element quality of $q = 0.0$ (refer to Equation 1.1). While QuADMESH+ does allow for degenerate quads to form following the matching process, it also relies on a smoothing algorithm to correct those degenerate quads. The ability to smooth a quad becomes increasingly difficult when a quad has multiple edges on the (constrained) domain boundary. Therefore, these quads are removed from the mesh and the geometry of their neighbors is altered in order to maintain similar boundary definition. Figure 5.4 illustrates this operation. Note that a degenerate quad is the result of matching two adjacent OE_L. Future work could investigate a new heuristic that prevents this occurrence.

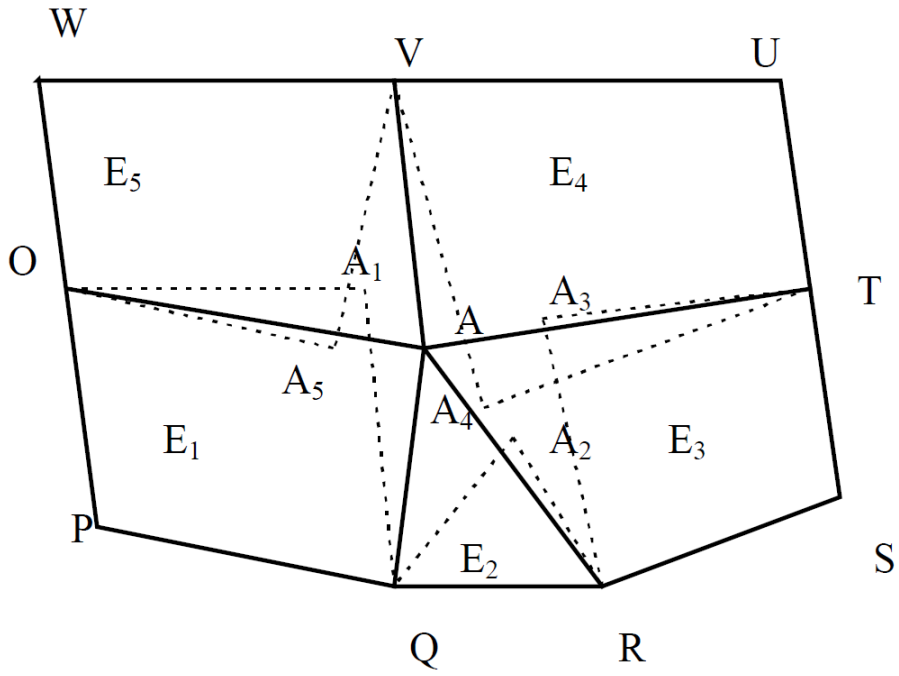


Figure 5.5: A finite element method implementation of element smoothing. *Dotted* lines indicate the ideal shape of a particular element E and, consequently, the ideal location of Vertex A for each respective element, i.e., A_1, A_2, A_3, A_4, and A_4.

5.3 Smoothing

With all possible Quad Vertex Merge and Doublet Collapse operations exhausted, the smoothing of the mesh is the final step in the QuADMESH+ algorithm. The process of smoothing a mesh is based on a simple premise: move vertices along the x and y axes such that the angles of triangular elements shift to 60 degrees and angles of quadrilateral elements shift to 90 degrees. Since this goal is not always obtainable for every vertex at once, many smoothing algorithms shift vertices of a mesh in an iterative fashion so that it may make adjustments to vertex-shifting as the algorithm advances [33]. This can be time consuming because it requires stopping criteria to terminate.

Instead, QuADMESH+ uses a non-iterative smoothing operation which assembles a global matrix of solvable equations using only the connectivity of each element in the mesh and the coordinates of the boundary vertices [2]. These equations describe the possible location of some vertex shared by multiple elements. Averaging these coordinate locations gives the final location of that vertex (see Figure 5.5). This is done for every vertex in the mesh. When solved, this finite element method smoother gives the final coordinates of each vertex in the mesh. The result is a very smooth and gridded quadrangular mesh.

Chapter 6: Analysis of Test Cases

Several different hydrodynamic domains of various sizes are meshed with triangular elements via ADMESH+ and then converted into quadrilateral elements using the new QuADMESH+ algorithm. These domains offer many unique characteristics and serve as ample tests for the capability of QuADMESH+. Several features worth noting are complex domain boundaries, a varying number of internal boundaries, abrupt transitions of curvature along the domain boundaries, the presence of local features, and bathymetry inputs. The original triangular mesh and final quadrangular mesh are compared visually and quantitatively. All informative data (mesh size, quality, valence, etc.) quantitative data (computational time duration) of each test case can be found in Appendix A.

6.1 Test Case 1: An Artificial Domain

Instead of local features or bathymetry inputs, this artificial domain only has unique boundary characteristics and is designed to have multiple interior boundaries with abrupt transitions between low and high curvature. Additionally, non-constant curvature is present on some of the domain boundaries. The purpose of this design is to create many instances of self-intersecting mesh layers without the need for a large

and complex mesh domain. The beginning phases of this research heavily relied on this test case.

The original mesh contains 5,312 triangles defined by 2,806 vertices while the final quadrangular mesh contains 2,560 quads defined by 2,711 vertices (Figure 6.1). The quality of the quadrangular mesh – while not nearly as good as the triangular mesh's average of 0.89 – is an acceptable 0.77 and within the range seen in Figure 3.14. The minimum element quality is 0.11, implying that further quad quality improvements on the boundary are needed.

A recurring theme for many of these test cases is the existence of vertices in the quadrangular mesh with valence exceeding 4. Ideally, the average valence of a quadrangular mesh should be slightly less than 4 because boundary vertices typically have a valence of less than 4, which brings down the global mesh average to 3.8. The valence of multiple vertices in Test Case 1 exceeds 4, with some being as high as 7. This is an issue for all of the test cases examined in this research and future work should be devoted to developing heuristics which prevent or correct such high valence.

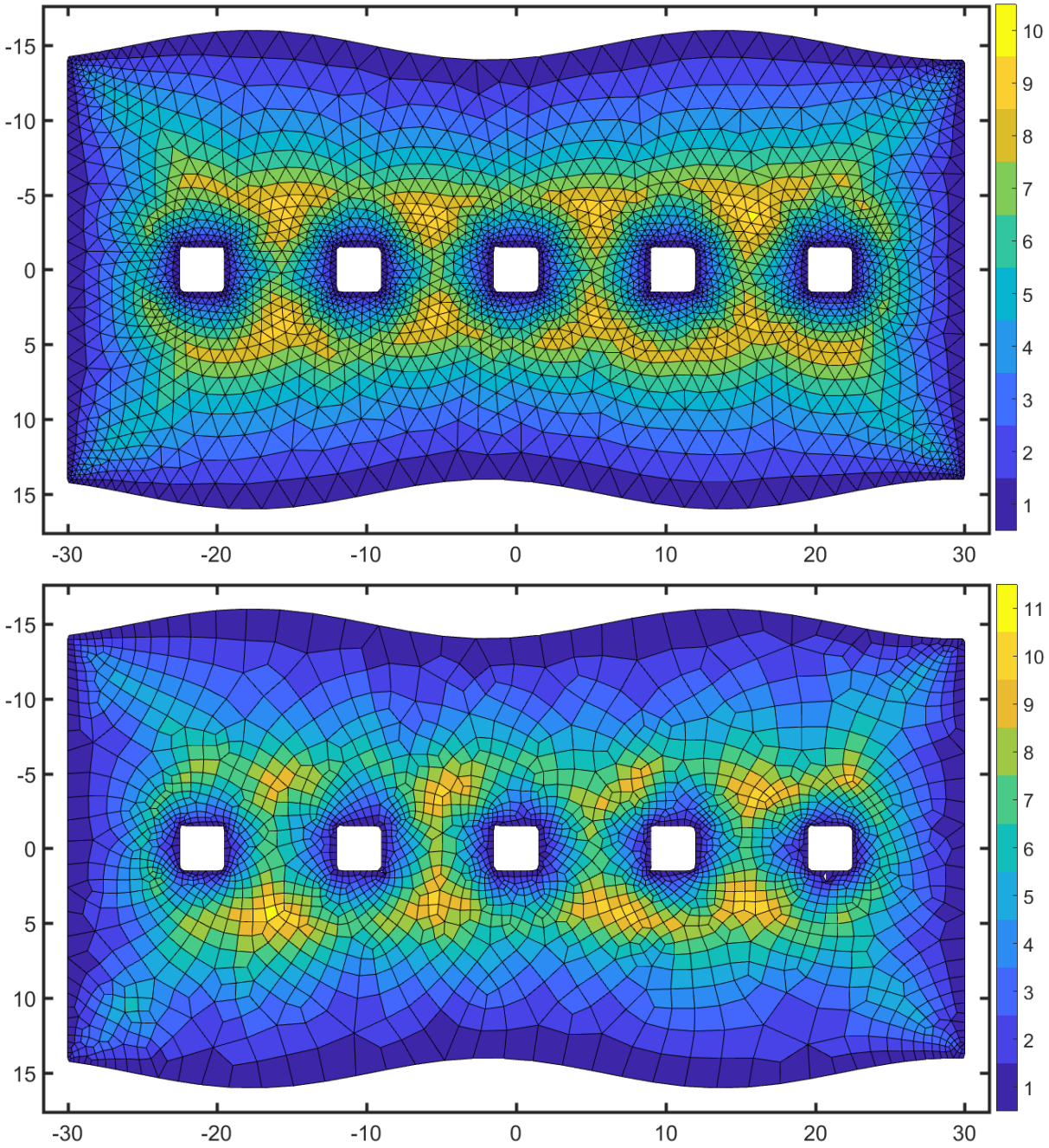


Figure 6.1: Mesh Layers for Test Case 1: An Artificial Domain. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh.

6.2 Test Case 2: Wetting and Drying

The second test case meshed by QuADMESH+ is a *Wetting and Drying* overland domain. These domains often experience interactions with fluid that alternate between "wet" and "dry" [3, 7]. Developing meshes for these domains is more involved because they are so-called "moving boundary problems", i.e., the wet and dried regions of the domain can change. Test Case 2 provides a worthy comparison with Test Case 1 due to their similar sizes and boundary complexities.

The original triangular mesh contains 4,978 triangles defined by 2,716 vertices while the final quadrangular mesh contains 2,468 quads defined by 2,695 vertices (Figure 6.2). The gap in mesh average quality between the two meshes is smaller than in Test Case 1, with the triangular mesh average quality of 0.90 still besting the quadrangular mesh average quality of 0.82 but not by much. The average and maximum valence of the quadrangular mesh are lower than in Test Case 1 too, computed as 3.7 and 6, respectively. Again, the minimum element quality of the quadrangular mesh (0.21) indicates a necessity for further quad quality improvement operators. Still, the mesh is of notably high-quality.

Note that the quadrangular mesh has a slight boundary alteration near (x, y) coordinate $(0, 5e^4)$, a result of the aforementioned quad removal function for low-quality quads on the domain boundary.

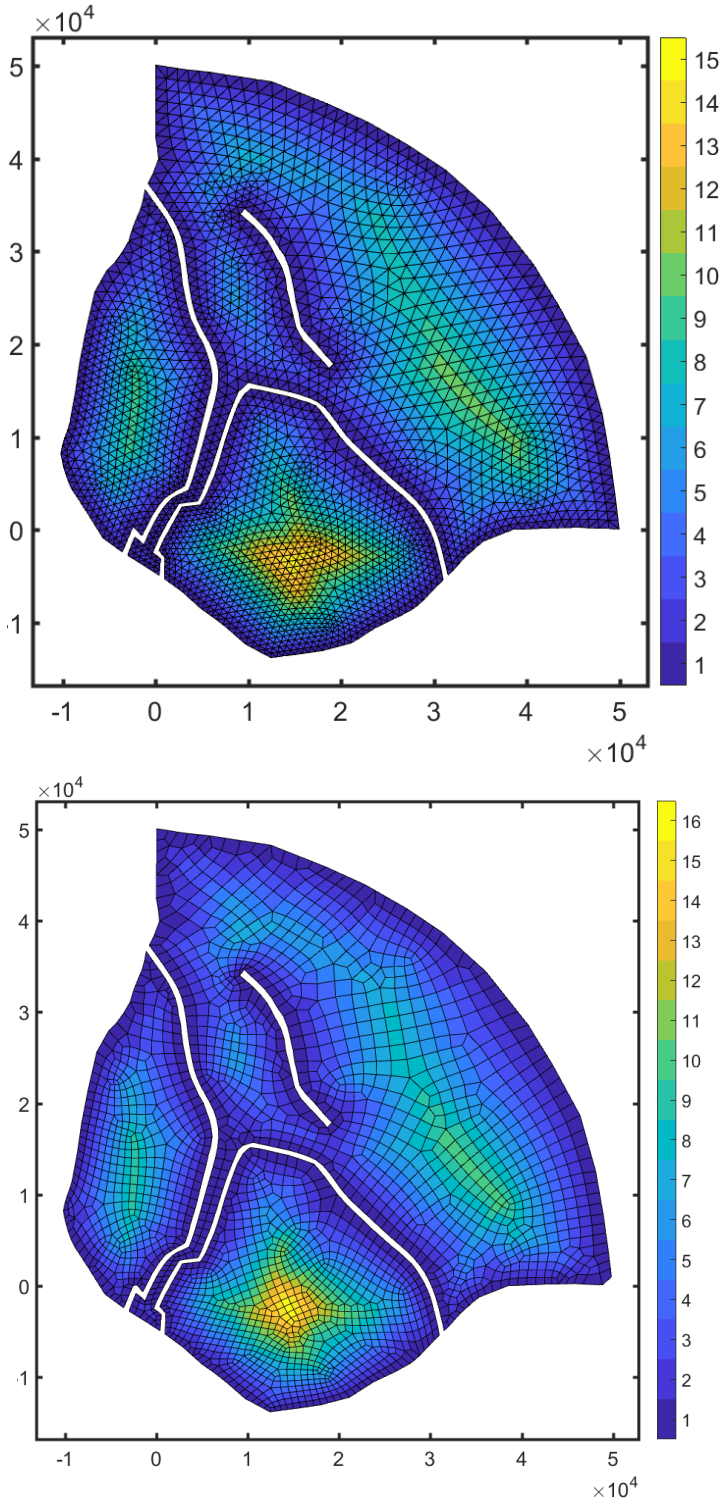


Figure 6.2: Mesh Layers for Test Case 2: Wetting and Drying. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh.

6.3 Test Case 3: Italy

The third test case meshed by QuADMESH+ is over a local domain surrounding the Italian Peninsula and the island of Sicily. This domain combines the ADMESH+ gradation feature ($g = 0.20$) with the presence of several interior domain boundaries and an open ocean boundary (Figure 6.3).

Markedly larger than the previous two test cases, he original triangular mesh for Italy contains 24,417 triangles defined by 13,005 vertices. The triangular mesh average quality is 0.78, with the minimum element quality in the mesh being 0.10. A quadrangulation of this mesh by QuADMESH+ contains 11,926 quads that are defined by 12,699 vertices. The final quad mesh average quality of 0.72 is close to the triangular mesh quality, which continues the trend seen in Test Cases 1 and 2. The minimum element quality is 0.02, the average vertex valence is 3.8 the maximum vertex valence is 6. These results, while promising, again reinforce the necessity for quad quality improvement. In particular, all of the real-world test case domains have a very low minimum element quality. However, these instances of exceptionally low-quality quads are outliers relative to the rest of the mesh, as seen in Figure 6.4. Note that the quality of quads tends to decrease on average for layers closer to the domain boundary, with the lowest average typically located in the boundary layer (Figure 6.5). For each of the real-world test cases, the range of element quality tend to increase and the extent to which element qualities are outliers tend to increase in the layers near the domain boundary. The boundary treatment algorithm should be improved and additional quality improvement operations should be implemented to prevent this.

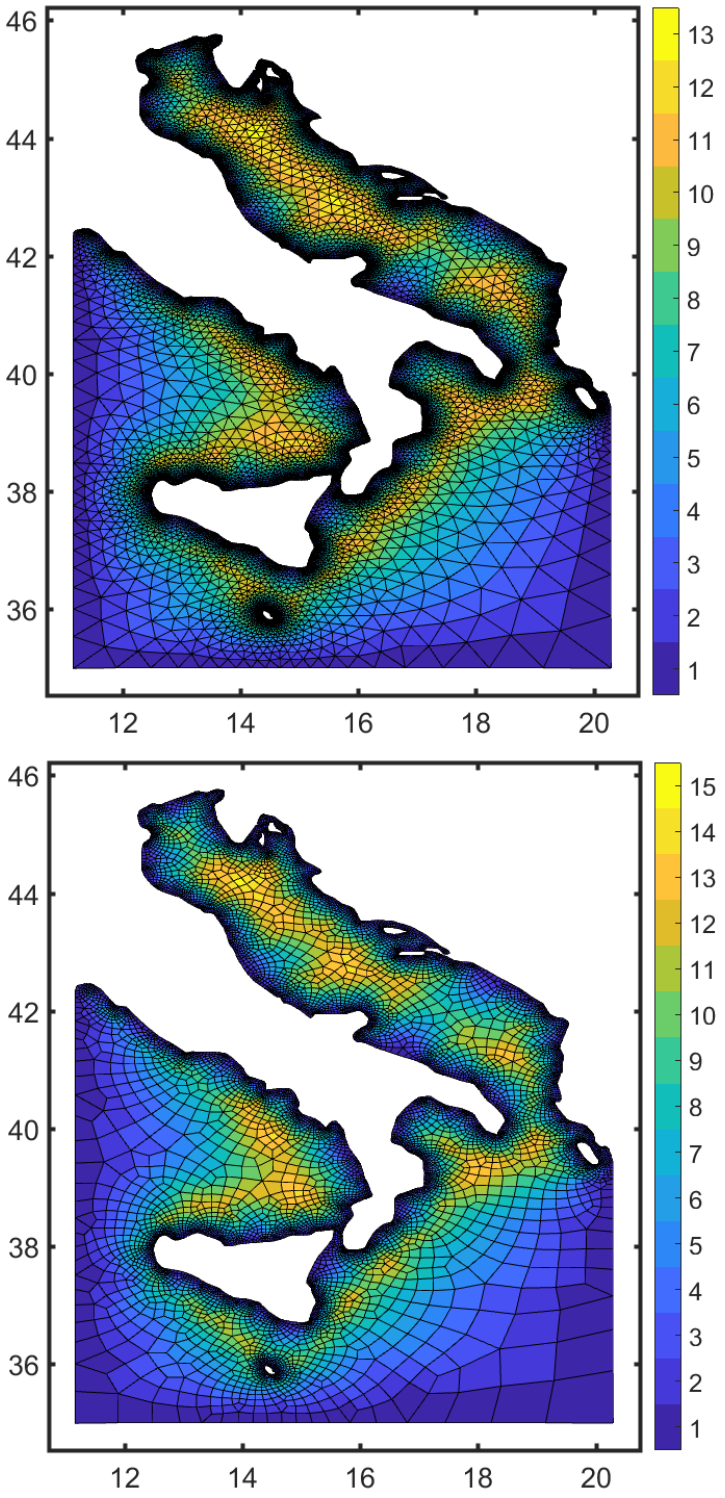


Figure 6.3: Mesh Layers for Test Case 3: Italy. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh.

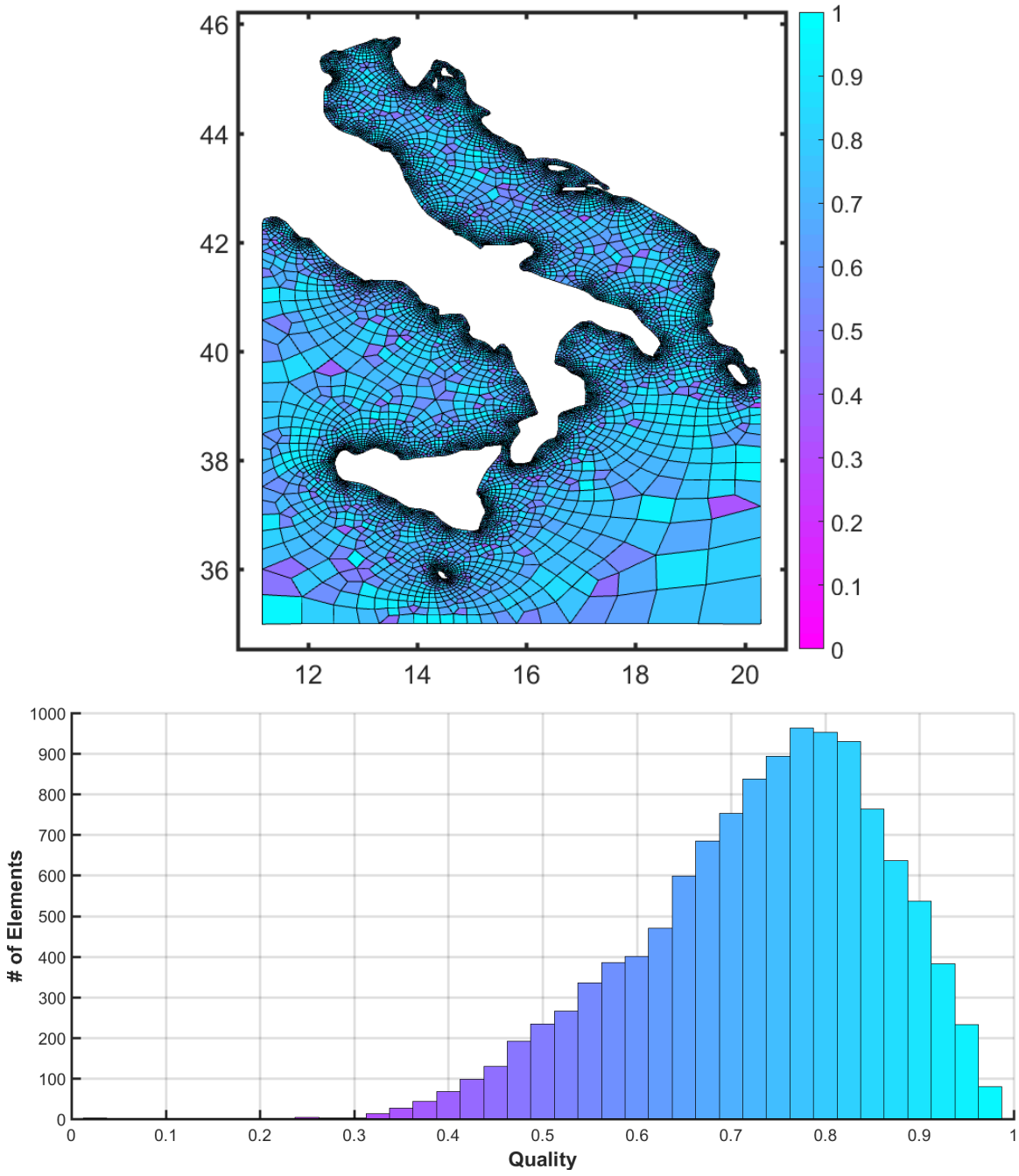


Figure 6.4: Quad quality colormap and histogram for Test Case 3: Italy. Note that the minimum element quality ($q = 0.02$) of the mesh is an outlier.

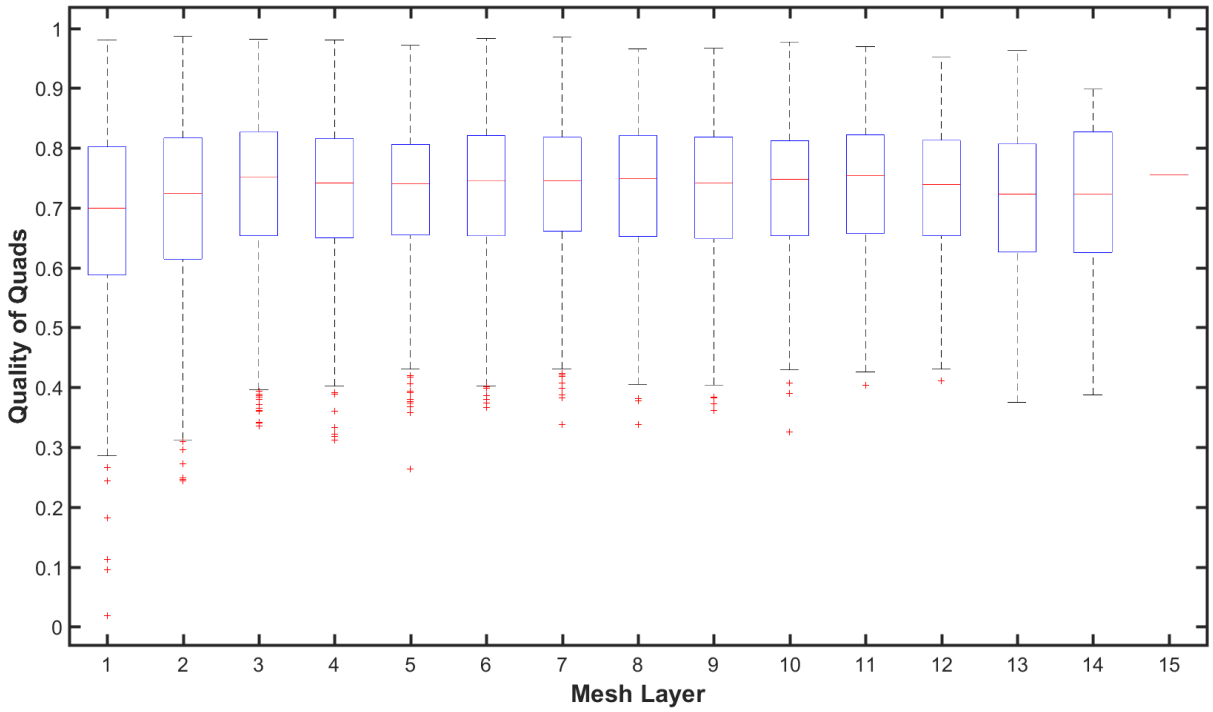


Figure 6.5: Boxplots of the element quality distributions for each layer of Test Case 3: Italy. The red horizontal line is the median element quality of each layer, the blue box indicates the first (*bottom*) and third (*third*) quartile for element quality of each layer, the *dashed* lines indicate the min. and max element quality of each layer (not including outlier elements), and the red markers indicate outlier element qualities of each layer.

6.4 Test Case 4: Lake Erie

The fourth test case is for a domain that is often used for numerical approximations to applications of the shallow water equations, specifically the transport of sediment and algae. In particular, Lake Erie has recently experienced multiple events of large harmful algal blooms (HABs) during the summer months, an event that is extremely relevant due to its hazardous affects on drinking supplies [29]. New endeavors to predict and model the transport of these HABs throughout the Great Lakes have gained traction, therefore development of increasingly efficient meshes are indeed necessary. As such, QuADMESH+ is applied to this domain in order to assist with these efforts.

The ADMESH+ produced triangular mesh for Lake Erie is marginally larger than that of the Italy mesh in Test Case 3. Containing 24,910 triangles defined by 13,266 vertices, the Lake Erie triangular mesh is of high-quality (0.78 average) and its topology resembles that of a typical real-world hydrodynamic domain. The resulting quadrangular mesh contains 12,064 quads defined by 12,857 vertices. This quadrangular mesh is also a comparably high-quality (0.71) to the original triangular mesh and to those of previous test cases, which implies that this algorithm is consistent regardless of mesh topology. However, the minimum element quality again is quite low (0.01). Again, note that this low-quality element are outliers located on the boundary and they are expected given the minimal quality improvement performed on the final quadrangular mesh.

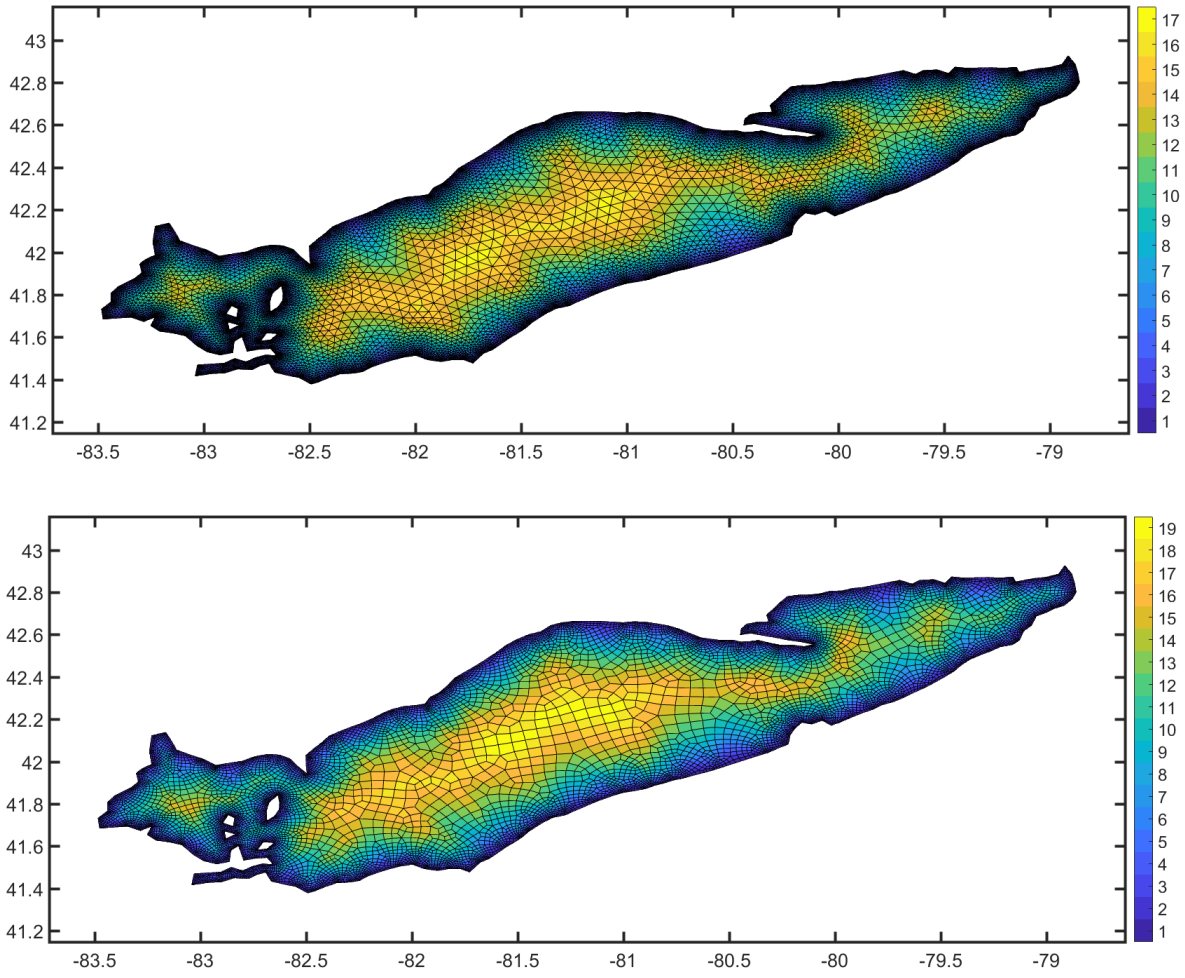


Figure 6.6: Mesh Layers for Test Case 4: Lake Erie. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh.

6.5 Test Case 5: Delaware Bay

The fifth test case examined is for the domain of the Delaware Bay, a region topologically and hydrodynamically similar to many others along the North American Atlantic coast. Physical phenomena such as sediment transport and storm surge are extremely relevant to these types of domains that may require dredging and reliable advanced warning systems for storm events. Applying both triangular and quadrangular mesh capabilities to this domain is a prudent test of this mesh generation technology.

The largest of the five mesh test cases, the triangular mesh for Test Case 5 contains 26,697 triangles defined by 14,449 vertices. The average element quality in the triangular mesh is 0.81, with a minimum element quality of 0.29. The final quadrangular mesh contains 12,908 quads defined by 13,973 vertices. The average element quality is 0.74 and the minimum element quality is 0.03. Again, this element quality is an outlier located on the domain boundary and further evidence that boundary treatment should be improved.

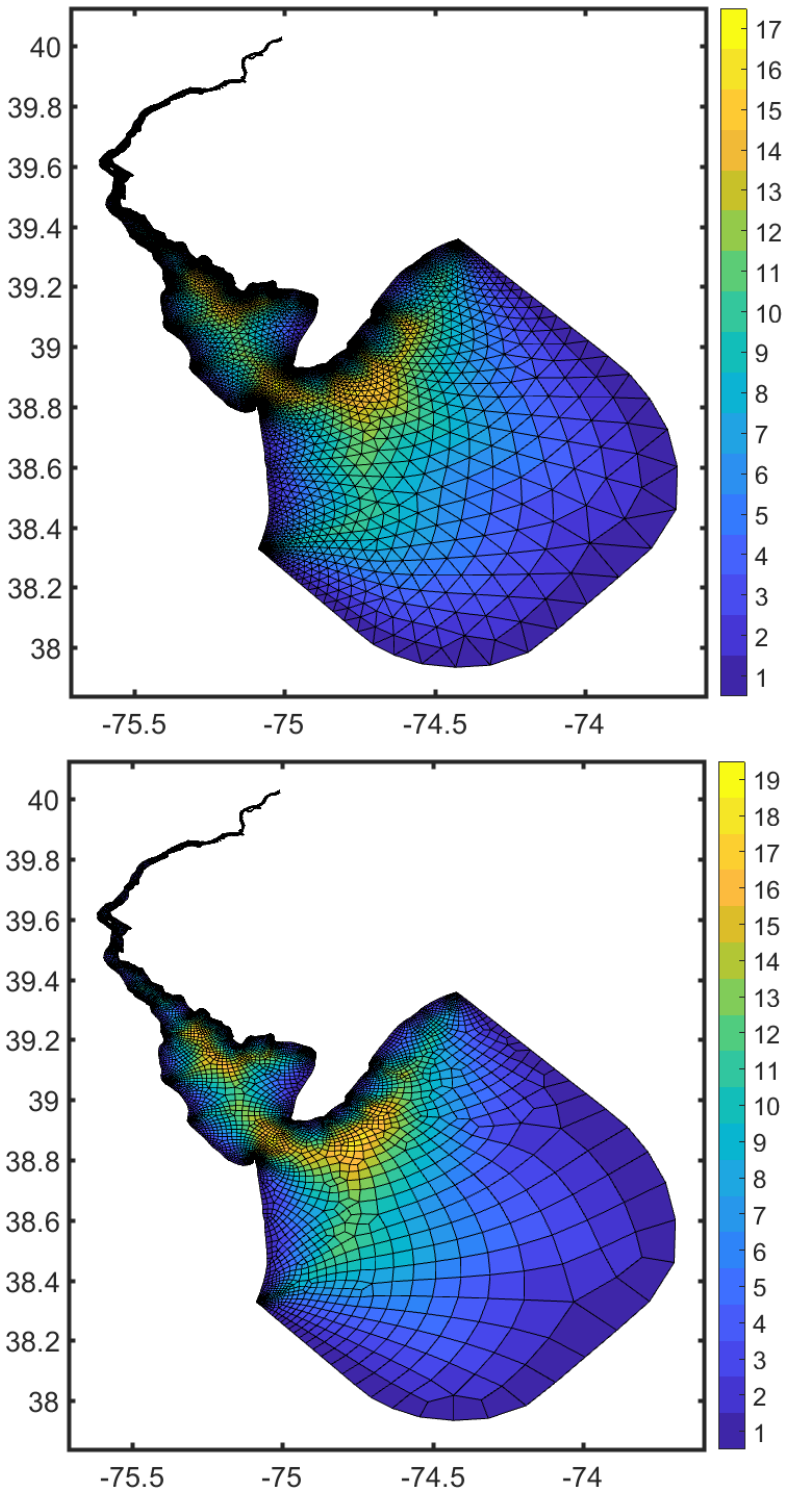


Figure 6.7: Mesh Layers for Test Case 5: Delaware Bay. *Top* is the original triangular mesh and *bottom* is the final quadrangular mesh.

6.6 Mixed-Element Test Cases

Given the trends of exceptionally low minimum element qualities seen in the quadrangular meshes, this research posits that many of the low-quality quads reside near the boundary of the mesh. In part, this is a result of the fact that elements with vertices on the boundary are more resistant to smoothing because boundary vertices are not allowed to move.

An elegant alternative to applying an exhaustive set of quality improvement operations to the quadrangular mesh is to instead use both quads and triangles in a mixed-element mesh. Triangles – with their smoothing-amenable geometry – would occupy regions where high resolution is needed such as the domain boundaries while quads would occupy all other regions, thus avoiding the issue of "unsmoothable" quads on the boundary. For this reason, the resulting mixed-element mesh would be of higher quality than the purely quadrangular mesh.

Under these guidelines, this research created a new, preliminary technique for creating mixed-element meshes. This technique first identifies regions of an original ADMESH+ triangular mesh by element-size, distance to the boundary, or mesh layer. Next, every triangle with at least one vertex located within these identified regions is selected for quadrangulation. What follows is identical to the Mesh Layers Approach for matching triangles to create quads. Figures 6.8 - 6.13 shows the resulting mixed-element mesh for the Western North Atlantic Tidal domain.

These preliminary results show that the average element quality of the mixed-element mesh for a large test case is an improvement relative to the original triangular mesh. Additionally, this paradigm is similar to past proposals for mixed-element meshes, where quads occupy the regions of relatively homogeneous hydrodynamics

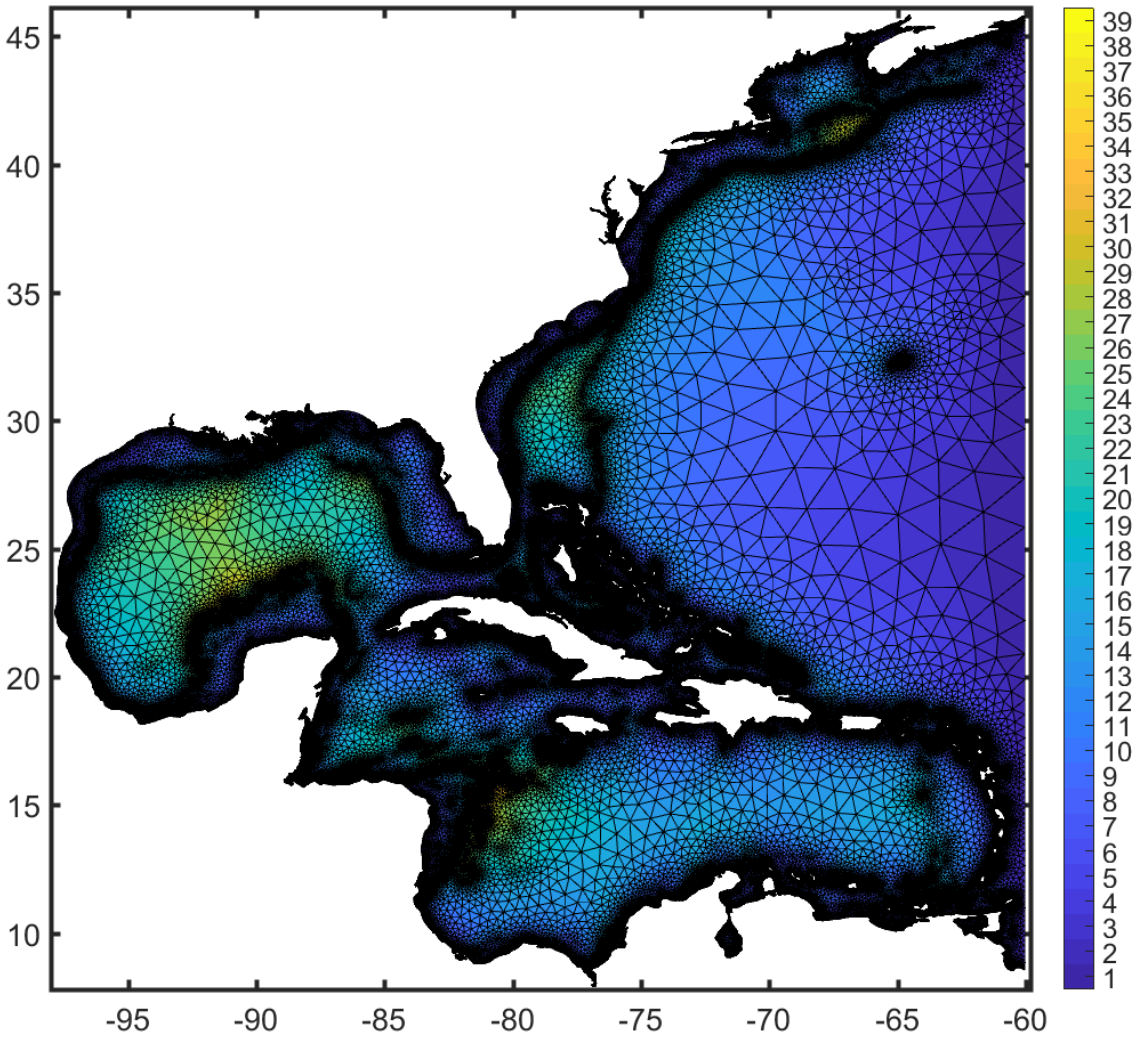


Figure 6.8: The triangular mesh layers (via ADMESH+) for the WNAT domain. The mesh contains 246,186 triangles defined by 127,572 vertices, the average element quality is 0.86, and the minimum element quality is 0.21.

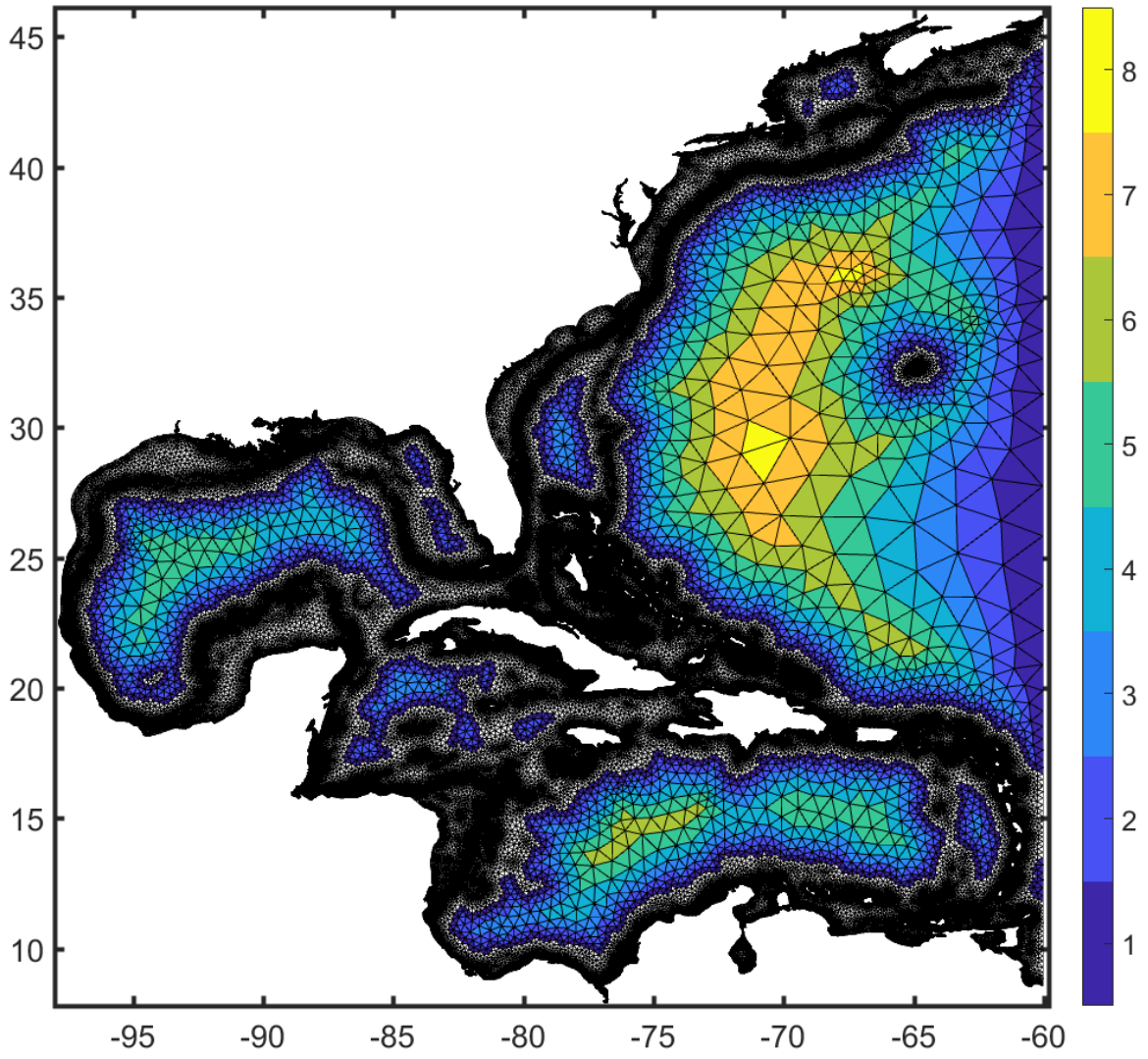


Figure 6.9: Mesh layers of the triangles in WNAT selected for quadrangulation. These triangles were selected because geometry exceeded a target edge size.

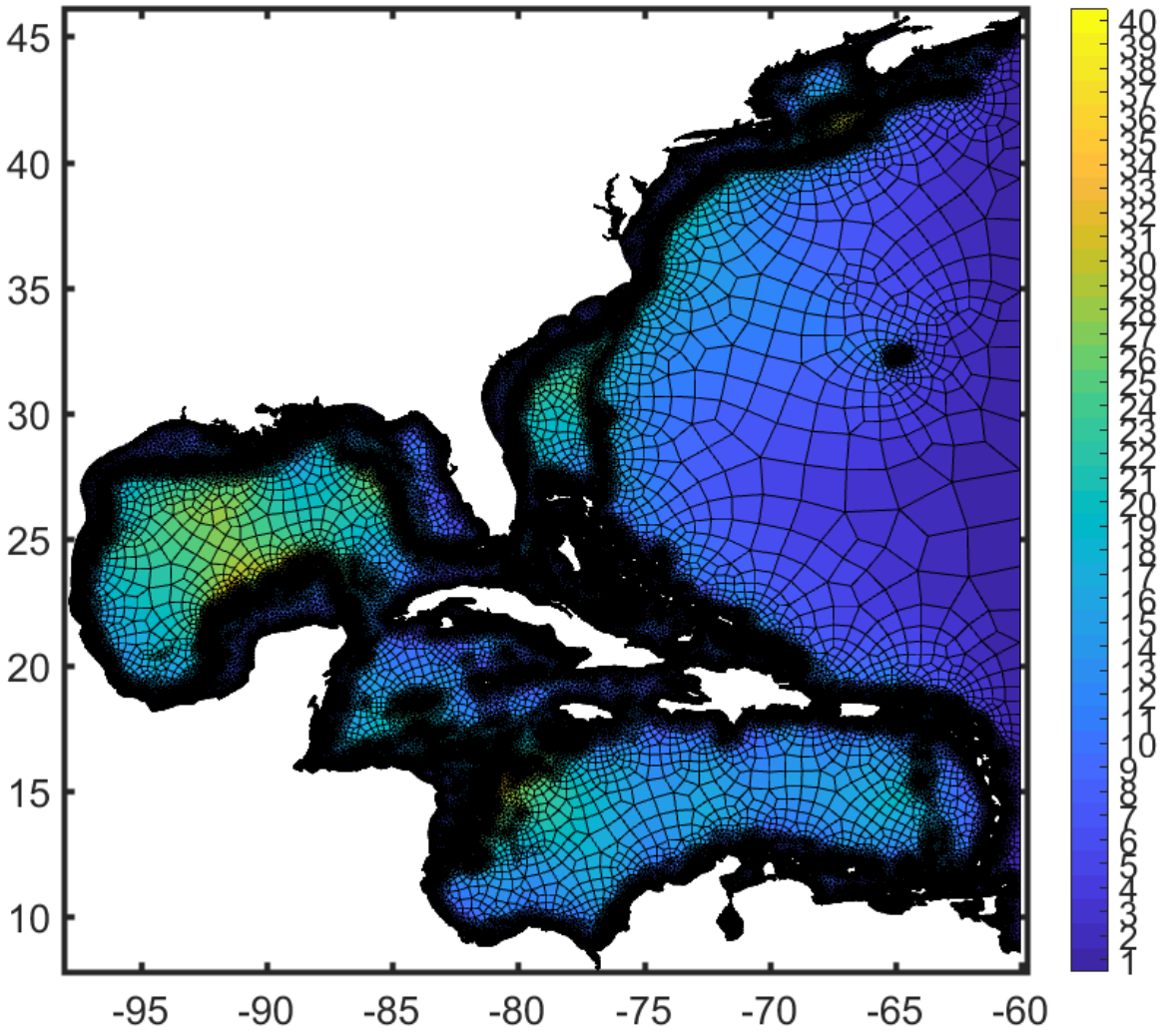


Figure 6.10: The mixed-element mesh layers (via ADMESH+) for the WNAT domain. The triangle-dominant mesh contains 243,416 elements defined by 127,494 vertices, the average element quality is 0.89, and the minimum element quality is 0.21.

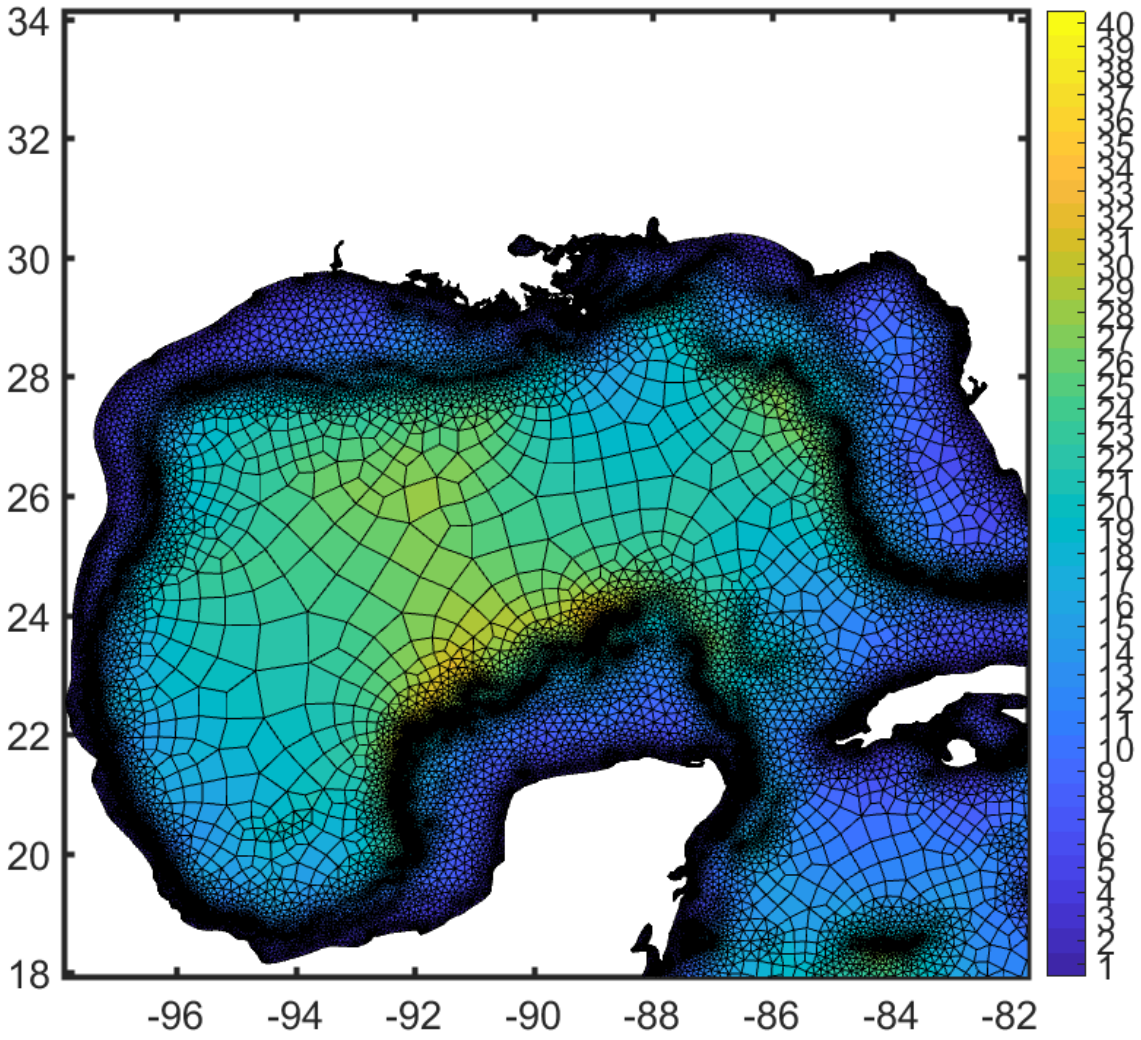


Figure 6.11: A zoomed view of Gulf Coast of the WNAT mixed-element Mesh Layers.

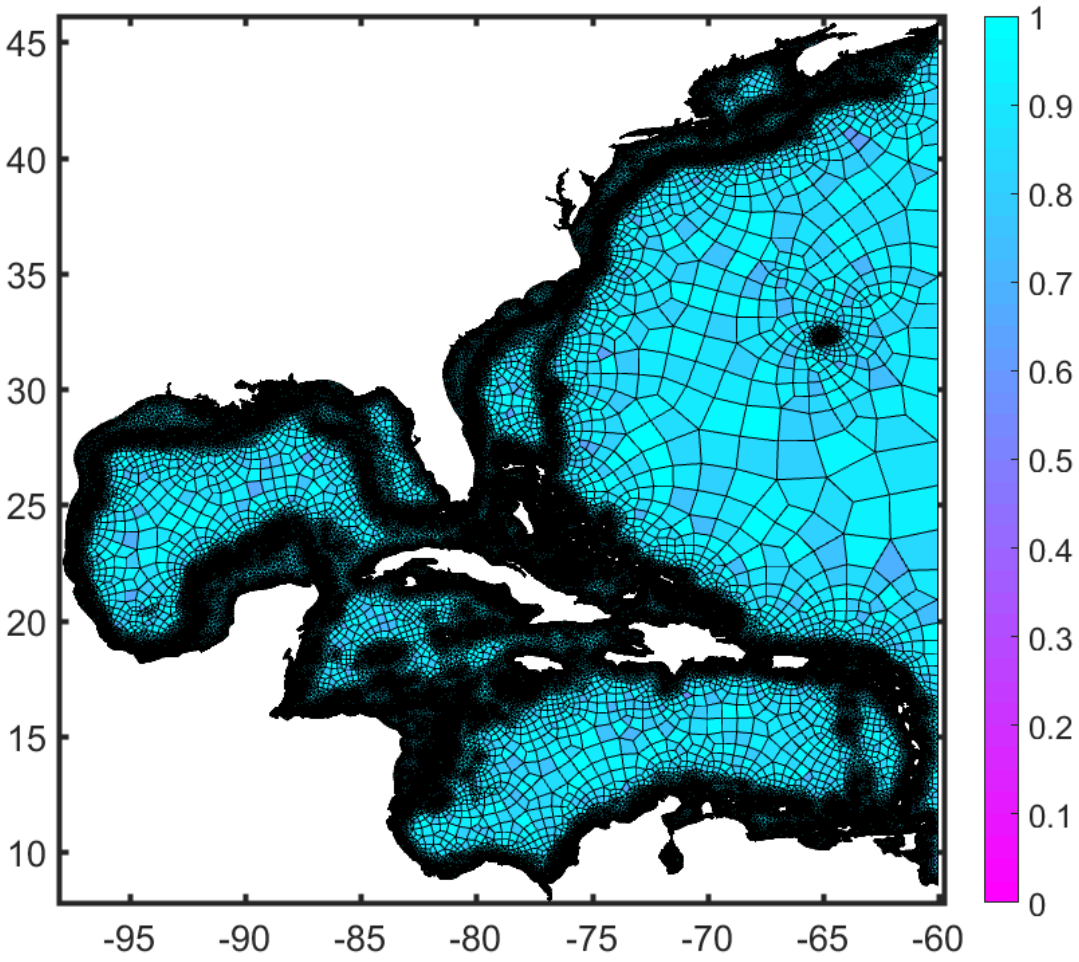


Figure 6.12: Mapping the element quality for the WNAT mixed-element mesh.

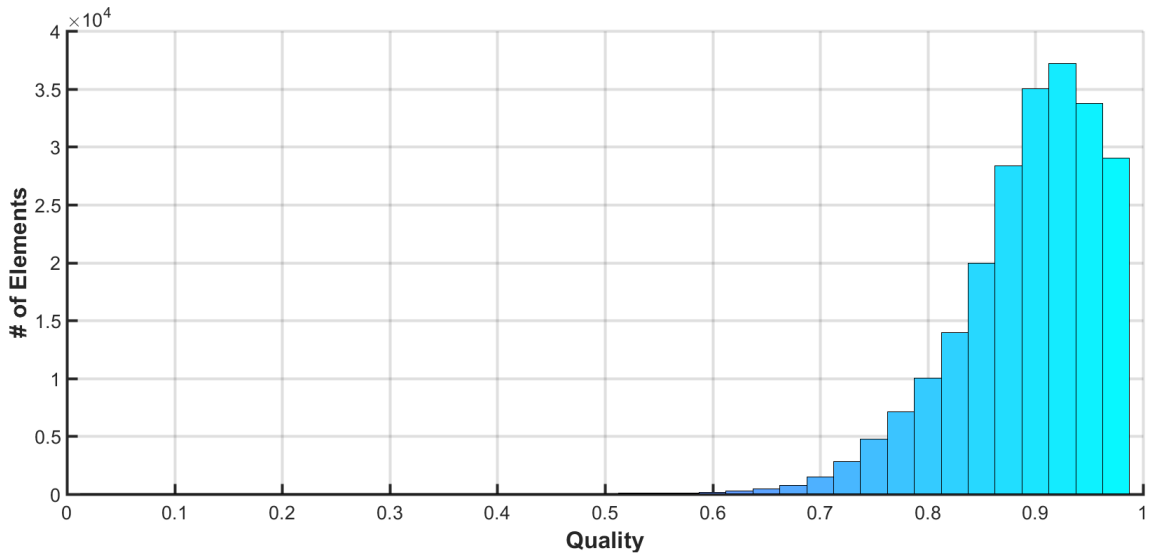


Figure 6.13: A histogram element quality for the WNAT mixed-element mesh. Note that the colors correspond to the colorbar and the colormap in Figure 6.12.

and triangles occupy the rest of the mesh [8, 21, 32]. Due to their similar applications, Test Cases 3 - 5 are also good candidates for mixed-element meshes. The results of applying the mixed-element technique to these test cases are shown on the pages that follow.

The results for the various mixed-element mesh domains all show an improvement in the average element quality and the minimum element quality compared to each respective triangular mesh. Some domains – wetting and drying – are specifically designed for using triangular elements because of the numerical models which use them, such as DG-SWEM [10]. However, using mixed-element meshes would allow triangles to be used for the overland wetting and drying domains and quads elsewhere, thus a redesign of models like DG-SWEM to accommodate quads is not necessary.

The mixed-element technique requires a similar amount of computational time (a time scale typically in seconds for meshes with fewer than 100,000 elements; via MATLAB). The exception to this was the WNAT domain, which required 2 hours to perform a smoothing due to the large number of vertices and elements in the domain. Further research and development of the mixed-element technique as well as parallelization of the smoothing routine would enhance these results. For a table of computational times for each step in the algorithm, see Appendix A.

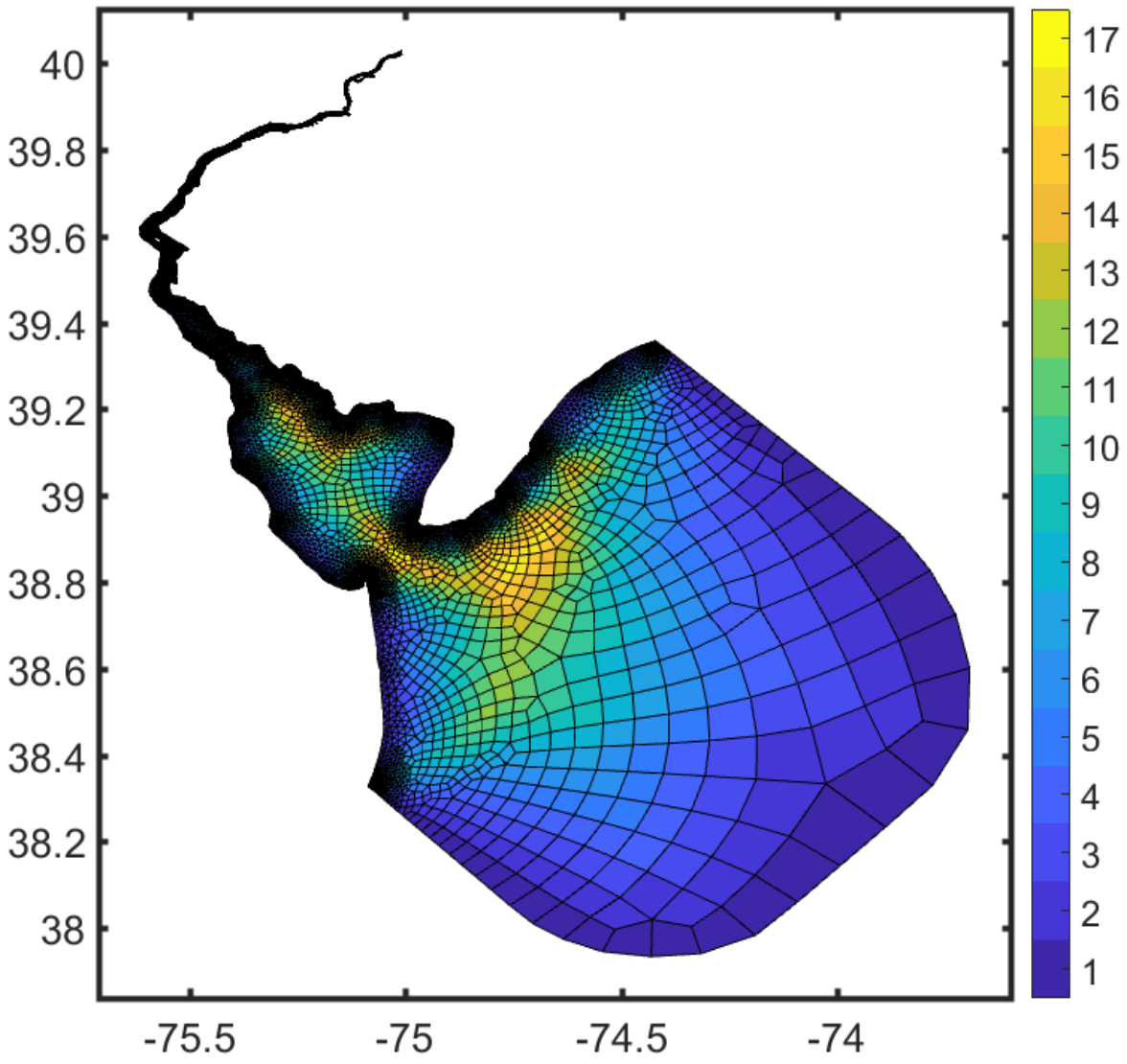


Figure 6.14: The mixed-element mesh layers (via ADMESH+) for the Delaware Bay domain. The triangle-dominant mesh contains 25,600 elements defined by 14,430 vertices, the average element quality is 0.83, and the minimum element quality is 0.34. Identifying triangles in this mesh for quadrangulation was performed by selecting all triangles of a specific edge size or located beyond a specific distance from the domain boundaries.

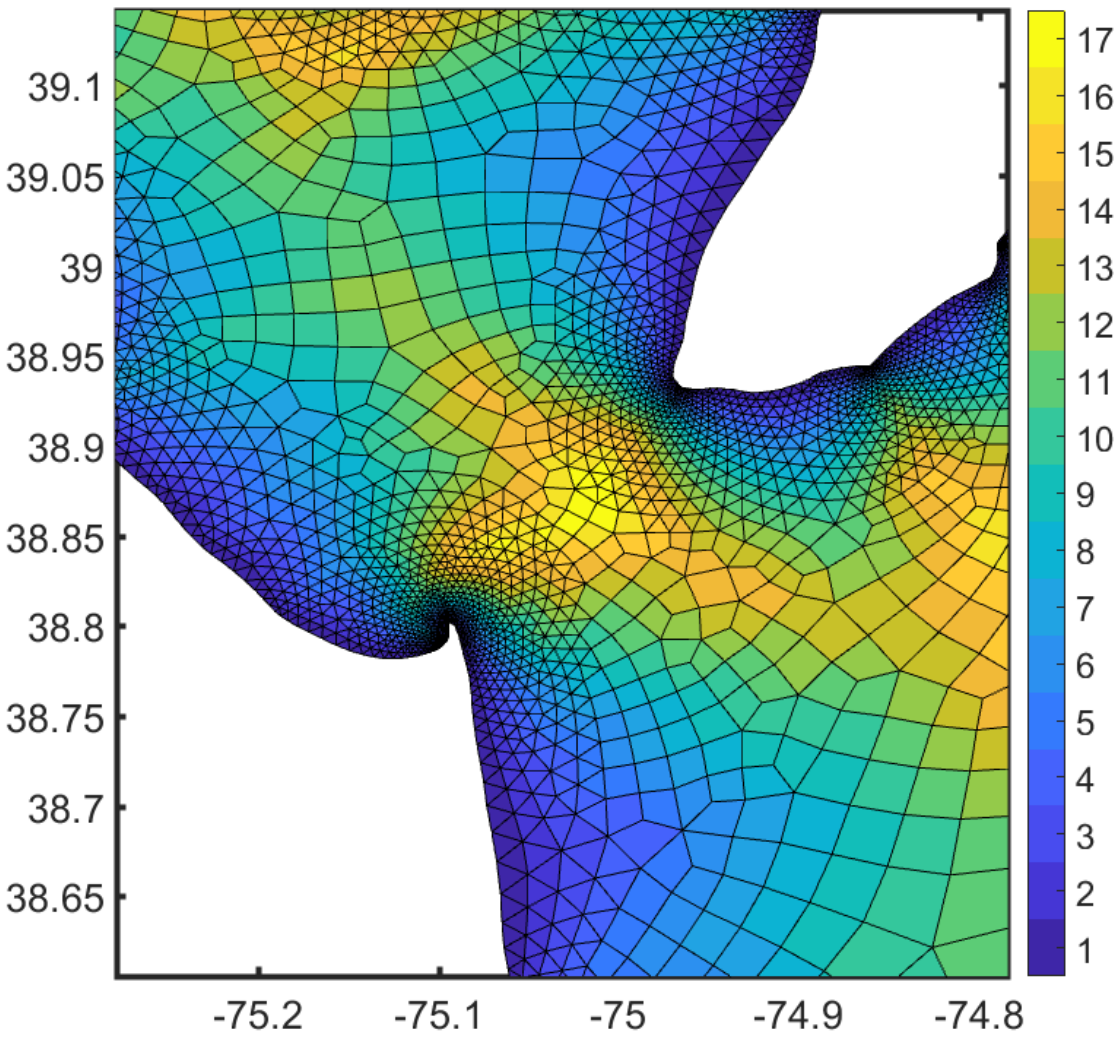


Figure 6.15: A zoomed view of the Delaware Bay mixed-element Mesh Layers.

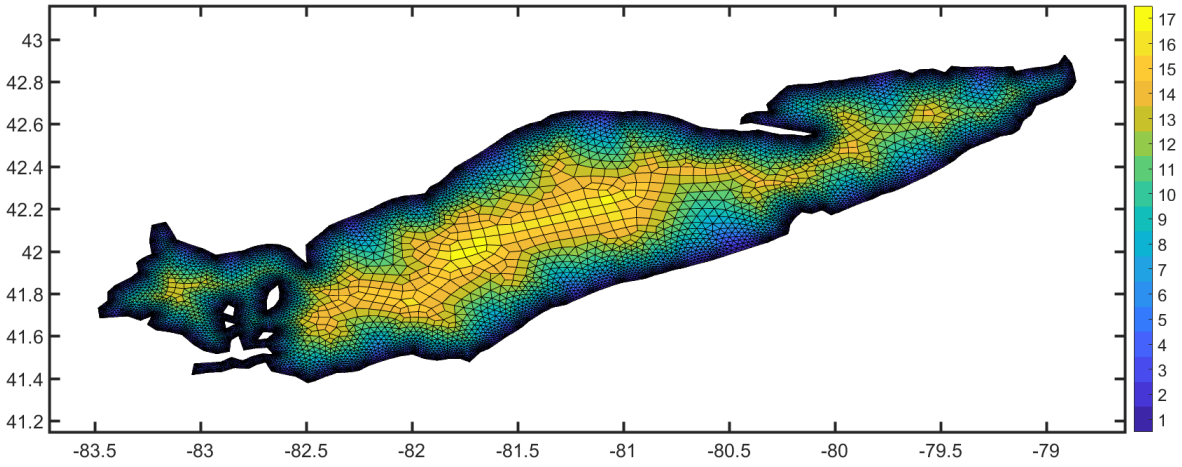


Figure 6.16: The mixed-element mesh layers (via ADMESH+) for the Lake Erie domain. The triangle-dominant mesh contains 24,446 elements defined by 13,257 vertices, the average element quality is 0.79, and the minimum element quality is 0.25. Identifying triangles in this mesh for quadrangulation was performed by selecting triangles within specific mesh layers.

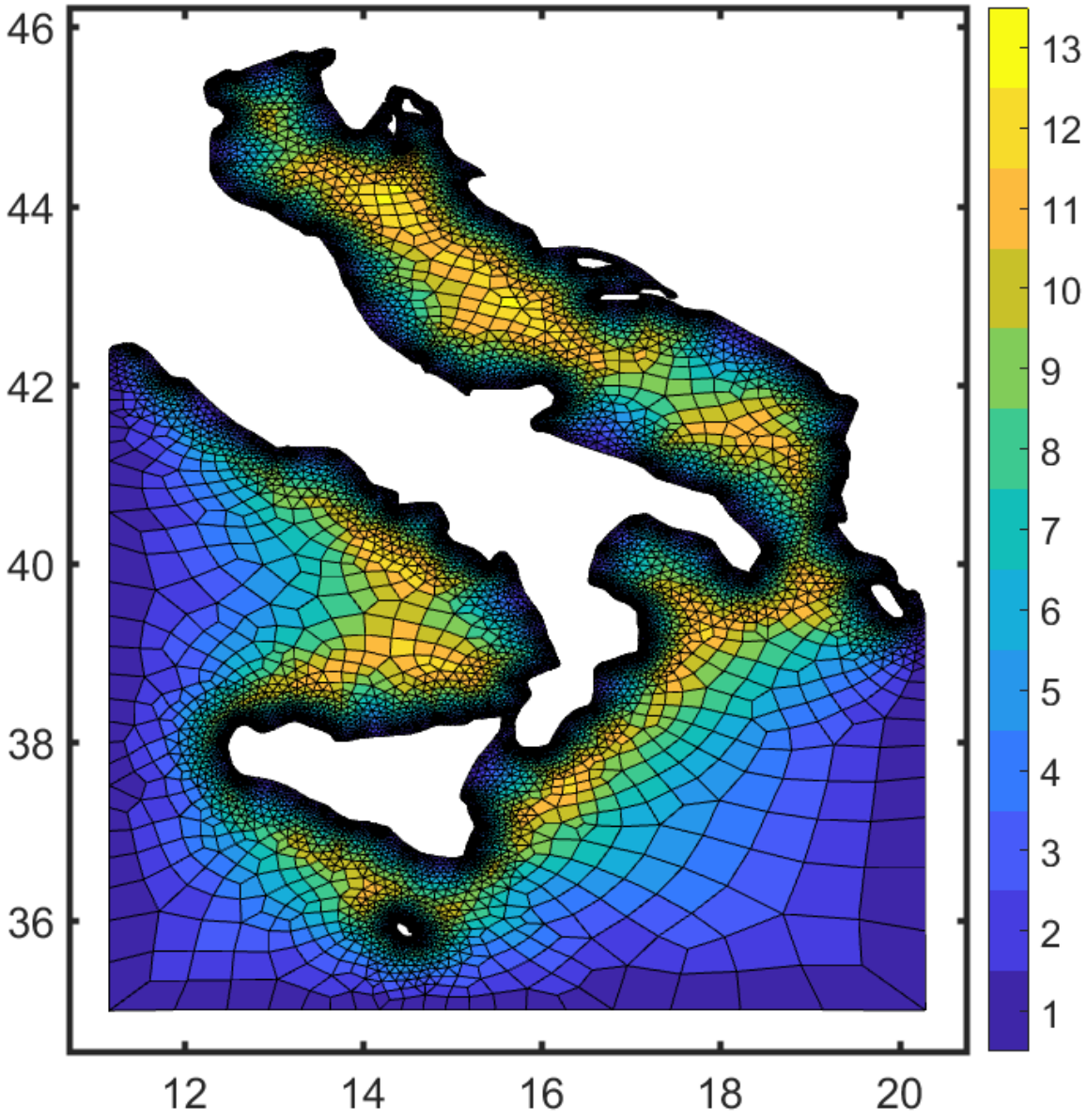


Figure 6.17: The mixed-element mesh layers (via ADMESH+) for the Delaware Bay domain. The triangle-dominant mesh contains 23,648 elements defined by 12,990 vertices, the average element quality is 0.81, and the minimum element quality is 0.09. Identifying triangles in this mesh for quadrangulation was performed by selecting triangles of specific edge sizes.

Chapter 7: Conclusions

The newly-developed QuADMESH+ algorithm that indirectly generates quadrangular meshes for hydrodynamic domains requires minimal user input and performs the process in an automated fashion. By using the Mesh Layers Approach, the innermost regions of a triangular mesh are identified and matched first before advancing outward toward the domain boundaries. Upon completion of the matching process, basic quad quality improvement operators such as Doublet Collapse, Quad Vertex Merge, and a Finite Element Method mesh smoother help create a quadrangular mesh with a high average element quality. The Mesh Layers Approach is easily modified for creating mixed-element meshes, where the advantages of quads and triangles alike work in tandem to create meshes of even higher-quality. Regions of a triangular mesh may be designated for quadrangulation based on various identifiers, such as edge size, mesh layer, distance to the nearest boundary, and regions defining the flow characteristics (direction of flow).

Improvements to QuADMESH+ should include a more efficient mesh smoother, a comprehensive set of quality improvements, vertex-valence control operators, and advancement of general programming strategies and implementation. These improvements alone would improve the average element quality of the mesh without much sacrifice to computational efficiency.

7.1 Future Work

With the preliminary techniques for a mixed-element mesh generator outlined, further development of ADMESH+ and QuADMESH+ is promising. Future work with either algorithm could involve the following:

1. **Providing Better Triangular Meshes for Quadrangulation** – QuADMESH+ would be better served by a few changes to ADMESH+, such as changing the Distmesh initial vertex distribution and smoothing process to create right-triangles instead of equilateral triangles. Doing so could create a triangular mesh with much more walkable mesh layers that simplify the matching process, and possibly lead to higher-quality quads. Another change to ADMESH+ could attempt to minimize vertex valence greater than 6. It has been observed that vertices with valence exceeding 6 generally have more inner-element attachments than those with valence of 6 or less (see Appendix A). Lastly, preventing consecutive outer-elements from forming on the domain boundary in the original triangular mesh could entirely prevent the formation of low-quality quads on the domain boundary. These three suggestions are intended for creating a better quadrangular mesh and are therefore not necessary when only triangular meshes are desired.

2. **Improved Matching Process** – The heuristic decision-making scheme currently employed by QuADMESH+ is generally successful but cannot prevent a triangle from becoming isolated. In order to avoid that, another method for

creating matchings within the mesh layers could be used, such as *Vertex Coloring* [23]. This hybrid method could be implemented within the Mesh Layers Approach to more efficiently and strategically form quads.

3. **Parallelization** – Both ADMESH+ and QuADMESH+ should be parallelized in order to create larger meshes on a more efficient time-scale. Further investigation n potential techniques that accomplish this should is necessary.

4. **Verification of Quad Computational Advantages** – It would be prudent to run numerical experiments to confirm the computational benefits of quads within fully-quadrangular and/or mixed-element meshes as suggested by Wirasaet et al. [32].

Appendix A: Statistics for Test Case Meshes

Table A.1: Computational time of each algorithm step for each test case domain.

Step in Algorithm	Test Case (TC) and Time (sec)								
	TC 1	TC 2	TC 2	$TC\,2_{ME}$	TC 3	$TC\,3_{ME}$	TC 4	$TC4_{ME}$	$WNAT_{ME}$
1. PreProcess	0.56	0.49	0.68	0.49	0.90	0.73	1.02	1.56	3.33
2. Interior Matching	2.68	2.83	14.32	7.41	17.05	3.25	17.16	22.40	83.59
3. Boundary Matching	0.23	0.22	1.08	0.47	1.07	0.41	1.36	0.50	3.43
4. Remove Isolated Triangles	1.10	0.90	4.61	0.87	4.98	0.77	5.23	1.12	6.69
5. Truncate Boundary Quads	0.00	0.53	4.78	0.00	5.58	0.00	6.30	0.00	0.00
6. Doublet Collapse	0.25	0.19	1.74	0.23	1.83	0.17	2.05	0.36	3.91
7. Quad Vertex Merge	0.37	0.22	1.80	1.65	1.94	0.17	2.09	2.05	3.97
8. Remove Unused Vertices	0.31	0.26	1.37	3.90	1.38	4.08	1.65	5.35	210.39
9. FEM Smoothing	3.16	1.45	24.88	46.34	24.24	44.53	31.79	31.05	7512.60
Total	8.66	7.11	55.26	61.36	58.97	54.11	68.66	63.82	7827.91

Table A.2: Mesh details for each test case domain.

Test Case	Element Type	# of Vertices	# of Elements	# of Layers	Quality		Valence		
					Mean	Min.	Mean	Min.	Max.
Test Case 1	Tri	2,806	5,312	10	0.89	0.59	5.7	2	7
	Quad	2,711	2,560	11	0.77	0.10	3.8	1	7
Test Case 2	Tri	2,716	4,978	15	0.90	0.61	5.5	1	7
	Quad	2,695	2,468	16	0.82	0.21	3.7	1	6
Test Case 3	Tri	13,005	24,417	13	0.78	0.10	5.6	1	8
	Quad	12,699	11,926	15	0.72	0.02	3.8	1	6
	Mixed	12,990	23,648	13	0.81	0.09	5.5	1	8
Test Case 4	Tri	13,266	24,910	17	0.78	0.31	5.6	1	7
	Quad	12,857	12,064	19	0.71	0.01	3.8	1	7
	Mixed	13,257	24,446	17	0.79	0.25	5.6	1	7
Test Case 5	Tri	14,449	26,697	17	0.81	0.29	5.5	2	8
	Quad	13,973	12,908	19	0.74	0.03	3.7	1	7
	Mixed	14,430	25,600	17	0.83	0.34	5.4	2	8
WNAT	Tri	127,572	246,186	39	0.86	0.21	5.8	1	8
	Mixed	127,494	243,416	40	0.89	0.21	5.7	1	8

Table A.3: Incidence of IE_L attachments for high and low valence vertices in various mesh test cases.

Triangular Mesh Test Case	# of Vertices	# of Elements	% of Elements That Are IE	% of Vertices That Have A Low Valence	% of Vertices That Have A High Valence	Avg. Number of IE Attachments	
						Low-Valence Vertices	High-Valence Vertices
Test Case 1	2806	5312	49.9	82.8	6.2	0.22	3.81
Test Case 2	2716	4978	48.0	81.2	2.2	0.08	3.62
Test Case 3	13005	24417	49.2	79.7	8.0	0.30	3.78
Test Case 4	13266	24910	48.8	81.1	6.6	0.24	3.81
Test Case 5	14449	26697	48.4	79.4	5.2	0.19	3.73

Bibliography

[1] CG Armstrong, DJ Robinson, RM McKeag, TS Li, SJ Bridgett, and RJ Donaghy. Applications of the medial axis transform in analysis modeling. In *NAFEMS, Proc. 5th Int. Conf. Reliability of FEM for Engineering Applications*, pages 415–426, 1995.

[2] Bala Balendran. A direct smoothing method for surface meshes. In *IMR*, pages 189–193, 1999.

[3] Andrea Balzano. Evaluation of methods for numerical simulation of wetting and drying in shallow water flow models. *Coastal Engineering*, 34(1):83–107, 1998.

[4] A. Bilgili, K. W. Smith, and D. R. Lynch. Battri: A two-dimensional bathymetry-based unstructured triangular grid generator for finite element circulation modeling. In *Computers and Geosciences, 32*, pages 632–642, 2006.

[5] Ted D Blacker and Michael B Stephenson. Paving: A new approach to automated quadrilateral mesh generation. *International Journal for Numerical Methods in Engineering*, 32(4):811–847, 1991.

[6] Houman Borouchaki and Pascal Frey. *Adaptive triangular-quadrilateral mesh generation*. PhD thesis, INRIA, 1996.

[7] Shintaro Bunya, Ethan J Kubatko, Joannes J Westerink, and Clint Dawson. A wetting and drying treatment for the runge–kutta discontinuous galerkin solution to the shallow water equations. *Computer Methods in Applied Mechanics and Engineering*, 198(17):1548–1562, 2009.

[8] Colton J Conroy, Ethan J Kubatko, and Dustin W West. Admesh+: An advanced, automatic unstructured mesh generator for shallow water models. *Ocean Dynamics*, 62(10-12):1503–1517, 2012.

[9] Coltonl Conroy. Admesh: An advanced mesh generator for hydrodynamic models, a thesis. 2010.

[10] Clint Dawson, Ethan J Kubatko, Joannes J Westerink, Corey Trahan, Christopher Mirabito, Craig Michoski, and Nishant Panda. Discontinuous galerkin methods for modeling hurricane storm surge. *Advances in Water Resources*, 34(9):1165–1176, 2011.

[11] Jack Edmonds. Maximum matching and a polyhedron with 0, 1-vertices. *Journal of Research of the National Bureau of Standards B*, 69(125-130):55–56, 1965.

[12] Francine Evans, Steven Skiena, and Amitabh Varshney. Optimizing triangle strips for fast rendering. In *Proceedings of the 7th conference on Visualization'96*, pages 319–326. IEEE Computer Society Press, 1996.

[13] SC Hagen, JJ Westerink, and RL Kolar. One-dimensional finite element grids based on a localized truncation error analysis. *International Journal for Numerical Methods in Fluids*, 32(2):241–261, 2000.

[14] SC Hagen, JJ Westerink, RL Kolar, and O Horstmann. Two-dimensional, unstructured mesh generation for tidal models. *International Journal for Numerical Methods in Fluids*, 35(6):669–686, 2001.

[15] Scott C Hagen, Olaf Horstmann, and Robert J Bennett. An unstructured mesh generation algorithm for shallow water modeling. *International Journal of Computational Fluid Dynamics*, 16(2):83–91, 2002.

[16] Amaury Johnen, Damien Ernst, and Christophe Geuzaine. Sequential decision-making approach for quadrangular mesh generation. *Engineering with Computers*, 31(4):729–735, 2015.

[17] C Krn Lee and SH Lo. A new scheme for the generation of a graded quadrilateral mesh. *Computers & structures*, 52(5):847–857, 1994.

[18] Der-Tsai Lee and Bruce J Schachter. Two algorithms for constructing a delaunay triangulation. *International Journal of Parallel Programming*, 9(3):219–242, 1980.

[19] Yongcai Liu, Wenliang Chen, and Yidong Bao. An efficient paving method of pure quad mesh generation. *DEStech Transactions on Materials Science and Engineering*, (icimm), 2016.

[20] SH Lo. Generating quadrilateral elements on plane and over curved surfaces. *Computers & structures*, 31(3):421–426, 1989.

[21] A. L. Maggi. Discontinuous galerkin methods for shallow water flow: developing a computational infrastructure for mixed element meshes, a thesis. 2011.

[22] Stéphane Maza, Frédéric Noël, and Jean-Claude Léon. Generation of quadrilateral meshes on free-form surfaces. *Computers & structures*, 71(5):505–524, 1999.

[23] Scott A Mitchell, Mohammed A Mohammed, Ahmed H Mahmoud, and Mohamed S Ebeida. Delaunay quadrangulation by two-coloring vertices. *Procedia Engineering*, 82:364–376, 2014.

[24] C Navarro, Nancy Hitschfeld-Kahler, and Eliana Scheihing. A parallel gpu-based algorithm for delaunay edge-flips. In *The 27th European Workshop on Computational Geometry, EuroCG*, volume 11, 2011.

[25] Steven J Owen. A survey of unstructured mesh generation technology. In *IMR*, pages 239–267, 1998.

[26] Steven J Owen, Matthew L Staten, Scott A Canann, and Sunil Saigal. Q-morph: an indirect approach to advancing front quad meshing. *International Journal for Numerical Methods in Engineering*, 44(9):1317–1340, 1999.

[27] O Persson. Distmesh-a simple mesh generator in matlab, 2012. *URL http://persson. berkeley. edu/distmesh/.[Online*, 2015.

[28] J-F Remacle, Jonathan Lambrechts, Bruno Seny, Emilie Marchandise, Amaury Johnen, and C Geuzainet. Blossom-quad: A non-uniform quadrilateral mesh generator using a minimum-cost perfect-matching algorithm. *International journal for numerical methods in engineering*, 89(9):1102–1119, 2012.

[29] Rachel Sebian. New transport capabilities and timesteppers for a discontinuous galerkin wave model, a thesis. 2016.

[30] William T Tutte. A family of cubical graphs. In *Mathematical Proceedings of the Cambridge Philosophical Society*, volume 43, pages 459–474. Cambridge Univ Press, 1947.

[31] Dustin W West. A multidimensional discontinuous galerkin modeling framework for overland flow and channel routing, a thesis. 2015.

[32] D Wirasaet, Ethan J Kubatko, CE Michoski, S Tanaka, JJ Westerink, and C Dawson. Discontinuous galerkin methods with nodal and hybrid modal/nodal triangular, quadrilateral, and polygonal elements for nonlinear shallow water flow. *Computer methods in applied mechanics and engineering*, 270:113–149, 2014.

[33] Tian Zhou and Kenji Shimada. An angle-based approach to two-dimensional mesh smoothing. In *IMR*, pages 373–384, 2000.

[34] JZ Zhu, OC Zienkiewicz, E Hinton, and J Wu. A new approach to the development of automatic quadrilateral mesh generation. *International Journal for Numerical Methods in Engineering*, 32(4):849–866, 1991.

[35] Alan K Zundel and Norman L Jones. An integrated surface water modeling system. In *Hydroinformatics*, volume 1, pages 231–237, 1996.